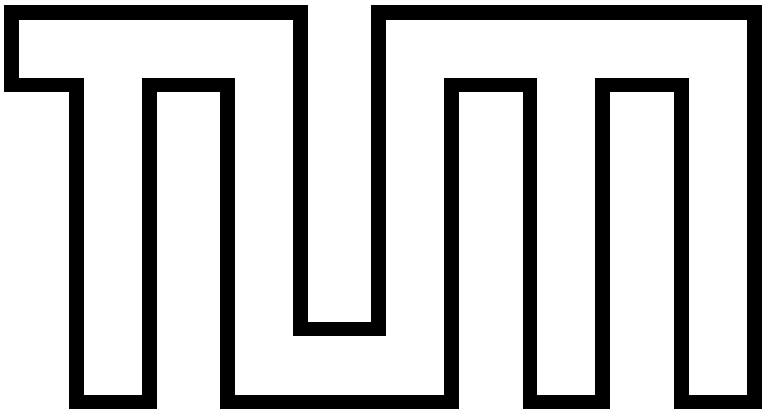


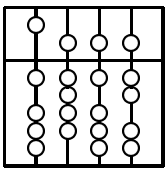
TECHNISCHE UNIVERSITÄT MÜNCHEN

FAKULTÄT FÜR INFORMATIK

Diplomarbeit

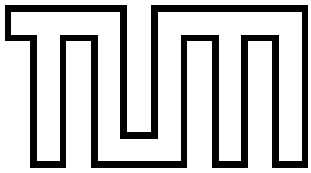
A Verified Unification Algorithm for Higher-Order Patterns

Bernd Grobauer



FAKULTÄT FÜR INFORMATIK

DER TECHNISCHEN
UNIVERSITÄT MÜNCHEN



Forschungs- und Lehreinheit IV
Programmiermethodik, Verteilte Systeme

Diplomarbeit

A Verified Unification Algorithm for Higher-Order Patterns

Bernd Grobauer*

Aufgabensteller:	Prof. Tobias Nipkow
Betreuer:	Prof. Tobias Nipkow
Abgabedatum:	15. Mai 1997

*E-mail: grobauer@informatik.tu-muenchen.de

Ich versichere, daß ich diese Diplomarbeit selbständig verfaßt und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

München, den 15. Mai 1997

Bernd Grobauer

Abstract

Applications like theorem provers and languages based on the typed lambda calculus use unification of typed lambda terms as basic interference mechanism. The general case has been treated by Gérard Huet [3], who provided an algorithm on which virtually every implementation of unification for typed lambda terms is based. Unlike the general case the unification of higher-order patterns, a subclass of lambda terms discovered by Dale Miller (see [5]), is decideable and has general unifiers that are easy to compute. Various theorem provers like Isabelle [11] now use unification of higher-order patterns.

The development of a unification algorithm for higher-order patterns by program transformation has been carried out by Tobias Nipkow [7]. The author starts out with an abstract specification of the algorithm based on rewrite rules. This algorithm is transformed to an ML-program in several steps. Proof sketches are given to show the correctness of the initial algorithm and the validity of each transformation step.

This masters thesis presents a similar development in a more formal way. We start with the specification of a unification algorithm for higher-order patterns represented in de Bruijn-notation. The specification is proved correct in a rigorous way — several facts from the theory of the lambda calculus are used without further proof. In four steps the algorithm is developed into a form very close to a functional program in Haskell or ML. The first two transformation steps only introduce a small structural change and a simple refinement of the original specification; their correctness is proved quite easily. Step three and step four which increase the efficiency of the algorithm require rather complex proofs.

The presentation of the transformation is a rigorous one: Certain isomorphisms and invariants are used without formally proving them. Also the treatment of an impure feature of the algorithm, the introduction of new variable names, is rather informal.

The original intention to do significant parts of the proof in Isabelle could not be carried out: Since the rigorous presentation is quite complex, it is not feasible to base a strictly formal development on it. The rigorous approach however did prove to be advantageous: Some errors that were detected in the development almost certainly would have remained unnoticed with a less formal approach.

Contents

1 Preliminaries **3**
1.1 Lambda Calculus . . . 3
1.2 Algorithms . . . 6
1.3 Auxiliary functions . . . 7

2 The unification algorithm — presentation and proof of correctness **9**
2.1 The algorithm . . . 9
2.2 Proof of correctness . . . 10

3 A small structural change **21**
3.1 Definition of *unif_2* . . . 21
3.2 Proof of correctness . . . 21

4 A refinement step **23**
4.1 Definition of *cases_3* . . . 23
4.2 Proof of correctness . . . 23

5 An optimization for flex-rigid-pairs **25**
5.1 Definition of *cases_4* . . . 25
5.2 Proof of correctness . . . 26

6 Introducing partial substitution **38**
6.1 Representation of substitutions . . . 38
6.2 Partial substitution and β-reduction by *devar* . . . 39
6.3 Definition of algorithm with partial substitution . . . 45
6.4 Proof of correctness . . . 46
6.5 Some thoughts about an implementation of *unif_5* . . . 51

A Lambda terms **52**
A.1 A theory of lambda terms . . . 52
A.2 Isomorphisms between *lterm_b* and *lterm_f* . . . 58

Chapter 1

Preliminaries

1.1 Lambda Calculus

1.1.1 Two representations for lambda terms

Lambda terms often are represented in de Bruijn-notation. There are no names for bound variables, instead a bound variable is represented by a natural number n, refering to the $(n+1)$-th lambda "above" the variable with respect to the syntax tree. If there is no such $(n+1)$-th lambda, the bound variable is called *loose*.

A possible datatype of lambda terms with free variables is

$$\begin{aligned}\textbf{datatype}\ \mathit{lterm}_b = {} & \mathsf{BVar}\ \mathit{nat} \\ | {} & \mathsf{Var}\ \mathit{nat} \\ | {} & \lambda\ \mathit{lterm}_b \\ | {} & \mathit{lterm}_b\ @\ \mathit{lterm}_b\end{aligned}$$

Here the free variables are represented by natural numbers. A theory about lambda terms based on this representation is presented in appendix A.1.

The subscript b in lterm_b hints at the binary composition '@'. Another form to represent lambda terms uses the multiple composition $F(\!|[s_{n-1}, \ldots, s_0]|\!)$ (the *functor-arguments form*) which stands for

$$(\cdots((F\ @\ s_{n-1})\ @\ s_{n-2})\cdots\ @\ s_0)$$

in lterm_b notation. The datatype of lambda terms built with the functor-arguments form instead of a binary application will be called lterm_f. Implicitly it is assumed that a functor-arguments form does not itself occur as functor and that a term with a leading lambda is not functor to an empty list of arguments (a term t is said to have a *leading lambda* if it is of the form $\lambda\ t$). All free and bound variables are functors, i.e. they always have a (possibly) empty list of arguments. This assumption is helpful for proofs by induction over the term structure, since bound and free variables can be subsumed under the functor-arguments form. The first functor-arguments form in a term t "under" a row of leading lambdas is called the *top-level* functor-arguments form of t.

In appendix A.2 we show that lterm_f and lterm_b are isomorphic. Both lterm_b and lterm_f will be used in the sequel; we can allow ourselves this freedom of expression, since the isomorphisms mentioned above give a transformation of any function or relation dealing with lterm_b into a function/relation dealing with lterm_f and vice-versa. Thus the theory of lambda terms based on the representation lterm_b gives rise to a theory based on lterm_f.

For large parts of the development lterm_f will prove to be more adequate. However in some cases the use of lterm_b simplifies formulation and proof. Besides, lterm_b is more commonly used.

1.1.2 Notation for lambda terms

The letters 'r', 's' and 't' will always denote lambda terms. Inside a lambda term the letters 'p', 'q', 'x', 'y' and 'z' stand for bound variables, 'D', 'E', 'F', 'G' and 'H' for free variables. The letters 'f', 'g' and 'h' denote either a bound variable or a free variable.

For (BVar i) we often write '$\underline{i}$'. The meaning of '$x + n$', where x is a bound variable $\underline{i}$ and n a natural number, is $\underline{i+n}$. Multiple lambdas like in '$\lambda\lambda \ldots \lambda\ t$' are abbreviated as '$\lambda_m\ t$', where m is the number of lambdas. If t is of the form $f(\!|\overline{s_n}|\!)$ then '$t(\!|\overline{r_n}|\!)$' denotes $f(\!|\overline{s_n} +\!\!+ \overline{r_n}|\!)$ (remember that a functor-arguments form is not supposed to occur as functor).

The free variables in a term t are denoted by '$\mathcal{FV}(t)$'.

1.1.3 Substitutions

Substitution

Any function ξ from free variables to lambda terms (without loose bound variables) determines a unique substitution *mk_subst* ξ:

$$\begin{aligned} &\mathit{mk_subst}\ \xi\ t = \textbf{case}\ t\ \textbf{of} \\ &\qquad \mid F \to (\xi F) \\ &\qquad \mid a \to a \\ &\qquad \mid \lambda\ t' \to \lambda\ (\mathit{mk_subst}\ \xi\ t') \\ &\qquad \mid r\ @\ s \to (\mathit{mk_subst}\ \xi\ r)\ @\ (\mathit{mk_subst}\ \xi\ s) \end{aligned}$$

It is easy to see that a substitution will map lambda terms without loose bound variables to lambda terms without loose bound variables.

Substitutions are denoted with greek letters. The restriction of a substitution σ to a set of variables V, written '$\sigma\restriction_V$' is defined in terms of its underlying function: if ξ is σ's underlying function from free variables to lambda terms, then $\sigma\restriction_V := \mathit{mk_subst}\ (\xi\restriction_V)$.

The most simple substitution, apart from the empty substitution $\emptyset$, only instantiates a single variable F with some term t, leaving all other variables unchanged; we call such a substitution a binding and write '$\{F{\mapsto}t\}$'. A binding can be represented by a pair $\langle F, t\rangle$ which is turned into a function from free variables to terms by *mk_bind*:

$$\begin{aligned} &\mathit{mk_bind}\ \langle F,t\rangle\ G = \textbf{if}\ (F = G)\ \textbf{then}\ t \\ &\qquad\qquad\qquad\qquad\qquad\quad \textbf{else}\ G \end{aligned}$$

So $\{F{\mapsto}t\} = \mathit{mk_subst}\ (\mathit{mk_bind}\ \langle F, t\rangle)$.

The set of free variables which are changed by a substitution σ is denoted with '$\mathcal{DOM}(\sigma)$'. We define the range of a substitution σ with respect to a set of variables V as $\mathcal{RNG}\langle\sigma, V\rangle := \bigcup_{F\in V} \mathcal{FV}(\sigma F)$. If V is the domain of σ, we simply write '$\mathcal{RNG}(\sigma)$'. So $\mathcal{RNG}\langle\sigma, V\rangle$ is equivalent to $\mathcal{RNG}(\sigma\restriction_V)$.

Composition '$\circ$' on substitutions is the usual functional composition.

Most of the substitutions in the sequel will have a special form:

Definition 1. *A substitution φ is called a* constructed substitution*, if there are free variables $G_{n-1}, \ldots, G_0$ with $G_i \neq G_j$ if $i \neq j$ and terms $r_{n-1}, \ldots, r_0$ in β-normal form such that*

$$\varphi = \{G_{n-1}{\mapsto}r_{n-1}\} \circ \ldots \circ \{G_0{\mapsto}r_0\}$$

and for all $0 \leqslant i \leqslant (n-1)$ it is $G_i \notin \mathcal{FV}(r_{n-1} \ldots r_i)$.

We further define the set of free variables of such a constructed substitution φ as

$$\mathcal{FV}(\varphi) = \{G_{n-1}, \ldots, G_0\} \cup \bigcup_{i=0}^{n-1} \mathcal{FV}(r_i)$$

An important lemma about constructed substitutions is the following:

Lemma 1. *A constructed substituion φ is idempotent.*

This follows immediately from

Lemma 2. *For a constructed substitution φ it is $\mathcal{DOM}(\varphi) \cap \mathcal{RNG}(\varphi) = \emptyset$*

Proof. Let φ be of the form $\{G_{n-1} \mapsto r_{n-1}\} \circ \ldots \circ \{G_0 \mapsto r_0\}$ for pairwise distinct free variables G_i, and $G_i \notin \mathcal{FV}(r_{n-1}, \ldots, r_i)$ for $0 \leqslant i \leqslant n-1$.

Suppose that $F \in \mathcal{DOM}(\varphi)$. Since the variables G_i are pairwise distinct, there is exactely one j with $F = G_j$.

Suppose now that $F \in \mathcal{RNG}(\varphi)$. Then there must be a k with $F \in \mathcal{FV}(r_k)$ and

$$F \in \mathcal{FV}(\{G_{n-1} \mapsto r_{n-1}\} \circ \ldots \circ \{G_{k+1} \mapsto r_{k+1}\} r_k) \qquad (*)$$

But $G_j \notin \mathcal{FV}(r_{n-1}, \ldots, r_j)$, so $k < j$. We can write

$$\begin{aligned}(\{G_{n-1} \mapsto r_{n-1}\} \circ \ldots \circ \{G_{k+1} \mapsto r_{k+1}\}) r_k = \\ (\{G_{n-1} \mapsto r_{n-1}\} \circ \ldots \circ \{F \mapsto r_j\})(\{G_{j-1} \mapsto r_{j-1}\} \circ \ldots \circ \{G_{k+1} \mapsto r_{k+1}\} r_k)\end{aligned}$$

All occurences of F will be removed from $(\{G_{j-1} \mapsto r_{j-1}\} \circ \ldots \circ \{G_{k+1} \mapsto r_{k+1}\}) r_k$ by $\{F \mapsto r_j\}$. But since $F \notin \mathcal{FV}(r_{n-1}, \ldots, r_j)$, no new occurrences will be introduced. So obviously

$$F \notin \mathcal{FV}(\{G_{n-1} \mapsto r_{n-1}\} \circ \ldots \circ \{G_{k+1} \mapsto r_{k+1}\} r_k)$$

which is a contradiction to $(*)$. Thus $F \notin \mathcal{RNG}(\varphi)$.

□

In the sequel 'σ', 'γ', 'ρ' and 'δ' will denote a substitution of any form, all other greek letters will denote substitutions that are either implicitely assumed to be constructed substitutions or can be shown to be constructed substitutions. In particular it can be shown for all the presented algorithms that only constructed substitutions are created.

1.1.4 Equivalence of lambda terms and substitutions

Higher-order unification of two terms s and t is the task to find a substitution σ such that $(\sigma s) \equiv (\sigma t)$, where '$\equiv$' denotes the reflexive, symmetric and transitive closure of $(\beta \cup \eta)$. We will call a pair of terms s and t which are to be unified a *unification problem* and write '$s \stackrel{?}{=} t$'. The letters u and v will always denote unification problems.

Equivalence extends to substitutions:

Definition 2.
Two substitutions σ and γ are said to be equivalent with respect to a set of variables $\mathcal{F}$, if

$$\forall F \in \mathcal{F}.\ \sigma F \equiv \gamma F$$

We use the notion of equivalence between substitutions in the following definition of a most general unifier:

Definition 3.
A substitution σ is a most general unifier of a unification problem u, if σ unifies u and for every unifier γ of u there is a substitution ρ such that $\rho \circ \sigma \equiv \gamma$ with respect to $\mathcal{FV}(u)$.

A useful concept when dealing with substitutions is a *renaming*:

Definition 4.
A substitution δ induced by a bijective function from free variables to free variables is called a renaming. We say that δ is a renaming from σ to γ if

$$(\delta \circ \sigma) \equiv (\gamma \circ \delta) \textit{with respect to all variables}$$

1.1.5 Higher-order patterns

A higher-order pattern is defined as follows:

Definition 5.
A term t in β-normal form is a higher-order pattern *if every free occurence of a variable F is in a subterm $F(\!|rrs|\!)$ of t such that rrs is η-equivalent to a list of distinct bound variables.*

In the sequel all lambda terms are either implicitly assumed to be patterns or are patterns by construction. To simplify notation we furthermore assume that all arguments to free variables are bound variables (not only η-equivalent to bound variables). Of course this requirement can be relaxed, e.g. by a preprocessing phase.

Higher-order patterns always have a β-normal form; we write '$(t)\downarrow_\beta$' to denote the β-normal form of t. Using fact 8 on page 57 one shows easily:

Fact 1. $((\sigma_{n-1} \circ \cdots \circ \sigma_0)t)\downarrow_\beta = (\sigma_{n-1}(\cdots(\sigma_0 t)\downarrow_\beta \cdots)\downarrow_\beta)\downarrow_\beta$

With respect to unification of higher-order patterns one shows also with Fact 8 on page 57:

Fact 2. *σ unifies a unification problem u iff σ unifies $(u)\downarrow_\beta$*

Since higher-order patterns in β-normal form never have free variables in argument position to a free variable, the following fact holds for constructed substitutions:

Fact 3. *For a (constructed) substitution φ and a higher-order pattern t in β-normal form, it is $\mathcal{FV}(\varphi t) = \mathcal{FV}((\varphi t)\downarrow_\beta)$*

1.2 Algorithms

1.1.1 Notational aspects

All algorithms are written in a mixture of ML and mathematics. Of ML we mainly use basic combinators over lists and the *maybe*-datatype

datatype *maybe* $\alpha =$ None $\mid \llcorner\alpha\lrcorner$
x_m**:**$[x_{m-1}, \ldots, x_0] = [x_m, x_{m-1}, \ldots, x_0]$
$[x_{m-1}, \ldots, x_0] +\!\!+ [y_{n-1}, \ldots, y_0] = [x_{m-1}, \ldots, x_0, y_{n-1}, \ldots, y_0]$
map f $[x_{m-1}, \ldots, x_0] = [fx_{m-1}, \ldots, fx_0]$

foldl f a (x**:**xs) $=$ *foldl* f (f a x) xs
foldl f a [] $= a$

In order to keep the notation less clumsy, obvious mapping functions (e.g. to lists or pairs) will not be written explicitly. As in Haskell, lists can be defined by list comprehension.

A list of m elements will often be written as $\overline{x_m}$ with

$$\overline{x_m} = [x_{m-1}, x_{m-2} \ldots, x_0]$$

Note that lists are indexed **in reverse order** starting with 0! This eases notation mainly when dealing with de Bruijn-notation. A bound variable $\underline{i}$ when used as index denotes i.

A list x_m of variables can be turned into a set of its elements denoted by '$\{\overline{x_m}\}$'. Dealing with higher-order patterns, where a list of arguments to a free variable F is required to be a list $\overline{x_m}$ of *distinct* bound variables, obviously it is $|\overline{x_m}| = |\{\overline{x_m}\}|$.

So called *Don't-care-elements* which are used for pattern matching in functional languages are denoted with the underscore '_'.

1.2.2 Treatment of new variables

The unification algorithm introduces new free variables. In a functional program this is realized by carrying along some parameter containing information about the free variables already used (e.g. the very set of all used variables). New variables are created by a function reading the parameter and calculating a new variable name. In the development of the algorithm there will be no explicit notation of this parameter (which is consistent with a monadic style of programming in Haskell (see for example [13, 4]), where such a parameter would be hidden in a state-monad). Often it suffices to consider in which context a new variable H was introduced: Obviously H is distinct from any variable already in use in that context. In some proofs however it is necessary to reason about the hidden parameter mentioned above.

1.3 Auxiliary functions

The following auxiliary functions will be used in the development of the algorithm.

1.3.1 Function *lookup*

lookup xxs y searches an association list xxs for a certain element y. In case of success, it not only returns the value associated with y, but also the rest of the association list that has not been examined:

$$
\begin{array}{l}
\mathit{lookup}\ xxs\ y = \textbf{case}\ xxs\ \textbf{of} \\
\qquad\qquad \mid [] \to \mathsf{None} \\
\qquad\qquad \mid \langle z, t\rangle{:}xs \to \textbf{if}\ (y = z) \\
\qquad\qquad\qquad\qquad\qquad \textbf{then}\ \llcorner\langle t, xs\rangle\lrcorner \\
\qquad\qquad\qquad\qquad\qquad\ \textbf{else}\ \mathit{lookup}\ xs\ y
\end{array}
$$

1.3.2 Function *idx*

The function *idx* takes a list and an element. If that element is in the list, the index of the element's first occurence is returned. Otherwise the result is None.

$$
\begin{array}{l}
\mathit{idx}\ xxs\ y = \textbf{case}\ xxs\ \textbf{of} \\
\qquad\qquad \mid [] \to \mathsf{None} \\
\qquad\qquad \mid x{:}xs \to \textbf{if}\ x = y\ \textbf{then}\ \llcorner|xs|\lrcorner \\
\qquad\qquad\qquad\qquad\qquad\ \textbf{else}\ \mathit{idx}\ xs\ y
\end{array}
$$

The function *idx* is characterized by the following two lemmas.

Lemma 3. *If all elements of* $\overline{x_m}$ *are pairwise distinct then* $\mathit{idx}\ \overline{x_m}\ x_i = i$

Lemma 4. $\mathit{idx}\ \overline{x_m}\ y = \mathsf{None} \Leftrightarrow y \notin \{\overline{x_m}\}$

Both lemmas are shown by induction over $\overline{x_m}$. Since Lemma 4 is easy to see, we only show Lemma 3

Proof. induction over m

$m = 1 \Rightarrow \overline{x_m} = [x_0], i = 0$
$\mathit{idx}\ [x_0]\ x_0 = |[]| = 0$

$m \longrightarrow m+1$ $\mathit{idx}\ \overline{x_{m+1}}\ x_i = \mathit{idx}\ [x_m, \ldots, x_0]\ x_i =$
case distinction over i

$i = m$ $= |\overline{x_m}| = m$

$i < m$ because $x_m \neq x_i$ since the elements in $\overline{x_{m+1}}$ are pairwise distinct, we have
$= \mathit{idx}\ \overline{x_m}\ x_i = i$ by induction hypothesis

□

1.3.3 *head_ap*: Application to a list's head

The function *head_ap* f xxs applies f to the head of xxs if xxs is not empty.

$$\begin{aligned} \mathit{head_ap}\ f\ xxs = {} & \textbf{case}\ xxs\ \textbf{of} \\ & \mid [] \to [] \\ & \mid x{:}xs \to (f\,x){:}xs \end{aligned}$$

Chapter 2

The unification algorithm — presentation and proof of correctness

2.1 The algorithm

The following unification algorithm is a modification of the algorithm presented in [7].

2.1.1 Reducing a single unification problem

The function $cases_1$ will try to solve a single unification problem. It is used together with the functions $otf\eta$ and tw. The first removes all leading lambdas of a unification problem, the latter orients it such that if a free variable occurs on only one of the unifcation problem's sides as top-level functor, then it occurs on the lhs.

Because a unification problem is brought into a special form by $otf\eta$ and tw, one can distinguish four possibly solvable cases: In the cases (flex-flex-1) and (flex-flex-2) (see definition of $cases_1$ below) the unification problem can be solved right away, in the cases (rigid-rigid) and (flex-rigid) new unification problems are generated. If none of the four cases matches a unification problem, the result is None.

Note that any variable H or H_i occuring in the definition is assumed to be a *new* variable!

$$cases_1\ (p(\overline{s_n}) \stackrel{?}{=} p(\overline{t_n})) = \llcorner\langle[s_{n-1} \stackrel{?}{=} t_{n-1}, \ldots, s_0 \stackrel{?}{=} t_0], \emptyset\rangle\lrcorner \qquad \text{(rigid-rigid)}$$

$$cases_1\ (F(\overline{x_m}) \stackrel{?}{=} p(\overline{s_n})) = \qquad \text{(flex-rigid)}$$

$$= \textbf{if } F \in \mathcal{FV}(\overline{s_n}) \textbf{ then } \mathsf{None} \textbf{ else if } p \notin \{\overline{x_m}\} \textbf{ then } \mathsf{None} \textbf{ else } \llcorner\langle[H_{n-1}(\overline{x_m}) \stackrel{?}{=} s_{n-1}, \ldots, H_0(\overline{x_m}) \stackrel{?}{=} s_0], \{F \mapsto \lambda_m\ (\underline{j})(\overline{H_n(zs)})\}\rangle\lrcorner$$

$$\textbf{where } zs = [\underline{m-1}, \ldots, \underline{0}]$$

$$x_j = p$$

$$cases_1\ (F(\overline{x_m}) \stackrel{?}{=} F(\overline{y_m})) = \llcorner\langle[], \{F \mapsto \lambda_m\ H(zs)\}\rangle\lrcorner \qquad \text{(flex-flex-1)}$$

$$\textbf{where } zs = [\underline{k} \mid 0 \leqslant k \leqslant (m-1), x_k = y_k]$$

$$cases_1\ (F(\overline{x_m}) \stackrel{?}{=} G(\overline{y_n})) = \llcorner\langle[], \{F \mapsto \lambda_m\ H(xs)\} \circ \{G \mapsto \lambda_n\ H(ys)\}\rangle\lrcorner \qquad \text{(flex-flex-2)}$$

$$\textbf{where } |xs| = |\{xs\}| = |ys| = |\{ys\}| = |\{\overline{x_m}\} \cap \{\overline{y_m}\}|,$$

$$x_{xs_i} = y_{ys_i} \text{for } 0 \leqslant i \leqslant (|xs| - 1)$$

2.1.2 On-the-fly eta-conversion

Leading lambdas on both sides of a unification problem have to be removed before $cases_1$ is employed. This is achieved with $otf\eta$. If there is a leading lambda on both sides of the unification problem, it is simply discarded. If there is a lambda only on one side, $otf\eta$ "generates" a lambda on the other side by η-conversion and then discards both lambdas.

We note that if a unification problem u is in β-normal form, then $(otf\eta\ u)$ is in β-normal form.

$$
\begin{aligned}
otf\eta\ (s \stackrel{?}{=} t) = \mathbf{case}\ (s \stackrel{?}{=} t)\ \mathbf{of}\\
&\mid (\lambda s' \stackrel{?}{=} \lambda t') \to otf\eta\ (s' \stackrel{?}{=} t')\\
&\mid (\lambda s' \stackrel{?}{=} t) \to otf\eta\ (s' \stackrel{?}{=} (\mathit{lift}\ t\ 0)\ @\ (\underline{0}))\\
&\mid (s \stackrel{?}{=} \lambda t') \to otf\eta\ ((\mathit{lift}\ s\ 0)\ @\ (\underline{0}) \stackrel{?}{=} t')\\
&\mid \mathbf{otherwise} \to s \stackrel{?}{=} t
\end{aligned}
$$

2.1.3 Orienting a unification problem

Since $cases_1$ only has a branch for (flex-rigid)-pairs, but none for "(rigid-flex)"-pairs, the unification problems have to be oriented.

$$
\begin{aligned}
tw(s \stackrel{?}{=} t) = \mathbf{case}\ s\ \mathbf{of}\\
&\mid F(\llbracket _ \rrbracket) \to (s \stackrel{?}{=} t)\\
&\mid \mathbf{otherwise} \to (t \stackrel{?}{=} s)
\end{aligned}
$$

2.1.4 Solving a list of unification problems

The function $unif_1$ carries out the unification of a list of unification problems: The functions $otf\eta$, tw and $cases_1$ are applied to the first unification problem in the list. The result is either None or another list of unification problems and a substitution. $unif_1$ applies the substitution to the remaining unification problems and calls itself recursively over the total list of unifcation problems. The resulting substitution is kept in the second component of the argument to $unif_1$ and returned in case of an empty list.

$$
\begin{aligned}
unif_1\ \langle uus, \varphi\rangle = \mathbf{case}\ uus\ \mathbf{of}\\
&\mid [] \to \llcorner\varphi\lrcorner\\
&\mid u{:}us \to \mathbf{case}\ (cases_1 \circ tw \circ otf\eta)\ u\ \mathbf{of}\\
&\qquad \mid \mathsf{None} \to \mathsf{None}\\
&\qquad \mid \llcorner\langle us', \varphi'\rangle\lrcorner \to unif_1\ \langle us' +\!\!+ (\varphi' us)\downarrow_\beta, \varphi' \circ \varphi\rangle
\end{aligned}
$$

2.2 Proof of correctness

2.2.1 Function $unif_1$ terminates

The following rather informal argument shows the termination of $unif_1$ by defining a suitable termination order. Once the termination of $unif_1$ has been shown, one can carry out proofs about $unif_1$ by computational induction.

Proof. We prove the termination of $unif_1$:

Suppose that in case (flex-rigid) the list of new unification problems has as last element a special marker symbol '•'. A possible result of $cases_1$ in the case (flex-rigid) would look like this:

$$\llcorner\langle [H_{n-1}(\llbracket\overline{x_m}\rrbracket) \stackrel{?}{=} s_{n-1}, \ldots, H_0(\llbracket\overline{x_m}\rrbracket) \stackrel{?}{=} s_0] +\!\!+ [\bullet], \{F \mapsto \lambda_m\ q(\llbracket\overline{H_n(\llbracket zs\rrbracket)}\rrbracket)\}\rangle\lrcorner$$

The markers are only needed for reasoning about a list of unification problems — in the calculation a marker is simply discarded as soon as it becomes the list's first element. The following observations can be made:

(a) there is never a free variable which was introduced in a (flex-rigid) case on a right hand side of a unification problem *in front of* the last marker

(b) if there is no marker present, there is no free variable which was introduced in a (flex-rigid) case in any unification problem of the list

(c) all the left hand sides of unifcation problems in front of the last marker are of the form '$H(\!|\overline{x_m}|\!)$' where H is a free variable that was introduced in a (flex-rigid) case.

(d) only cases (flex-rigid) and (flex-flex-2) can occur in front of the last marker

Observation (d) follows directly from observations (a) and (c).

With every list of unification problems a triple $\langle N, C_M, C\rangle$ is associated, where

N is the number of free variables of the list, *not* counting the variables introduced in (flex-rigid) cases.

C_m is the complexity of the right hand sides of all unification problems in front of the last marker (if there is no marker then $C_m = 0$)

C is the complexity of all the unification problems after the last marker (or of the total list, if there is no marker)

The *complexity* of a term is hereby defined as the number of functors in it.

Claim: the lexicographic order over $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$ constitutes a termination order for $unif_1$. This is shown by considering how the triple associated with a list of unification problems changes with each single step (= application of $cases_1$ to the first unification problem in the list).

case distinction over existence of a marker

there is at least one marker in the list Because of observation (d), only two cases have to be considered:

case distinction over branches in $cases_1$

(flex-rigid) New variables are introduced, but N does not change: the variable which is substituted (and thus "vanishes") is a variable which was introduced in a (flex-rigid) cases (observation (c)). Likewise all the new variables introduced by this substitution are introduced in a (flex-rigid) cases. But these variables are not counted in N.

The component C_m is decreased, since one functor vanishes. The substitution does not change the complexity in front of the last marker, because of observation (a).

(flex-flex-2) The number of free variables does not change: One variable which was not introduced by (flex-rigid) (observation (a)) vanishes, but a new one is introduced instead.

The component C_m is decreased, because one functor vanishes. The substitution does not change the complexity in front of the marker, because the number of functors is not affected by it: Suppose that a free variable F in some rhs in front of the marker is substituted with $\lambda_m\ H(\!|xs|\!)$. F always occurs as a functor to a (possibly empty) list of bound variables ys. If $m = 0$ or ys is empty then the result of the substitution is $H(\!|xs +\!\!+ ys|\!)$ or $\lambda_m\ H(\!|xs|\!)$ respectively. Obviously there is exactely one functor in this expression. Otherwise the β-reduction following the substitution will reduce the two functors in $(\lambda_m\ H(\!|xs|\!))(\!|ys|\!)$ to a single functor.

there is no marker in the list case distinction over branches in $cases_1$

(rigid-rigid) The number of free variables N does not change. However the overall complexity is reduced: two functors vanish.

(flex-rigid) The number of free variables N decreases by one: since there is no marker, definitely a variable that was not introduced by (flex-rigid) vanishes (see observation (b)). All the new variables introduced at this point are introduced by (flex-rigid) and therefore not counted in N.

(flex-flex-1) The number of variables does not change. However the overall complexity decreases because two functors vanish. The substitution does not change the complexity, because the number of funtors is not affected by it.

(flex-flex-2) The number of free variables decreases by one.

□

2.2.2 Analyzing *cases*$_1$

Unification really is done in *cases*$_1$; *unif*$_1$ only collects the result and feeds *cases*$_1$ with the subproblems created by an earlier call of *cases*$_1$. Thus the correctness proof will mostly be about *cases*$_1$ (however the necessity to keep track of used and new free variables complicates the final proof about *unif*$_1$ a bit). Before proving three basic propositions about *cases*$_1$ we have to consider the role of $otf\eta$. The following proposition makes sure that the use of $otf\eta$ does not affect the unification process.

Proposition 1. *A substitution σ unifies a unification problem u iff σ unifies $(otf\eta\ u)$*

Proof. We prove with computational induction over $otf\eta$.

case distinction over u

$u = (\lambda\ s \stackrel{?}{=} \lambda\ t)$ σ unifies $otf\eta\ (\lambda\ s \stackrel{?}{=} \lambda\ t)$

$\Leftrightarrow$ ⟨definition of $otf\eta$ on page 10⟩

σ unifies $otf\eta(s \stackrel{?}{=} t)$

$\Leftrightarrow$ ⟨induction hypothesis⟩

σ unifies $s \stackrel{?}{=} t \Leftrightarrow \sigma s \equiv \sigma t$

$\Leftrightarrow$ ⟨with Fact 5 on page 57⟩

$\lambda\ \sigma s \equiv \lambda\ \sigma t \Leftrightarrow \sigma(\lambda\ s) \equiv \sigma(\lambda\ t)$

$u = (\lambda\ s \stackrel{?}{=} t)$ σ unifies $otf\eta\ (\lambda\ s \stackrel{?}{=} t)$

$\Leftrightarrow$ ⟨definition of $otf\eta$ on page 10⟩

σ unifies $otf\eta\ (s \stackrel{?}{=} (lift\ t\ 0)\ @\ \underline{0})$

$\Leftrightarrow$ ⟨induction hypothesis⟩

σ unifies $(s \stackrel{?}{=} (lift\ t\ 0)\ @\ \underline{0}) \Leftrightarrow \sigma s \equiv \sigma((lift\ t\ 0)\ @\ \underline{0})$

$\Leftrightarrow$ ⟨with Fact 7 on page 57⟩

$\sigma s \equiv (lift\ \sigma t\ 0)\ @\ \underline{0}$

$\Leftrightarrow$ ⟨with Fact 6 on page 57⟩

$\lambda\ \sigma s \equiv \sigma t \Leftrightarrow \sigma(\lambda\ s) \equiv \sigma t$

$u = (s \stackrel{?}{=} \lambda\ t)$ like case above

any other form obvious, since $u = otf\eta\ u$

□

Especially in the following proof the use of de Bruijn-notation makes things quite complicated: Terms and substitutions have to be constructed which would be a lot easier with named bound variables.

Proposition 2. *Let $(cases_1 \circ tw \circ otf\eta)\ u = \llcorner\langle us, \varphi\rangle\lrcorner$ for a unification problem u in β-normal form and let σ be a unifier of u. If $\mathcal{DOM}(\sigma) \cap N = \mathcal{RNG}(\sigma) \cap N = \emptyset$ for the set N of new variables introduced in cases$_1$, then there is a substitution γ such that $\gamma \circ \varphi \equiv \sigma$ with respect to $\mathcal{DOM}(\sigma)$, and $\mathcal{RNG}(\gamma) \subseteq \mathcal{RNG}(\sigma)$.*

Proof. Be $u' = (tw \circ otf\eta)u$. Because of proposition 1 we know that σ unifies u iff σ unifies u'.

case distinction over form of u'

(rigid-rigid) clear, since empty substitution is returned

(flex-rigid) u' has the form $F(\overline{x_m}) \stackrel{?}{=} p(\overline{s_n})$, $\varphi = \{F \mapsto \lambda_m\ \underline{j}(\overline{H_n(zs)})\}$ with $x_j = p$, $zs = [\underline{m-1}, \ldots, \underline{0}]$. Suppose σ unifies u'. Then $(\sigma F)(\overline{x_m}) \equiv p(\overline{\sigma s_n})$. Because the relation $(\beta \cup \eta)$ is Church-Rosser and there are no β- or η-redexes "outside" the σs_i in $p(\overline{\sigma s_n})$ there is a term $p(\overline{s'_n})$ such that $(\sigma F)(\overline{x_m})\ (\beta \cup \eta)^*\ p(\overline{s'_n})$ and $p(\overline{\sigma s_n})\ (\beta \cup \eta)^*\ p(\overline{s'_n})$.

There must be a term t such that $(\sigma F) \stackrel{*}{\longleftarrow}_\eta \lambda_m\ t$ and $\lambda_m\ t$ has no loose bound variables. This is easy to see. If (σF) has m or more than m leading lambdas then take t such that $\lambda_m\ t = \sigma F$. Obviously (σF) has no loose bound variables, since substitutions only instantiate free variables with patterns without loose bound variables. If (σF) has less than m leading lambdas then η-expand (σF) until m leading lambdas are reached. η-conversion does not introduce loose bound variables.

Obviously $(\lambda_m\ t)(\overline{x_m})\ (\beta \cup \eta)^*\ p(\overline{s'_n})$. Since a β-reduction at the m top-level redexes does not introduce new β- or η-redexes (bound variables are substituted by bound variables) these β-reductions can be carried out as the very last reductions. Thus there is a term t' such that $t\ (\beta \cup \eta)^*\ t'$ and

$$(\lambda_m\ t)(\overline{x_m})\ (\beta \cup \eta)^*\ (\lambda_m\ t')(\overline{x_m}) \xrightarrow{m}_\beta p(\overline{s'_n})$$

With Lemma 15 on page 53:

$$(\lambda_m\ t')(\overline{x_m}) \xrightarrow{m}_\beta t'[x_m + (m-1) \backslash m-1] \ldots [x_0 \backslash 0]$$

and therefore we have

$$p(\overline{s'_n}) = t'[x_m + (m-1) \backslash m-1] \ldots [x_0 \backslash 0]$$

The chain of application traverses the structure of t'. So t' must be of the form $\underline{j}(\overline{t'_n})$ with

- $x_j = p$ because of Lemma 17 on page 54
- $t'_i[x_m + (m-1) \backslash m-1] \ldots [x_0 \backslash 0] = s'_i$ for $0 \leqslant i \leqslant (n-1)$

Since $(\lambda_m\ t)$ has no loose bound variables and β- and η-reduction do not introduce loose bound variables it is obvious that $\lambda_m\ t' = \lambda_m\ \underline{j}(\overline{t'_n})$ has no loose bound variables. It is easy to see that $\lambda_m\ t'_i$ also has no loose bound variables for $0 \leqslant i \leqslant (n-1)$. So with Lemma 21 on page 55:

$$(\lambda_m\ t'_i)(zs) \xrightarrow{m}_\beta t'_i$$

and therefore

$$\lambda_m\ t' = \lambda_m\ \underline{j}(\overline{t'_n}) \stackrel{*}{\longleftarrow}_\beta \lambda_m\ \underline{j}([(\lambda_m\ t'_{n-1})(zs), \ldots, (\lambda_m\ t'_0)(zs)])$$

Thus

$$\sigma F \equiv \lambda_m\ \underline{j}([(\lambda_m\ t'_{n-1})(zs), \ldots, (\lambda_m\ t'_0)(zs)]) \qquad (*)$$

Analyzing the way the equivalence between σF and $\lambda_m\ \underline{j}([(\lambda_m\ t'_{n-1})(zs), \ldots, (\lambda_m\ t'_0)(zs)])$ was shown, it is easy to see that no free variables were introduced. Therefore

$$\mathcal{FV}(\lambda_m\ \underline{j}([(\lambda_m\ t'_{n-1})(zs), \ldots, (\lambda_m\ t'_0)(zs)])) \subseteq \mathcal{FV}(\sigma F) \qquad (**)$$

Consider

$$\gamma := (\{H_{n-1} \mapsto \lambda_m\ t'_{n-1}\} \circ \ldots \circ \{H_0 \mapsto \lambda_m\ t'_0\} \circ (\sigma \restriction_{\mathcal{DOM}(\sigma) \setminus \{F\}}))$$

We claim that $\gamma \circ \varphi \equiv \sigma$ with respect to $\mathcal{DOM}(\sigma)$.

case distinction over elements of $\mathcal{DOM}(\sigma)$

case F $(\gamma \circ \varphi)F = \gamma(\lambda_m\ \underline{j}(\overline{H_n(zs)}))$

$= \langle$by assumption the variables H_i are not changed by $\sigma\restriction_{\mathcal{DOM}(\sigma)\setminus\{F\}}$, since they were introduced by $cases_1\rangle$
$(\{H_{n-1}\mapsto\lambda_m\ t'_{n-1}\}\circ\ldots\circ\{H_0\mapsto\lambda_m\ t'_0\})\ \lambda_m\ \underline{j}(\!|\overline{H_n(\!|zs|\!)}|\!)$
$= \langle$by assumption the variables H_i are not in $\mathcal{RNG}(\sigma)$; because of $(**)$ it follows that $H_i \notin \mathcal{FV}(\lambda_m\ t'_j)$ for $0 \leqslant i,j \leqslant (n-1)\rangle$
$\lambda_m\ \underline{j}([\!|(\lambda_m\ t'_{n-1})(\!|zs|\!),\ldots,(\lambda_m\ t'_0)(\!|zs|\!)]\!|)$
$\equiv \langle$with $(*)\rangle$
σF

case $G \neq F$ $(\gamma\circ\varphi)G = (\{H_{n-1}\mapsto\lambda_m\ t'_{n-1}\}\circ\ldots\circ\{H_0\mapsto\lambda_m\ t'_0\})\ (\sigma\restriction_{\mathcal{DOM}(\sigma)\setminus\{F\}}\ G)$
$= \langle$by assumption the variables H_i are not in $\mathcal{RNG}(\sigma\restriction_{\mathcal{DOM}(\sigma)\setminus\{F\}})$ and $H_i \neq G$ also by assumption since $G \in \mathcal{DOM}(\sigma)\rangle$
σG

Considering the definition of γ and $(**)$ it is easy to see that $\mathcal{RNG}(\gamma) \subseteq \mathcal{RNG}(\sigma)$.

(flex-flex-1)

u' has the form $F(\!|\overline{x_m}|\!) \stackrel{?}{=} F(\!|\overline{y_m}|\!)$, $\varphi = \{F\mapsto\lambda_m\ H(\!|zs|\!)\}$ with $zs = [\underline{k} \mid 0 \leqslant k \leqslant (m-1), x_k = y_k]$. Suppose that σ unifies u'. Then $(\sigma F)(\!|\overline{x_m}|\!) \equiv (\sigma F)(\!|\overline{y_m}|\!)$. The needed substitution γ will be of the form $\{H\mapsto s\}\circ\sigma\restriction_{\mathcal{DOM}(\sigma)\setminus\{F\}}\circ\varphi$. We are looking for a term s such that $\{H\mapsto s\}\circ\sigma\restriction_{\mathcal{DOM}(\sigma)\setminus\{F\}}\circ\varphi \equiv \sigma$ with respect to $\mathcal{DOM}(\sigma)$. With the same argument as in the previous case, this is equivalent to finding an s such that $(\{H\mapsto s\}\circ\varphi)F \equiv \sigma F$.

If there is no k such that $x_k = y_k$ then because of Fact 10 on page 57 we know that $\sigma F \equiv \lambda_m\ t$ and that t has no loose bound variables. So $\{H\mapsto t\}$ is a valid binding and it is $(\{H\mapsto t\}\circ\varphi)F = \{H\mapsto t\}(\lambda_m\ H) = \lambda_m\ t \equiv \sigma F$.

Suppose now that there is at least one k such that $x_k = y_k$.

Assumption: there are $\overline{z_m}$ and $\overline{z'_m}$ such that

$$(\lambda_{|zs|}\ (\sigma F)(\!|\overline{z_m}|\!))(\!|zs|\!) \xrightarrow{*}_\beta (\sigma F)(\!|\overline{z'_m}|\!) \quad \text{where } z'_i = \underline{i} \text{ if } x_i = y_i$$

Then consider $\{H\mapsto\lambda_{|zs|}\ ((\sigma F)(\!|\overline{z_m}|\!))\}$:
$(\{H\mapsto\lambda_{|zs|}\ ((\sigma F)(\!|\overline{z_m}|\!))\}\circ\varphi)F = \lambda_m\ ((\lambda_{|zs|}\ ((\sigma F)(\!|\overline{z_m}|\!)))(\!|zs|\!))$
$\xrightarrow{*}_\beta \langle$with the assumption$\rangle$
$\lambda_m\ ((\sigma F)(\!|\overline{z'_m}|\!))$
$\equiv \langle$From the assumption we know that $\overline{z'_m}$ is equal to $[\underline{m-1},\ldots,\underline{0}]$ in all positions i with $x_i = y_i$. Since $(\sigma F)(\!|\overline{x_m}|\!) \equiv (\sigma F)(\!|\overline{y_m}|\!)$, with Fact 12 on page 57$\rangle$
$\lambda_m\ ((\sigma F)(\!|[\underline{m-1},\ldots,\underline{0}]|\!))$
$\equiv \langle$which is η-equivalent to (σF) (Lemma 20 on page 54, (σF) does not have loose bound variables)$\rangle$
σF

We now show the assumption about about $\overline{z_m}$ and $\overline{z'_m}$ by giving a witness. Define

$$z_i = \begin{cases} \underline{j} & \exists j.zs_j = \underline{i} \\ \underline{0} & \text{otherwise} \end{cases}$$

With the definition of zs it is easy to see that no element occurs twice in zs. Therefore the definition above completely determines $\overline{z_m}$.

With Lemma 19 on page 54 we see that

$$(\lambda_{|zs|}\ (\sigma F)(\!|\overline{z_m}|\!))(\!|zs|\!) = (\sigma F)(\!|\overbrace{[zs_{z_{m-1}},\ldots,zs_{z_0}]}^{\overline{z'_m}:=}|\!)$$

Consider i with $0 \leqslant i \leqslant (m-1)$ and $x_i = y_i$.
$z'_i = zs_{z_i}$
$= \langle$by definition of $\overline{z_m}$ it is $z_i = \underline{j}$ with $zs_j = \underline{i}\rangle$

$\underline{i}$

So the assumption holds.

(flex-flex-2) u' has the form $F(\!|\overline{x_m}|\!) \stackrel{?}{=} G(\!|\overline{y_n}|\!)$, $\varphi = \{F \mapsto \lambda_m\ H(\!|xs|\!)\} \circ \{G \mapsto \lambda_n\ H(\!|ys|\!)\}$ with $|xs| = |ys| = |\{\overline{x_m}\} \cap \{\overline{y_n}\}|$, the elements in xs and ys are pairwise distinct and $x_{xs_i} = y_{ys_i}$ for $0 \leqslant i \leqslant (|xs| - 1)$. Suppose that σ unifies u'. Then $(\sigma F)(\!|\overline{x_m}|\!) \equiv (\sigma G)(\!|\overline{y_n}|\!)$. We want to find a term r such that $(\{H \mapsto r\} \circ \varphi)F \equiv \sigma F$ and $(\{H \mapsto r\} \circ \varphi)G \equiv \sigma G$.

Suppose that there is no element common to $\overline{x_m}$ and $\overline{y_n}$. Then, using Fact 11 on page 57, we know that $\sigma F \equiv \lambda_m\ t$ and $\sigma G \equiv \lambda_n\ s$ where neither t nor s have loose bound variables and it is $t \equiv s$. Consider $\{H \mapsto t\}$. It is

$$(\{H \mapsto t\} \circ \varphi)F = \{H \mapsto t\}(\lambda_m\ H) = (\lambda_m\ t) \equiv \sigma F \tag{2.1}$$

$$(\{H \mapsto t\} \circ \varphi)G = \{H \mapsto t\}(\lambda_n\ H) = (\lambda_n\ t) \equiv (\lambda_n\ s) \equiv \sigma G \tag{2.2}$$

Suppose now that there is at least one element common to $\overline{x_m}$ and $\overline{y_n}$.

Assumption: there are zs, $\overline{z_m}$ and $\overline{z'_m}$ such that

$$(\lambda_{\lfloor xs \rfloor}\ ((\sigma F)(\!|zs|\!)))(\!|xs|\!) \longrightarrow_\beta^* (\sigma F)(\!|\overline{z_m}|\!) \quad \text{where } z_i = \underline{i} \text{ if } \exists j.x_i = y_j \tag{1}$$

$$(\lambda_{\lfloor xs \rfloor}\ ((\sigma F)(\!|zs|\!)))(\!|ys|\!) \longrightarrow_\beta^* (\sigma F)(\!|\overline{z'_m}|\!) \quad \text{where } z'_i = \underline{j} \text{ if } x_i = y_j \tag{2}$$

Consider $r = \lambda_{\lfloor xs \rfloor}\ ((\sigma F)(\!|zs|\!))$:
$(\{H \mapsto r\} \circ \varphi)F = \lambda_m\ ((\lambda_{\lfloor xs \rfloor}\ ((\sigma F)(\!|zs|\!)))(\!|xs|\!))$

$\longrightarrow_\beta^*$ ⟨with (1) of assumption⟩

$\lambda_m\ ((\sigma F)(\!|\overline{z_m}|\!))$

$\equiv$ ⟨by definition of $\overline{z_m}$ and since $(\sigma F)(\!|\overline{x_m}|\!) \equiv (\sigma G)(\!|\overline{y_n}|\!)$ with Fact 14 on page 57 (instantiate 's', 't' and '$\overline{z'_m}$' of the fact with '(σF)', '(σG)' and '$[\underline{m-1}, \ldots, \underline{0}]$' respectively)⟩

$\lambda_m\ ((\sigma F)(\!|[\underline{m-1}, \ldots, \underline{0}]|\!))$

$\equiv$ ⟨which is η-equivalent to (σF) (Lemma 20 on page 54, (σF) has no loose bound variables)⟩

s

Equally for G:
$(\{H \mapsto r\} \circ \varphi)G = \lambda_n\ ((\lambda_{\lfloor xs \rfloor}\ ((\sigma F)(\!|zs|\!)))(\!|ys|\!))$

$\longrightarrow_\beta^*$ ⟨with (2) of assumption⟩

$\lambda_n\ ((\sigma F)(\!|\overline{z'_m}|\!))$

$\equiv$ ⟨by definition of $\overline{z'_m}$ and since $(\sigma F)(\!|\overline{x_m}|\!) \equiv (\sigma G)(\!|\overline{y_n}|\!)$ with Fact 13 on page 57 (instantiate 's', 't' and '$\overline{z_n}$' of the fact with '(σF)', '(σG)', and '$[\underline{m-1}, \ldots, \underline{0}]$' respectively)⟩

$\lambda_n\ ((\sigma G)(\!|[\underline{n-1}, \ldots, \underline{0}]|\!))$

$\equiv$ ⟨which is η-equivalent to (σG) (Lemma 20 on page 54)⟩

σG

We now show the assumption about zs by giving a witness:

$$zs_i = \begin{cases} \underline{k} \text{ with } xs_k = \underline{i} & \exists j.x_i = y_j \\ \underline{0} & \text{otherwise} \end{cases}$$

An analysis of the specification of xs shows that zs is well defined and completely determined.

With Lemma 19 on page 54 we see that

$$(\lambda_{\lfloor xs \rfloor}\ ((\sigma F)(\!|zs|\!)))(\!|xs|\!) = t(\!|\overbrace{[xs_{zs_{m-1}}, \ldots, xs_{zs_0}]}^{\overline{z_m} :=}|\!)$$

Consider i with $\exists j.x_i = y_j$.
$z_i = xs_{zs_i}$

$=$ ⟨by definition of zs it is $zs_i = \underline{k}$ with $xs_k = \underline{i}$⟩

$\underline{i}$

Also with Lemma 19 on page 54:

$$(\lambda_{\lfloor xs \rfloor} ((\sigma F)(\llbracket zs \rrbracket)))(\llbracket ys \rrbracket) = t(\overbrace{[ys_{zs_{m-1}}, \ldots, ys_{zs_0}]}^{\overline{z'_m}:=})$$

Consider i with $x_i = y_j$.

$z'_i = ys_{zs_i}$

$= \langle$by definition of zs it is $zs_i = \underline{k}$ with $xs_k = \underline{i}$. Further $x_{xs_k} = y_{ys_k}$ by definition of xs and ys. But also $x_i = y_j$ and every element only occurs once in $\overline{x_m}$ and $\overline{y_n}\rangle$

$\underline{j}$

So the assumption holds.

□

Proposition 3. *If* $(cases_1 \circ tw \circ otf\eta)\ u = \llcorner\langle us, \varphi\rangle\lrcorner$ *for a unification problem* u *in* β*-normal form then* (φu) *and* us *have the same set of unifiers.*

Proof. Be $u' = (tw \circ otf\eta)u$. We know that u and u' have the same set of unifiers (Proposition 1 on page 12) and therefore φu and $\varphi u'$ also have the same set of unifiers.

case distinction over form of u'

(rigid-rigid) u' has the form $p(\llbracket \overline{s_n} \rrbracket) \stackrel{?}{=} p(\llbracket \overline{t_n} \rrbracket)$, $\varphi = \emptyset$. Obviously any σ unifies u' iff σ unifies $[s_{n-1} \stackrel{?}{=} t_{n-1}, \ldots, s_0 \stackrel{?}{=} t_0]$

(flex-rigid) u' has the form $F(\llbracket \overline{x_m} \rrbracket) \stackrel{?}{=} p(\llbracket \overline{s_n} \rrbracket)$, $\varphi = \{F \mapsto \lambda_m\ \underline{j}(\overline{H_n(\llbracket zs \rrbracket)})\}$ with $x_j = p$, $zs = [\underline{m-1}, \ldots, \underline{0}]$. We note that

$$\varphi u' = ((\lambda_m\ \underline{j}(\overline{H_n(\llbracket zs \rrbracket)}))(\llbracket \overline{x_m} \rrbracket) \stackrel{?}{=} p(\llbracket \overline{s_n} \rrbracket))$$

Observe the lhs of $\varphi u'$: $(\lambda_m\ \underline{j}(\overline{H_n(\llbracket zs \rrbracket)}))(\llbracket \overline{x_m} \rrbracket)$

$\equiv \langle$with Lemma 18 on page 54$\rangle$

$x_j(\llbracket H_n(\llbracket \overline{x_m} \rrbracket) \rrbracket) = p(\llbracket H_n(\llbracket \overline{x_m} \rrbracket) \rrbracket)$

We observe that σ unifies $p(\llbracket H_n(\llbracket \overline{x_m} \rrbracket) \rrbracket) \stackrel{?}{=} p(\llbracket \overline{s_n} \rrbracket)$ iff σ unifies $[H_{n-1}(\llbracket \overline{x_m} \rrbracket) \stackrel{?}{=} s_{n-1}, \ldots, H_0(\llbracket \overline{x_m} \rrbracket) \stackrel{?}{=} s_0]$

(flex-flex-1)

u' has the form $F(\llbracket \overline{x_m} \rrbracket) \stackrel{?}{=} F(\llbracket \overline{y_m} \rrbracket)$, $\varphi = \{F \mapsto \lambda_m\ H(\llbracket zs \rrbracket)\}$ with $zs = [\underline{k} \mid 0 \leqslant k \leqslant (m-1), x_k = y_k]$. Since the list of new unification problems is empty, we have to show that φ actually unifies u', that is $(\lambda_m\ H(\llbracket zs \rrbracket))(\llbracket \overline{x_m} \rrbracket) \equiv (\lambda_m\ H(\llbracket zs \rrbracket))(\llbracket \overline{y_m} \rrbracket)$. For $zs = []$ this is obvious. Suppose that zs is nonempty.

$(\lambda_m\ H(\llbracket zs \rrbracket))(\llbracket \overline{x_m} \rrbracket)$

$\longrightarrow_\beta^* \langle$with Lemma 19 on page 54$\rangle$

$H(\llbracket [x_{zs_{\lfloor zs \rfloor - 1}}, \ldots, x_{zs_0}] \rrbracket)$

$= \langle$definition of $zs\rangle$

$H(\llbracket [y_{zs_{\lfloor zs \rfloor - 1}}, \ldots, y_{zs_0}] \rrbracket)$

$\longleftarrow_\beta^* \langle$with Lemma 19 on page 54$\rangle$

$(\lambda_m\ H(\llbracket zs \rrbracket))(\llbracket \overline{y_m} \rrbracket)$

(flex-flex-2) u' has form $F(\llbracket \overline{x_m} \rrbracket) \stackrel{?}{=} G(\llbracket \overline{y_n} \rrbracket)$, $\varphi = \{F \mapsto \lambda_m\ H(\llbracket xs \rrbracket)\} \circ \{G \mapsto \lambda_n\ H(\llbracket ys \rrbracket)\}$ with $|xs| = |ys| = |\{\overline{x_m}\} \cap \{\overline{y_n}\}|$, the elements of xs and ys are pairwise distinct and $x_{xs_i} = y_{ys_i}$ for $0 \leqslant i \leqslant (|xs| - 1)$. We have to show that $(\lambda_m\ H(\llbracket xs \rrbracket))(\llbracket \overline{x_m} \rrbracket) \equiv (\lambda_n\ H(\llbracket ys \rrbracket))(\llbracket \overline{y_n} \rrbracket)$. For $xs = ys = []$ this is obvious. Suppose that xs and ys are nonempty.

$(\lambda_m\ H(\llbracket xs \rrbracket))(\llbracket \overline{x_m} \rrbracket)$

$\xrightarrow{*}_\beta$ ⟨with Lemma 19 on page 54⟩
$H(\!|[x_{xs_{|xs|-1}}, \ldots, x_{xs_0}]|\!)$
$=$ ⟨definition of xs and ys⟩
$H(\!|[y_{ys_{|xs|-1}}, \ldots, y_{ys_0}]|\!)$
$\xleftarrow{*}_\beta$ ⟨with Lemma 19 on page 54⟩
$(\lambda_n\ H(\!|ys|\!))(\!|\overline{y_n}|\!)$

□

Proposition 4. *If* $(cases_1 \circ tw \circ otf\eta)\ u = \mathsf{None}$ *for a unification problem* u *in* β*-normal form then* u *not unifyable.*

Proof. Be $u' = (tw \circ otf\eta)u$. We know that a substitution σ unifies u iff σ unifies u' (Lemma 1 on page 12). Assume that $cases_1\ u' = \mathsf{None}$. Then u' is either of the form $p(\!|\overline{s_n}|\!) \stackrel{?}{=} q(\!|\overline{t_n}|\!)$ with $a \neq b$ and thus clearly not unifyable, or u' is of the form $F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)$ and $F \in \mathcal{FV}(\overline{s_n})$. Then u' is not unifyable because of Fact 9 on page 57. □

2.2.3 Proof of correctness

The following propositions use the findings about $cases_1$ in order to prove that $unif_1$ finds most general unifiers. The first of them is rather complicated, because we prove by induction and need a induction hypothesis which is strong enough to carry the proof through. Its proof is the first proof to reason about constructed substitutions.

Proposition 5. *If* $\varphi uus = uus$ *for a list of unification problems* uus *in* β*-normal form and* $unif_1\ \langle uus, \varphi\rangle = \llcorner\varphi'\lrcorner$*, then there is a* ψ *with* $\psi\circ\varphi = \varphi'$ *and* $\mathcal{FV}(\psi)\cap(\mathcal{RNG}(\varphi)\backslash\mathcal{FV}(uus)) = \emptyset$*. This* ψ *unifies* uus *and is most general in the following sense: if* σ *unifies* uus *and* $\mathcal{RNG}\langle\sigma, \mathcal{FV}(uus)\rangle \cap \mathcal{FV}(\psi) = \emptyset$ *then exists a* γ *with* $\gamma \circ \psi \equiv \sigma$ *with respect to* $\mathcal{FV}(uus)$*. For domain and range of* γ *it is* $\mathcal{DOM}(\gamma) \subseteq \mathcal{RNG}(\psi)$ *and* $\mathcal{RNG}(\gamma) \subseteq \mathcal{RNG}\langle\sigma, \mathcal{FV}(uus)\rangle$*.*

Proof. We prove by computational induction over $unif_1$

case distinction over uus

$uus = []$ $unif_1\ \langle[], \varphi\rangle = \llcorner\varphi\lrcorner$. Take $\psi = \emptyset$. It is $\emptyset \circ \varphi = \varphi$, $\emptyset$ unifies $[]$ and $\sigma \circ \emptyset = \sigma$ for any σ

$uus = u{:}us$ Then with the definition of $unif_1$ on page 10

$unif_1\ \langle u{:}us, \varphi\rangle =$ **case** $(cases_1 \circ tw \circ otf\eta)\ u$ **of**
$\quad\mid \mathsf{None} \rightarrow \mathsf{None}$
$\quad\mid \llcorner\langle us', \mu\rangle\lrcorner \rightarrow unif_1\ \langle us' +\!\!+ (\mu\ us)\downarrow_\beta, \mu \circ \varphi\rangle$

So if $unif_1\ \langle u{:}us, \varphi\rangle = \llcorner\varphi'\lrcorner$ then

$$(cases_1 \circ tw \circ otf\eta)\ u = \llcorner\langle us', \mu\rangle\lrcorner \tag{I}$$
$$unif_1\ \langle us' +\!\!+ (\mu\ us)\downarrow_\beta, \mu \circ \varphi\rangle = \llcorner\varphi'\lrcorner \tag{II}$$

An analysis of $cases_1$ shows that us' is in β-normal form and that $(\mu \circ \varphi)us' = us'$ (use that u is in β-normal form and $\varphi u = u$). Further $(\mu \circ \varphi)(\mu\ us)\downarrow_\beta = (\mu\ us)\downarrow_\beta$ since $(\mu \circ \varphi)$ is a constructed substitution (and therefore idempotent, see Lemma 1 on page 5) and $\varphi us = us$. So the induction hypothesis can be used on (II): We know that there is a ψ' that unifies $us' +\!\!+ (\mu\ us)\downarrow_\beta$ and

$$\psi' \circ \mu \circ \varphi = \varphi' \tag{1}$$
$$(\mathcal{FV}(\psi') \cap (\mathcal{RNG}(\mu \circ \varphi)\backslash\mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta))) = \emptyset \tag{2}$$

The induction hypothesis also says that if there is a ρ' which unifies $us' +\!\!+ (\mu\ us)\downarrow_\beta$ and $\mathcal{RNG}\langle\rho', us' +\!\!+ (\mu\ us)\downarrow_\beta\rangle \cap \mathcal{FV}(\psi') = \emptyset$ then exists a ρ with

$$\rho \circ \psi' \equiv \rho' \quad \text{with respect to } \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta) \tag{3}$$
$$\mathcal{DOM}(\rho) \subseteq \mathcal{RNG}(\psi') \tag{4}$$
$$\mathcal{RNG}(\rho) \subseteq \mathcal{RNG}\langle\rho', \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)\rangle \tag{5}$$

We now prove the proposition for $\psi := \psi' \circ \mu$.

With (1) we see that $\psi \circ \varphi = \varphi'$. We further show $\mathcal{FV}(\psi) \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus)) = \emptyset$:
$\mathcal{FV}(\psi) \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus)) = \mathcal{FV}(\psi' \circ \mu) \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus))$

$=$ ⟨definition of the free variables of a constructed substitution⟩

$(\mathcal{FV}(\psi') \cup \mathcal{FV}(\mu)) \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus))$

$=$ ⟨distributivity of $\cap$ over $\cup$⟩

$(\mathcal{FV}(\psi') \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus))) \cup (\mathcal{FV}(\mu) \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus)))$

Clearly $\mathcal{FV}(\mu) \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus)) = \emptyset$, since $\mathcal{DOM}(\mu) \subseteq \mathcal{FV}(uus)$ and $\mathcal{RNG}(\mu)$ consists of new variables, as an analysis of $cases_1$ shows.

Now for $\mathcal{FV}(\psi') \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus))$: Suppose that $F \in \mathcal{RNG}(\varphi)$ and $F \notin \mathcal{FV}(uus)$. Because $\mathcal{DOM}(\mu) \subseteq \mathcal{FV}(uus)$ but $F \notin \mathcal{FV}(uus)$ it is $F \in \mathcal{RNG}(\mu \circ \varphi)$. So if we can show that $F \notin \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)$ then we have

$$\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus) \subseteq (\mathcal{RNG}(\mu \circ \varphi) \backslash \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta))$$

and therefore because of (2) also $\mathcal{FV}(\psi') \cap (\mathcal{RNG}(\varphi) \backslash \mathcal{FV}(uus)) = \emptyset$. An analysis of $cases_1$ shows that $\mathcal{FV}(us') \subseteq \mathcal{RNG}(\mu) \cup \mathcal{FV}(u)$. Since $\mathcal{RNG}(\mu)$ is new and $F \notin \mathcal{FV}(uus)$, clearly it is $F \notin \mathcal{FV}(us')$. For the same reasons it is $F \notin \mathcal{FV}((\mu\ us)\downarrow_\beta)$.

Does ψ unify uus? We know that ψ' unifies $us' +\!\!+ (\mu\ us)\downarrow_\beta$. From Proposition 3 on page 16 we know that us' and μu have the same set of unifiers. So ψ' unifies μu and therefore ψ unifes u. Because ψ' unifies $(\mu\ us)\downarrow_\beta$, ψ' also unifies $\mu\ us$ (see Fact 2 on page 6). Thus ψ unifies us.

Let σ unify uus and

$$\mathcal{RNG}\langle \sigma, \mathcal{FV}(uus) \rangle \cap \mathcal{FV}(\psi) = \emptyset \tag{6}$$

Let $\sigma' = \sigma\restriction_{\mathcal{FV}(uus)}$. For the set N of new variables introduced in μ it is $N \subseteq \mathcal{FV}(\psi)$ since $\psi = \psi' \circ \mu$. So with (6) it follows that $\mathcal{RNG}(\sigma') \cap N = \emptyset$. It is obvious that $\mathcal{DOM}(\sigma') \cap N = \emptyset$ since the variables in N are new with respect to uus. So Proposition 2 on page 12 can be used on (I): There is a ρ' such that

$$\rho' \circ \mu \equiv \sigma' \quad \text{with respect to } \mathcal{FV}(uus) \tag{7}$$

$$\mathcal{RNG}(\rho') \subseteq \mathcal{RNG}(\sigma') \tag{8}$$

We now show that ρ' unifies $us' +\!\!+ (\mu\ us)\downarrow_\beta$. Since σ' unifies uus and $\rho' \circ \mu \equiv \sigma'$ with respect to $\mathcal{FV}(uus)$ it is clear that ρ' unifies $(\mu\ us)$ and thus (see Fact 2 on page 6) ρ' unifies $(\mu\ us)\downarrow_\beta$. ρ' also unifies μu and because of Proposition 3 on page 16 it unifies us'.

Since ρ' unifies $us' +\!\!+ (\mu\ us)\downarrow_\beta$ there is a ρ such that (3), (4) and (5) hold for ρ if $\mathcal{RNG}\langle \rho', us' +\!\!+ (\mu\ us)\downarrow_\beta \rangle \cap \mathcal{FV}(\psi') = \emptyset$ (using the induction hypothesis); this however follows easily from (8), (6) and the fact that $\mathcal{FV}(\psi') \subseteq \mathcal{FV}(\psi)$.

We now try to find a substitution γ such that $\gamma \circ \psi \equiv \sigma$ with respect to $\mathcal{FV}(uus)$.

case distinction

$\forall F \in \mathcal{FV}(uus).\mathcal{FV}(\mu F) \subseteq \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)$ In this case we show that ρ has the desired properties

For $F \in \mathcal{FV}(uus)$:

$(\rho \circ \psi' \circ \mu)F = (\rho \circ \psi')(\mu F)$

$\equiv$ ⟨since $\mathcal{FV}(\mu F) \subseteq \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)$ and because of (3)⟩

$\rho'(\mu F) = (\rho' \circ \mu)F$

$\equiv$ ⟨with (7)⟩

$\sigma' F$

Furthermore $\mathcal{DOM}(\rho) \subseteq \mathcal{RNG}(\psi') \subseteq \mathcal{RNG}(\psi)$ with (4) and $\mathcal{RNG}(\rho) \subseteq \mathcal{RNG}\langle \rho', us' +\!\!+ (\mu\ us)\downarrow_\beta \rangle \subseteq \mathcal{RNG}(\sigma')$ with (5) and (8). Thus ρ has the necessary properties.

$\exists F \in \mathcal{FV}(uus).\mathcal{FV}(\mu F) \not\subseteq \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)$

Analyzing the situation one finds that such an F must be in $\mathcal{FV}(u)$ but not in $\mathcal{FV}(us)$ and $F \in \mathcal{DOM}(\mu)$. Furthermore μ only can be the result of a (flex-flex-1) or a (flex-flex-2) step since in all other cases $\mathcal{RNG}(\mu) \subseteq \mathcal{FV}(us')$. In both cases it is $|\mathcal{FV}(\mu F)| = 1$.

Let $\{H\} = \mathcal{FV}(\mu F)$. So μ is either $\{\lambda_m\ H(\!|xs|\!) \mapsto F\}$ in case of (flex-flex-1) for some m and some xs or $\{\lambda_m\ H(\!|xs|\!) \mapsto F\} \circ \{\lambda_n\ H(\!|ys|\!) \mapsto G\}$ in case of (flex-flex-2) for some m, n, xs and ys.

Because of (7) we know that $\sigma' F \equiv (\rho' \circ \mu)F = \rho'(\mu F) = \rho'(\lambda_m\ H(\!|xs|\!))$. We show that $\rho'\restriction_{\{H\}} \circ \rho$ is a substitution with the desired properties:

$(\rho'\restriction_{\{H\}} \circ \rho \circ \psi' \circ \mu)F = (\rho'\restriction_{\{H\}} \circ \rho \circ \psi')(\mu F) = (\rho'\restriction_{\{H\}} \circ \rho \circ \psi')(\lambda_m\ H(\!|xs|\!))$

$=$ ⟨With (2) follows that $H \notin \mathcal{FV}(\psi')$, since $H \notin \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)$ but $H \in \mathcal{RNG}(\mu \circ \varphi)$⟩

$(\rho'\restriction_{\{H\}} \circ \rho)(\lambda_m\ H(\!|xs|\!))$

$=$ ⟨because of (4) and $H \notin \mathcal{FV}(\psi')$⟩

$\rho'\restriction_{\{H\}}(\lambda_m\ H(\!|xs|\!)) \equiv \sigma' F$

The same argument shows, if μ was built by (flex-flex-2) that $(\rho'\restriction_{\{H\}} \circ \rho \circ \psi' \circ \mu)G \equiv \sigma' G$. What about a variable E with $\mathcal{FV}(\mu E) \subseteq \mathcal{FV}(us' +\!\!+ (\mu\ us)\downarrow_\beta)$? Obviously we can use the findings from the previous case, so $(\rho \circ \psi' \circ \mu)E \equiv \sigma' E$. Further we know that $H \notin \mathcal{FV}(\mu E)$. With $H \notin \mathcal{FV}(\psi')$ it follows that $H \notin \mathcal{FV}((\psi' \circ \mu)E)$. From (5), (8) and (6) follows that $H \notin \mathcal{RNG}(\rho)$, so $H \notin \mathcal{RNG}((\rho \circ \psi' \circ \mu)E)$. Therefore

$$(\rho'\restriction_{\{H\}} \circ \rho \circ \psi' \circ \mu)E \equiv (\rho \circ \psi' \circ \mu)E \equiv \sigma' E$$

As a last step we have to check the domain and codomain of $\rho'\restriction_{\{H\}} \circ \rho$. We already have shown in the previous case that $\mathcal{DOM}(\rho) \subseteq \mathcal{RNG}(\psi)$. Obviously

$$\mathcal{DOM}(\rho'\restriction_{\{H\}} \circ \rho) = (\mathcal{DOM}(\rho) \cup \{H\}) \subseteq \mathcal{RNG}(\psi)$$

We also have shown that $\mathcal{RNG}(\rho) \subseteq \mathcal{RNG}(\sigma')$. Because $\mathcal{FV}(\rho'\restriction_{\{H\}} H) \subseteq \mathcal{FV}(\sigma' F)$ it follows that

$$\mathcal{RNG}(\rho'\restriction_{\{H\}} \circ \rho) \subseteq \mathcal{RNG}(\sigma')$$

□

Proposition 6. *If* $unif_1\ \langle uus, \varphi\rangle = \mathsf{None}$ *then* uus *not unifyable.*

Proof. We prove by computational induction over $unif_1$. Suppose that $unif_1\ \langle u{:}us, \varphi\rangle = \mathsf{None}$. Since

$$\begin{aligned} unif_1\ \langle u{:}us, \varphi\rangle = \mathbf{case}\ &(cases_1 \circ tw \circ otf\eta)\ u\ \mathbf{of} \\ &\mid \mathsf{None} \to \mathsf{None} \\ &\mid \llcorner\langle us', \mu\rangle\lrcorner \to unif_1\ \langle us' +\!\!+ (\mu\ us)\downarrow_\beta, \mu \circ \varphi\rangle \end{aligned}$$

this is the case iff either (1) or (2) holds:

$$(cases_1 \circ tw \circ otf\eta)\ u = \mathsf{None} \tag{1}$$

$$unif_1\ \langle us' +\!\!+ (\mu\ us)\downarrow_\beta, \mu \circ \varphi\rangle = \mathsf{None} \tag{2}$$

Assume (1). With Proposition 4 on page 17 follows that u is not unifyable, therefore uus is not unifyable.

Assume (2). From the induction hypothesis follows that $us' +\!\!+ (\mu\ us)\downarrow_\beta$ is not unifyable. Suppose now that γ unifies uus. Set $\gamma' := \gamma\restriction_{\mathcal{FV}(uus)}$. Find a renaming δ such that $\mathcal{RNG}(\delta \circ \gamma') \cap N = \emptyset$, where N is the set of new variables introduced in $cases_1$. Since N is new with respect to uus, we have $\mathcal{DOM}(\delta \circ \gamma') \cap N = \emptyset$. From Fact 8 on page 57 follows that $\delta \circ \gamma'$ unifies uus. Because of Proposition 2 on page 12 there is a ρ such that $(\rho \circ \mu) \equiv (\delta \circ \gamma')$ with respect to uus. Obviously ρ unifies μu and (with Proposition 3 on page 16) ρ also unifies us'. Since $\rho \circ \mu$ unifies us, Fact 2 on page 6 states that θ also unifies $(\mu\ us)\downarrow_\beta$. So ρ unifies $us' +\!\!+ (\mu\ us)\downarrow_\beta$ in contradiction to the induction hypothesis. Thus uus is not unifyable. □

Now we prove the correctness of unif_1.

Proposition 7 (Correctness of unif_1). *A unification problem u in β-normal form is unifyable iff $\mathit{unif}_1\langle[u],\emptyset\rangle = \llcorner\varphi\lrcorner$ for some φ. In this case, φ is a most general unifier of u.*

Proof. It follows from Proposition 6 on the preceding page that for a unifyable unification problem u there is a substitution φ such that $\mathit{unif}_1\ \langle[u],\emptyset\rangle = \llcorner\varphi\lrcorner$.

Suppose that $\mathit{unif}_0\langle[u],\emptyset\rangle = \llcorner\varphi\lrcorner$. Then φ unifies u because of Proposition 5 on page 17. Thus u is unifyable.

Suppose now that σ unifies u. Set $\sigma' := \sigma\restriction_{\mathcal{FV}(u)}$. If $\mathcal{RNG}(\sigma')\cap\mathcal{FV}(\varphi)\neq\emptyset$ then find a renaming δ such that $\mathcal{RNG}(\delta\circ\sigma')\cap\mathcal{FV}(\varphi)=\emptyset$. From Fact 8 on page 57 follows that $\delta\circ\sigma'$ unifies u. Because of Proposition 5 on page 17 there is a γ such that $\gamma\circ\varphi\equiv\delta\circ\sigma'$ with respect to $\mathcal{FV}(u)$. Again with Fact 8 on page 57 follows that $\delta^{-1}\circ\gamma\circ\varphi\equiv\sigma'$ with respect to $\mathcal{FV}(u)$. □

Chapter 3

A small structural change

In the first version of the algorithm, every substitution returned by $cases_1$ is applied to the complete list of unification problems. Now the complete substitution that has been calculated will be applied only to the first unification problem in every step. Thereby unnecessary substitutions can be avoided in case of a ununifyable unification problem.

3.1 Definition of $unif_2$

There are two obvious possibilities for the application of the calculated substitution to the first unification problem: Either directly before the application of $cases_1 \circ tw \circ otf\eta$, or as last step before the recursive call of $unif_2$. We choose the second option; it looks clumsier, mainly because of the use of *head_ap*, but eases the proof of Proposition 11.

$$
\begin{aligned}
unif_2\ \langle uus, \varphi\rangle = \ & \textbf{case}\ uus\ \textbf{of} \\
& \mid [] \to \llcorner\varphi\lrcorner \\
& \mid u{:}us \to \textbf{case}\ (cases_1 \circ tw \circ otf\eta)\ u\ \textbf{of} \\
& \qquad \mid \mathsf{None} \to \mathsf{None} \\
& \qquad \mid \llcorner\langle us', \varphi'\rangle\lrcorner \to unif_2\ \langle head_ap(\lambda v.((\varphi' \circ \varphi)v)\downarrow_\beta)(us' +\!\!+ us), \varphi' \circ \varphi\rangle
\end{aligned}
$$

3.2 Proof of correctness

In the correctness proof we will relate $unif_1$ to $unif_2$ and show that they return the same result for arguments that are in a certain relation. We have to be sure that the same new variables are introduced in both versions. Therefore it is necessary to pay attention the parameter mentioned in section 1.2.2.

Correctness is a direct corollary of the following proposition.

Proposition 8.
Let $\mathcal{DOM}(\varphi) \cap \mathcal{FV}(uus) = \emptyset$ for a substitution φ and a list of unification problems uus in β-normal form. Let there be a list of unification problems vvs such that either $uus = vvs = []$ or $uus = u{:}us$ and $vvs = v{:}vs$ with $u = v$ and $(\varphi vs)\downarrow_\beta = us$. Let the parameter significant for the choice of new variables be the same for $unif_1\ \langle uus, \varphi\rangle$ and $unif_2\ \langle vvs, \varphi\rangle$. Then

$$unif_1\ \langle uus, \varphi\rangle = unif_2\ \langle vvs, \varphi\rangle$$

Proof. We prove by computational induction over $unif_1$.

case distinction over uus

$uus = []$ $\Rightarrow vvs = []$ and $unif_1\ \langle [], \varphi\rangle = unif_2\ \langle [], \varphi\rangle = \llcorner\varphi\lrcorner$

$uus = (u{:}us) \Rightarrow vvs = (v{:}vs)$ and by assumption $u = v$. So with definitions of $unif_1$ on page 10 and of $unif_2$ on the preceding page:

$$unif_1\ \langle u{:}us, \varphi\rangle = \textbf{let } res_1 = (cases_1 \circ tw \circ otf\eta)\ u \textbf{ in case } res_1 \textbf{ of } | \mathsf{None} \to \mathsf{None} \mid \llcorner\langle us', \psi\rangle\lrcorner \to unif_1\ \langle us' +\!\!+ (\psi us)\downarrow_\beta, \psi \circ \varphi\rangle$$

$$unif_2\ \langle u{:}vs, \varphi\rangle = \textbf{let } res_2 = (cases_1 \circ tw \circ otf\eta)\ u \textbf{ in case } res_2 \textbf{ of } | \mathsf{None} \to \mathsf{None} \mid \llcorner\langle us', \psi\rangle\lrcorner \to unif_1\ \langle head_ap\ \lambda v.((\psi \circ \varphi)v)\downarrow_\beta\ \ us' +\!\!+ us, \psi \circ \varphi\rangle$$

We know that $res_1 = res_2$ since the parameter significant for the choice of new variables is the same in both cases. Thus the same new variables are introduced in res_1 and res_2. We also can conclude that the parameter is changed in the same way in both $unif_1$ and $unif_2$.

With $res_1 = res_2$ we have that either $unif_1\ \langle u{:}us, \varphi\rangle = unif_2\ \langle u{:}vs, \varphi\rangle = \mathsf{None}$ or

$$\begin{aligned} unif_1\ \langle u{:}us, \varphi\rangle &= unif_1\ \langle us' +\!\!+ (\psi us)\downarrow_\beta, \psi \circ \varphi\rangle \\ unif_2\ \langle u{:}vs, \varphi\rangle &= unif_2\ \langle head_ap\ \lambda v.((\psi \circ \varphi)v)\downarrow_\beta\ \ us' +\!\!+ vs, \psi \circ \varphi\rangle \end{aligned}$$

Can the induction hypothesis be used? We already clarified the question about the parameter significant for the choice of new variables. An analysis of $cases_1$ (definition on page 9) shows that

(a) us' is in β-normal form and $us' = \varphi us' = (\psi \circ \varphi)us'$ (since $\mathcal{DOM}(\varphi) \cap \mathcal{FV}(u) = \emptyset$ and u is in β-normal form)

(b) $\mathcal{DOM}(\psi) \subseteq \mathcal{FV}(u)$

(c) $\mathcal{RNG}(\psi)$ consists of new variables

(d) $\mathcal{FV}(us') \subseteq (\mathcal{FV}(u) \cup \mathcal{RNG}(\psi))\backslash\mathcal{DOM}(\psi)$

We observe that

- $us' +\!\!+ (\psi us)\downarrow_\beta$ is in β-normal form because us is in β-normal form by assumption and (a)
- $((\psi \circ \varphi)(us' +\!\!+ vs))\downarrow_\beta = (us' +\!\!+ (\psi us)\downarrow_\beta)$ since by (a) it is $(\psi\circ\varphi)us' = us'$ and $(\varphi vs)\downarrow_\beta = us$ by assumption (use Fact 1 on page 6)
- $\mathcal{DOM}(\psi \circ \varphi) \cap \mathcal{FV}(us' +\!\!+ (\psi us)\downarrow_\beta) = \emptyset$ because it is $\mathcal{DOM}(\varphi) \cap \mathcal{FV}(uus) = \emptyset$ by assumption and because of (b), (c) and (d)

With these observations it is easy to see that the induction hypothesis can be used.

□

Proposition 9 (Correctness of *unif*$_2$). *$unif_1\ \langle[u], \emptyset\rangle = unif_2\ \langle[u], \emptyset\rangle$ for u in β-normal form.*

Proof. Obviously all the requirements of proposition 8 on the page before are met. Especially the same parameter significant for the choice of new variables will be used for $unif_1$ and $unif_2$, since the set of used free variables in $unif_1\ \langle[u], \emptyset\rangle$ is the same as in $unif_2\ \langle[u], \emptyset\rangle$. □

Chapter 4

A refinement step

Two remaining underspecifications in $cases_1$ are now removed. The function *unif* remains basically unchanged, only '$cases_1$' is changed to '$cases_3$':

$$
\begin{array}{l}
unif_3\ \langle uus, \varphi\rangle = \textbf{case}\ uus\ \textbf{of} \\
\qquad\qquad | \ [] \to \llcorner\varphi\lrcorner \\
\qquad\qquad | \ u{:}us \to \textbf{case}\ (cases_3 \circ tw \circ otf\eta)\ u\ \textbf{of} \\
\qquad\qquad\qquad\qquad | \ \mathsf{None} \ \to \mathsf{None} \\
\qquad\qquad\qquad\qquad | \ \llcorner\langle us', \varphi'\rangle\lrcorner \to unif_3\ \langle head_ap(\lambda v.((\varphi' \circ \varphi)v)\downarrow_\beta)(us' +\!\!+ us), \varphi' \circ \varphi\rangle
\end{array}
$$

4.1 Definition of $cases_3$

Only the branches (flex-rigid) and (flex-flex-2) are changed. In the former we specify how to find the j with $x_j = a$, in the later the calculation of xs and ys is given explicitly. The other branches are exactely like in the definition of $cases_1$, on page 9.

$$
\begin{array}{l}
cases_3\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)) = \qquad\qquad \text{(flex-rigid)} \\
= \textbf{if}\ F \in \mathcal{FV}(\overline{s_n}) \\
\qquad \textbf{then}\ \mathsf{None} \\
\qquad \textbf{else case}\ idx\ \overline{x_m}\ p\ \textbf{of} \\
\qquad\qquad | \ \mathsf{None} \ \to \mathsf{None} \\
\qquad\qquad | \ \llcorner j\lrcorner \to \langle [H_{n-1}(\overline{x_m}) \stackrel{?}{=} s_{n-1}, \ldots, H_0(\overline{x_m}) \stackrel{?}{=} s_0], \{F \mapsto \lambda_m\ (\underline{j})(\overline{H_n(\!|zs|\!)})\}\rangle \\
\quad \textbf{where}\ zs = [\underline{m-1} \ldots \underline{0}]
\end{array}
$$

$$
\begin{array}{l}
cases_3\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} G(\!|\overline{y_n}|\!)) = \llcorner\langle [], \{F \mapsto \lambda_m\ H(\!|xs|\!)\} \circ \{G \mapsto \lambda_n\ H(\!|ys|\!)\}\rangle\lrcorner \qquad \text{(flex-flex-2)} \\
\qquad\qquad\qquad\qquad \textbf{where}\ \overline{y'_n} = map\ (idx\ \overline{x_m})\ \overline{y_n}, \\
\qquad\qquad\qquad\qquad\qquad\qquad xs = [\underline{j} \mid 0 \leqslant k \leqslant (n-1), y'_k = \llcorner j\lrcorner], \\
\qquad\qquad\qquad\qquad\qquad\qquad ys = [\underline{k} \mid 0 \leqslant k \leqslant (n-1), y'_k = \llcorner_\lrcorner]
\end{array}
$$

4.2 Proof of correctness

The proof of correctness for version three follows from two lemmas about the changed branches (flex-rigid) and (flex-flex-2).

4.2.1 A lemma about (flex-rigid)

Lemma 5. $cases_1\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)) = cases_3\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!))$

Proof. With Lemma 4 on page 7 it is easy to see that $cases_1\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)) = \mathsf{None}$ whenever $cases_3\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)) = \mathsf{None}$. Lemma 3 on page 7 shows that the j calculated in $cases_3$ satisfies the requirement of the **where**-clause in $cases_1$. □

4.2.2 A lemma about (flex-flex-2)

Lemma 6. *The lists xs and ys as calculated in the* (flex-flex-2) *branch of $cases_3$ satisfy the requirements specified in $cases_1$, i.e.*

1. $|xs| = |\{xs\}| = |ys| = |\{ys\}| = |\{\overline{x_m}\} \cap \{\overline{y_m}\}|$
2. $x_{xs_i} = y_{ys_i}$ *for* $0 \leqslant i \leqslant (|xs| - 1)$

Proof. The list $\overline{y'_n}$ is used in $cases_3$ to calculate xs and ys (see definition of $cases_3$ on the page before). It is $\overline{y'_n} = map\ (idx\ \overline{x_m})\ \overline{y_n}$. Any single element y_i gives rise to an element of the form $\llcorner__\lrcorner$, iff $y_i \in \{\overline{x_m}\}$ (Lemma 4 on page 7). It is obvious that the number of elements of the form $\llcorner__\lrcorner$ in $\overline{y'_n}$ is $|\{\overline{x_m}\} \cap \{\overline{y_m}\}|$ since the elements in x_m/y_n are pairwise distinct. That $|ys| = |\{ys\}| = |\{\overline{x_m}\} \cap \{\overline{y_m}\}|$ follows directly from the definition of ys. Since the elements in x_m/y_n are pairwise distinct, all the elements of form $\llcorner__\lrcorner$ in $\overline{y'_n}$ are pairwise distinct. Thus $|xs| = |\{xs\}| = |\{\overline{x_m}\} \cap \{\overline{y_m}\}|$.

For any i with $0 \leqslant i \leqslant (|xs| - 1)$ it is $xs_i = \llcorner j \lrcorner$ and $ys_i = k$ for some k where $x_j = y_k$; this follows from the definitions of xs and ys and Lemma 3 on page 7. Thus

$$x_{xs_i} = x_j = y_k = y_{ys_i}$$

□

4.2.3 Correctness

Proposition 10. *The refinements made in version three satisfy the specification of version two*

Proof. Version two and version three only differ in the branches (flex-rigid) and (flex-flex-2). For (flex-rigid), Lemma 5 on the preceding page states equality. Branch (flex-flex-2) in $cases_3$ only differs from its version in $cases_1$ in the way xs and ys are specified. Lemma 6 states that the definition in $cases_3$ satisfies the requirements of the definition in $cases_1$. □

Chapter 5

An optimization for flex-rigid-pairs

An analysis of the branch (flex-rigid) shows that the new variables introduced by this branch will be removed by subsequent applications of the branches (flex-rigid) or (flex-flex-2). These variables are merely used to descend in the structure of the rhs of a flex-rigid problem. In version four of the algorithm a function *proj* will be used to solve flex-rigid problems without introducing this kind of new variables. It corresponds to an interleaving of (flex-rigid)- and (flex-flex-2)-branches.

Only *cases* is changed, so $unif_4$ is similar to $unif_3$; merely '$cases_3$' has been changed to '$cases_4$':

$$
\begin{array}{l}
unif_4\ \langle uus, \varphi\rangle = \textbf{case}\ uus\ \textbf{of} \\
\qquad\qquad | \ [] \rightarrow \llcorner\varphi\lrcorner \\
\qquad\qquad | \ u{:}us \rightarrow \textbf{case}\ (cases_4 \circ tw \circ otf\eta)\ u\ \textbf{of} \\
\qquad\qquad\qquad | \ \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad | \ \llcorner\langle us', \varphi'\rangle\lrcorner \rightarrow unif_4\ \langle head_ap(\lambda v.((\varphi' \circ \varphi)v)\downarrow_\beta)(us' +\!\!+ us), \varphi' \circ \varphi\rangle
\end{array}
$$

5.1 Definition of *cases*$_4$

5.1.1 A new flex-rigid branch for *cases*

Only the (flex-rigid)-branch is changed. The other branches remain as in $cases_3$ (see on page 23 and on page 9).

$$
\begin{array}{l}
cases_4\ (F(\overline{x_m}) \stackrel{?}{=} p(\overline{s_n})) = \textbf{if}\ F \in \mathcal{FV}(\overline{s_n}) \qquad \text{(flex-rigid)} \\
\qquad\qquad\qquad\qquad \textbf{then}\ \mathsf{None} \\
\qquad\qquad\qquad\qquad \textbf{else let}\ res\ = proj_4\ \overline{x_m}\ \langle\emptyset, p(\overline{s_n})\rangle \\
\qquad\qquad\qquad\qquad\qquad \textbf{in case}\ res\ \ \textbf{of} \\
\qquad\qquad\qquad\qquad\qquad\quad | \ \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\qquad\qquad\quad | \ \llcorner\langle\psi, t\rangle\lrcorner \rightarrow \llcorner\langle [], \psi \circ \{F{\mapsto}\lambda_m\ t\}\rangle\lrcorner
\end{array}
$$

5.1.2 The *proj*-function

The function *proj* models the application of $unif_3$ to a single flex-rigid problem. Whereas $unif_3$ descends in the structure of the problem's rhs by creating a number of new unification problems for every rigid functor that is encountered, *proj* steps recursively through that structure. It can be shown that *proj* always terminates if it is started with an argument $\langle\varphi, t\rangle$ such that $\varphi t = t$: For an argument $\langle\varphi, t\rangle$ to *proj* associate with t the pair of the number of functors in t and the number of lambdas in t. The lexicographic order over $\mathbb{N} \times \mathbb{N}$ constitutes a termination order for $proj_4$.

$$
proj_4\ zs\ \langle\varphi, t\rangle = \textbf{case}\ t\ \textbf{of}
$$

$$
\begin{array}{l}
\quad \mid \lambda t' \rightarrow \textbf{let } res \ = proj_4\ ((map\ (\lambda x.lift\ x\ 0)\ zs) +\!\!+ [\underline{0}])\ \langle \varphi, t' \rangle \\
\qquad\quad \textbf{in case } res \ \textbf{ of} \\
\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\quad\ \ \mid \llcorner \langle \varphi', t'' \rangle \lrcorner \rightarrow \llcorner \langle \varphi', \lambda t'' \rangle \lrcorner \\
\mid p(\overline{[\![s_m]\!]}) \rightarrow \textbf{case } idx\ zs\ p \textbf{ of} \\
\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\quad\ \ \mid \llcorner j \lrcorner \rightarrow \textbf{let } res \ = lproj_4\ zs\ \langle \varphi, \overline{s_m} \rangle \\
\qquad\qquad\qquad\quad \textbf{in case } res \ \textbf{ of} \\
\qquad\qquad\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\quad\ \ \mid \llcorner \langle \varphi', \overline{s'_m} \rangle \lrcorner \rightarrow \llcorner \langle \varphi', \underline{j}(\overline{[\![s'_m]\!]}) \rangle \lrcorner \\
\mid F(\overline{[\![x_m]\!]}) \rightarrow \textbf{let } \overline{x'_m} = map\ (idx\ zs)\ \overline{x_m} \\
\qquad\qquad\qquad\quad \varphi' = \{F \mapsto \lambda_m\ H([\![\underline{k} \mid 0 \leqslant k \leqslant (m-1), x'_k = \llcorner _ \lrcorner]\!])\} \circ \varphi \\
\qquad\qquad\qquad\quad t' = H([\![\underline{l} \mid 0 \leqslant k \leqslant (m-1), x'_k = \llcorner l \lrcorner]\!]) \\
\qquad\qquad\quad\ \ \textbf{in } \llcorner \langle \varphi', t' \rangle \lrcorner
\end{array}
$$

$$
\begin{array}{l}
lproj_4\ zs\ \langle \varphi, tts \rangle = \textbf{case } tts \textbf{ of} \\
\qquad\qquad\quad \mid [] \rightarrow \llcorner \langle \varphi, [] \rangle \lrcorner \\
\qquad\qquad\quad \mid t\textbf{:}ts \rightarrow \textbf{let } res_1 \ = proj_4\ zs\ \langle \varphi, (\varphi t)\downarrow_\beta \rangle \\
\qquad\qquad\qquad\quad \textbf{in case } res_1 \ \textbf{ of} \\
\qquad\qquad\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\quad\ \ \mid \llcorner \langle \varphi', t' \rangle \lrcorner \rightarrow \textbf{let } res_2 \ = lproj_4\ zs\ \langle \varphi', ts \rangle \\
\qquad\qquad\qquad\qquad\qquad\quad \textbf{in case } res_2 \ \textbf{ of} \\
\qquad\qquad\qquad\qquad\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\qquad\qquad\quad\ \ \mid \llcorner \langle \varphi'', ts' \rangle \lrcorner \rightarrow \llcorner \langle \varphi'', t'\textbf{:}ts' \rangle \lrcorner
\end{array}
$$

5.2 Proof of correctness

5.2.1 Reasoning about $unif_3$

The following fact holds because the tail of a list of unification problems remains unchanged until the last unification problem that originated from the list's head has been solved.

Fact 4. *$unif_3\ \langle u\textbf{:}us, \varphi \rangle$ can be rewritten as*

$$
\begin{array}{l}
unif_3\ \langle u\textbf{:}us, \varphi \rangle = \textbf{let } res = unif_3\ \langle [u], \varphi \rangle \\
\qquad\qquad\qquad\quad \textbf{in case } res \textbf{ of} \\
\qquad\qquad\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\quad\ \ \mid \llcorner \varphi' \lrcorner \rightarrow unif_3\ \langle head_ap(\lambda v.(\varphi' v)\downarrow_\beta) us, \varphi' \rangle
\end{array}
$$

With this rewrite rule the follwing more complicated rewrite rule can be shown:

Lemma 7. *If for a unification problem u in β-normal form and a substitution φ it is $(\varphi u)\downarrow_\beta = u$ then $unif_3\ \langle u\textbf{:}us, \varphi \rangle$ can be rewritten as*

$$
\begin{array}{l}
unif_3\ \langle u\textbf{:}us, \varphi \rangle = \textbf{let } res = unif_3\ \langle [u], \emptyset \rangle \\
\qquad\qquad\qquad\quad \textbf{in case } res \textbf{ of} \\
\qquad\qquad\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\quad\ \ \mid \llcorner \psi \lrcorner \rightarrow unif_3\ \langle head_ap\ (\lambda v.((\psi \circ \varphi) v)\downarrow_\beta)\ us, \psi \circ \varphi \rangle
\end{array}
$$

Proof. We rewrite $unif_3\ \langle u\textbf{:}us, \varphi \rangle$ with Fact 4 to

$$
\begin{array}{l}
unif_3\ \langle u\textbf{:}us, \varphi \rangle = \textbf{let } res = unif_3\ \langle [u], \varphi \rangle \\
\qquad\qquad\qquad\quad \textbf{in case } res \textbf{ of} \\
\qquad\qquad\qquad\quad\ \ \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\quad\ \ \mid \llcorner \varphi' \lrcorner \rightarrow unif_3\ \langle head_ap\ (\lambda v.(\varphi' v)\downarrow_\beta)\ us, \varphi' \rangle
\end{array}
$$

Assumption: If ψ and φ are substitutions such that $\psi \circ \varphi$ is a constructed substitution and for a list of unification problems uus in β-normal form it is $\varphi uus = uus$, then either $unif_3\ \langle uus, \psi \circ \varphi\rangle = unif_3\ \langle uus, \psi\rangle = \mathsf{None}$ or $unif_3\ \langle uus, \psi \circ \varphi\rangle = \llcorner\varphi'\lrcorner$, $unif_3\ \langle uus, \psi\rangle = \llcorner\psi'\lrcorner$ for some φ',$\psi'$and $\psi' \circ \varphi = \varphi'$

Obviously the assumption is applicable to $unif_3\ \langle [u], \varphi\rangle$, choosing ψ as $\emptyset$: either $unif_3\ \langle [u], \varphi\rangle = unif_3\ \langle [u], \emptyset\rangle = \mathsf{None}$ or $unif_3\ \langle uus, \varphi\rangle = \llcorner\varphi'\lrcorner$, $unif_3\ \langle uus, \emptyset\rangle = \llcorner\psi\lrcorner$ for some φ', ψ' and $\psi \circ \varphi = \varphi'$. Thus the rewrite rule

$$
\begin{aligned}
unif_3\ \langle u{:}us, \varphi\rangle = {} & \textbf{let } res = unif_3\ \langle [u], \emptyset\rangle \\
& \textbf{in case } res \textbf{ of} \\
& \quad \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
& \quad \mid \llcorner\psi\lrcorner \rightarrow unif_3\ \langle head_ap\ (\lambda v.((\psi \circ \varphi)v)\downarrow_\beta)\ us, \psi \circ \varphi\rangle
\end{aligned}
$$

is justified.

We now prove the assumption by computational induction over $unif_3$:

It is (using the definition of $unif_3$ on page 23)

$$
\begin{aligned}
unif_3\ \langle u{:}us, \psi \circ \varphi\rangle = {} & \textbf{let } res = (cases_3 \circ tw \circ otf\eta)\ u \\
& \textbf{in case } res \textbf{ of} \\
& \quad \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
& \quad \mid \llcorner\langle us', \psi'\rangle\lrcorner \rightarrow unif_3\ \langle head_ap\ (\lambda v.((\psi' \circ \psi \circ \varphi)v)\downarrow_\beta)\ us' +\!\!+ us, \psi' \circ \psi \circ \varphi\rangle
\end{aligned}
$$

and

$$
\begin{aligned}
unif_3\ \langle u{:}us, \psi\rangle = {} & \textbf{let } res = (cases_3 \circ tw \circ otf\eta)\ u \\
& \textbf{in case } res \textbf{ of} \\
& \quad \mid \mathsf{None} \ \rightarrow \mathsf{None} \\
& \quad \mid \langle us', \psi'\rangle \rightarrow unif_3\ \langle head_ap\ (\lambda v.((\psi' \circ \psi)v)\downarrow_\beta)\ us' +\!\!+ us, \psi' \circ \psi\rangle
\end{aligned}
$$

If $(cases_3 \circ tw \circ otf\eta)\ u = \mathsf{None}$ then $unif_3\ \langle u{:}us, \psi \circ \varphi\rangle = unif_3\ \langle u{:}us, \psi\rangle = \mathsf{None}$. Otherwise in case of success: For $us' +\!\!+ us = []$ everything is clear. Suppose that $us' +\!\!+ us$ is not empty. An analysis of *cases* shows that if $\varphi u = u$ and u is in β-normal form then $\varphi us' = us'$ and us' is in β-normal form. Thus, using $\varphi us = us$, we find that $\varphi(us' +\!\!+ us) = us' +\!\!+ us$ and $us' +\!\!+ us$ is in β-normal form. Suppose that u' is the first element of $us' +\!\!+ us$. In order to use the induction hypothesis we have to show that $((\psi' \circ \psi)u')\downarrow_\beta = ((\psi' \circ \psi \circ \varphi)u')\downarrow_\beta$.

$((\psi' \circ \psi \circ \varphi)u')\downarrow_\beta$

$= \langle$with Fact 1 on page 6$\rangle$

$((\psi' \circ \psi)(\varphi u')\downarrow_\beta)\downarrow_\beta$

$= \langle \varphi u' = u'$ and u' is in β-normal form$\rangle$

$((\psi' \circ \psi)u')\downarrow_\beta$

So the induction hypothesis can be used.

□

5.2.2 Preparation of the correctness proof

The correctness proof will relate the substitutions created by $unif_3$ and $unif_4$. In the subsequent proofs we will assume that whenever version four of the algorithm introduces a new variable then $unif_3$ introduces the same variable in the corresponding step. In order to show that this assumption is valid, one has to consider the hidden parameter significant for the choice of new variables mentioned in section 1.2.2:

An analysis of $unif_3$ and $unif_4$ shows that the new variables introduced in the (flex-rigid)-branch of $cases_3$ have no counterpart in the calculation of by $unif_4$. On the other hand all free variables introduced in $unif_3$ by the branches (flex-flex-1) and (flex-flex-2) have counterparts in the calculation done by $unif_4$.

The hidden parameter and two functions which read the parameter and return a new variable name (one function for the branch (flex-rigid), the other function for the branches (flex-flex-1) and (flex-flex-2)) can be specified such that the introduction of new variables in the (flex-rigid)-branch of $cases_3$ is totally independent of the introduction of new variables in (flex-flex-1) and (flex-flex-2).

The specification of such a parameter obviously constitutes a refinement of version three. Suppose that version four of the algorithm uses the same parameter. Suppose further that version four only uses the function for the introduction of new variables, which was used in the branches (flex-flex-1) and (flex-flex-2) of version three. Then the assumption made above can be shown.

As we will show, the relation between the results of $unif_3$ and $unif_4$ is captured by the notion of *β-approximation*:

Definition 6. *Substitution ψ is called β-approximated by substitution φ, written '$\psi \overset{\beta}{\Longleftarrow} \varphi$', if*

- $\mathcal{DOM}(\psi) \subseteq \mathcal{DOM}(\varphi)$
- $\mathcal{RNG}(\psi) \subseteq \mathcal{RNG}(\varphi)$
- $\forall F \in \mathcal{DOM}(\psi).\varphi F \overset{*}{\longrightarrow}_\beta \psi F$

There is an important lemma about $\beta-$approximations:

Lemma 8. *If $\psi \overset{\beta}{\Longleftarrow} \varphi$ and $\psi' \overset{\beta}{\Longleftarrow} \varphi'$ with*

- *$\psi' \circ \psi$ and $\varphi' \circ \varphi$ are constructed substitutions*
- *$(\mathcal{DOM}(\varphi') \backslash \mathcal{DOM}(\psi')) \cap \mathcal{RNG}(\psi) = \emptyset$*

it is $(\psi' \circ \psi) \overset{\beta}{\Longleftarrow} (\varphi' \circ \varphi)$

Proof. It is obvious that $\mathcal{DOM}(\psi' \circ \psi) \subseteq \mathcal{DOM}(\varphi' \circ \varphi)$. We show that $\mathcal{RNG}(\psi' \circ \psi) \subseteq \mathcal{RNG}(\varphi' \circ \varphi)$. Suppose that

$$F \in \mathcal{RNG}(\psi' \circ \psi) \tag{1}$$
$$F \notin \mathcal{RNG}(\varphi' \circ \varphi) \tag{2}$$

case distinction

$F \in \mathcal{RNG}(\psi')$ But then $F \in \mathcal{RNG}(\varphi')$ and thus obviously $F \in \mathcal{RNG}(\varphi' \circ \varphi)$ which is a contradiction to (2).

$F \in \mathcal{RNG}(\psi), F \notin \mathcal{RNG}(\psi')$ Since $F \in \mathcal{RNG}(\psi' \circ \psi)$ we can conclude that

$$F \notin \mathcal{DOM}(\psi') \tag{3}$$

From $F \in \mathcal{RNG}(\psi)$ we conclude that there is a $G \in \mathcal{DOM}(\psi)$ such that $F \in \mathcal{FV}(\psi G)$. Thus also

$$F \in \mathcal{FV}(\varphi G) \tag{4}$$

since $\varphi G \overset{*}{\longrightarrow}_\beta \psi G$, and β-expansion does not remove free variables in a term.

With (2) and (4) we can conclude that $F \in \mathcal{DOM}(\varphi')$. Thus with (3) it is

$$F \in \mathcal{DOM}(\varphi') \backslash \mathcal{DOM}(\psi')$$

But also $F \in \mathcal{RNG}(\psi)$, so

$$(\mathcal{DOM}(\varphi') \backslash \mathcal{DOM}(\psi')) \cap \mathcal{RNG}(\psi) \neq \emptyset$$

This is a contradiction to the assumption. So $F \in \mathcal{RNG}(\varphi' \circ \varphi)$.

We now show that $\forall F \in \mathcal{DOM}(\psi' \circ \psi).(\varphi' \circ \varphi)F \xrightarrow{*}_\beta (\psi' \circ \psi)F$.

case distinction

$F \in \mathcal{DOM}(\psi)$ Suppose that $D \in \mathcal{FV}(\psi F)$, so

$$D \in \mathcal{RNG}(\psi) \tag{5}$$

case distinction

$D \in \mathcal{DOM}(\varphi')$ Then $D \in \mathcal{DOM}(\psi')$ because of (5) and

$$((\mathcal{DOM}(\varphi') \backslash \mathcal{DOM}(\psi')) \cap \mathcal{RNG}(\psi)) = \emptyset$$

by assumption. Thus $\varphi' D \xrightarrow{*}_\beta \psi' D$

$D \notin \mathcal{DOM}(\varphi')$ Then $D \notin \mathcal{DOM}(\psi')$ since $\mathcal{DOM}(\psi') \subseteq \mathcal{DOM}(\varphi')$. Thus $\varphi' D = \psi' D = D$ and therefore $\varphi' D \xrightarrow{*}_\beta \psi' D$

Since $\varphi F \xrightarrow{*}_\beta \psi F$ and for every $D \in \mathcal{DOM}(\psi F)$ it is $\varphi' D \xrightarrow{*}_\beta \psi' D$ it follows that

$$(\varphi' \circ \varphi)F \xrightarrow{*}_\beta (\psi' \circ \psi)F$$

$F \notin \mathcal{DOM}(\psi)$ $\Rightarrow F \in \mathcal{DOM}(\psi')$. It follows that $F \notin \mathcal{DOM}(\varphi)$; otherwise it is $F \in \mathcal{DOM}(\varphi)$ and $F \in \mathcal{DOM}(\varphi')$ (because of $F \in \mathcal{DOM}(\psi')$). This however is impossible for the constructed substitution $\varphi' \circ \varphi$.

So $(\varphi' \circ \varphi)F = \varphi'(\varphi F) = \varphi' F$

$\xrightarrow{*}_\beta$ $\langle$because of $\psi' \xLeftarrow{\beta} \varphi'\rangle$

$\psi' F = \psi'(\psi F) = (\psi' \circ \psi)F$

□

5.2.3 Proof of correctness

As was mentioned above, the proof of correctness will relate the results of version three and version four. Here we prove that $proj_4$ actually modells the application of $unif_3$ to a single flex-rigid problem. We assume that whenever $proj_4$ introduces a new variable then $unif_3$ introduces the same new variable in the corresponding step. Variables are called "new" always with respect to the context they were introduced in. In the following proof the context is not explicitely given when a new variable is introduced, since it always can be deduced easily.

Proposition 11. *Let* $res_3 := unif_3\langle[F(\!|\overline{x_m}|\!) \stackrel{?}{=} s], \varphi\rangle$ *and* $res_4 := proj_4\ \overline{x_m}\ \langle\psi, s\rangle$ *with*

(a) $\psi \xLeftarrow{\beta} \varphi$

(b) s *in* β*-normal form,* $\varphi s = s$

(c) $F \notin \mathcal{FV}(\psi)$, $F \notin \mathcal{DOM}(\varphi)$ *and* $F \notin \mathcal{FV}(s)$

Then either $res_3 = res_4 = \mathsf{None}$ *or* $res_3 = \llcorner\varphi'\lrcorner$ *and* $res_4 = \llcorner\langle\psi', s'\rangle\lrcorner$ *and*

(1) there are θ *and* μ *with* $\theta \circ \varphi = \varphi'$ *and* $\mu \circ \psi = \psi'$

(2) $\mu \xLeftarrow{\beta} \theta$

(3) $\mathcal{DOM}(\mu) = \mathcal{FV}(s)$ *+ new variables,* $\mathcal{DOM}(\theta) = \mathcal{FV}(s) \cup \{F\}$ *+ new variables*

(4) $(\mathcal{DOM}(\theta) \backslash \mathcal{DOM}(\mu)) \cap \mathcal{RNG}(\psi) = \emptyset$

(5) $\mathcal{RNG}(\theta)$ *and* $\mathcal{FV}(s')$ *consist of new variables*

(6) $\theta F \xrightarrow{*}_\beta \psi'(\lambda_m\ s')$

Proof. We prove by computational induction over $proj_4$

case distinction over s

$s = \lambda\ s'$ We rewrite res_3 with the definition of $unif_3$ on page 23

$unif_3\langle[F(\!|\overline{x_m}|\!) \stackrel{?}{=} \lambda\ s'], \varphi\rangle = \mathbf{case}\ (cases_3 \circ tw \circ otf\eta)\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} \lambda\ s')\ \mathbf{of}\ldots$

$=$ ⟨definition of $otf\eta$ on page 10, definition of *lift* on page 52⟩

$\mathbf{case}\ (cases_3 \circ tw \circ otf\eta)\ (F(\!|(map\ (\lambda x.lift\ x\ 0)\ \overline{x_m}) +\!\!+ [\underline{0}]|\!) \stackrel{?}{=} s')\ \mathbf{of}\ldots$

$=$ ⟨definition of $unif_3$ on page 23⟩

$unif_3\langle[F(\!|(map\ (\lambda x.lift\ x\ 0)\ \overline{x_m}) +\!\!+ [\underline{0}]|\!) \stackrel{?}{=} s'], \varphi\rangle$

On the other hand we have by definition of $proj_4$ on page 25 that

$$res_4 = proj_4\ \overline{x_m}\ \langle\psi, \lambda\ s'\rangle = \mathbf{let}\ res = proj_4\ ((map\ (\lambda x.lift\ x\ 0)\ \overline{x_m}) +\!\!+ [\underline{0}])\ \langle\psi, s'\rangle$$
$$\mathbf{in\ case}\ res\ \mathbf{of}$$
$$\mid \mathsf{None} \rightarrow \mathsf{None}$$
$$\mid \llcorner\langle\psi', s''\rangle\lrcorner \rightarrow \llcorner\langle\psi', \lambda\ s''\rangle\lrcorner$$

Obviously the induction hypothesis can be used on res and res_3. So either $res_3 = res_4 = \mathsf{None}$ or there are θ and μ with the wanted properties, especially

$$\theta F \xrightarrow{*}_\beta \psi'(\lambda_{m+1}\ s'') = \psi'(\lambda_m\ (\lambda\ s''))$$

$s = p(\!|\overline{s_n}|\!)$ With the definitions of $unif_3$ and $proj_4$ on page 23 and on page 25 respectively it is easy to see that $unif_3$ fails in the calculation of the functor q that will "replace" p iff $proj_4$ fails in the functor's calculation. Furthermore $unif_3$ and $proj_4$ will always calculate the same functor q. So

$res_3 = unif_3\langle[F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)], \varphi\rangle$

$=$ ⟨definition of $unif_3$ on page 23⟩

$unif_3\langle[H_{n-1}(\!|\overline{x_m}|\!) \stackrel{?}{=} ((\{F{\mapsto}\lambda_m\ q(\!|\overline{H_n(\!|zs|\!)}|\!)\} \circ \varphi)s_{n-1})\downarrow_\beta, H_{n-2}(\!|\overline{x_m}|\!) \stackrel{?}{=} s_{n-2}, \ldots,$
$, H_0(\!|\overline{x_m}|\!) \stackrel{?}{=} s_0], \{F{\mapsto}\lambda_m\ q(\!|\overline{H_n(\!|zs|\!)}|\!)\} \circ \varphi\rangle$

With the definition of $proj_4$ on page 25 :

$$res_4 = proj_4\ \overline{x_m}\ \langle\psi, p(\!|\overline{s_n}|\!)\rangle = \mathbf{let}\ res = lproj_4\ \overline{x_m}\ \langle\psi, \overline{s_n}\rangle$$
$$\mathbf{in\ case}\ res\ \mathbf{of}$$
$$\mid \mathsf{None} \rightarrow \mathsf{None}$$
$$\mid \llcorner\langle\psi', \overline{s'_n}\rangle\lrcorner \rightarrow \llcorner\langle\psi', q(\!|\overline{s'_n}|\!)\rangle\lrcorner$$

Assumption:
If for

$$r_1 := unif_3\langle[H_{n-1}(\!|\overline{x_m}|\!) \stackrel{?}{=} (\varphi s_{n-1})\downarrow_\beta, H_{n-2}(\!|\overline{x_m}|\!) \stackrel{?}{=} s_{n-2}, \ldots, H_0(\!|\overline{x_m}|\!) \stackrel{?}{=} s_0], \varphi\rangle$$
$$r_2 := lproj_4\ \overline{x_m}\ \langle\psi, \overline{s_n}\rangle$$

it is

(a') $\psi \stackrel{\beta}{\Longleftarrow} \varphi$

(b') $\overline{s_n}$ in β-normal form, $((\mathcal{DOM}(\varphi) \backslash \mathcal{DOM}(\psi)) \cap \mathcal{FV}(\overline{s_n})) = \emptyset$

(c') $H_i \notin \mathcal{FV}(\psi)$, $H_i \notin \mathcal{DOM}(\varphi)$ and $H_i \notin \mathcal{FV}(\overline{s_n})$ for $0 \leqslant i \leqslant (n-1)$

then either $r_1 = r_2 = \mathsf{None}$ or $r_1 = \llcorner\varphi'\lrcorner$, $r_2 = \llcorner\langle\psi', \overline{s'_n}\rangle\lrcorner$. In the latter case:

(1') there are θ and μ with $\theta \circ \varphi = \varphi'$ and $\mu \circ \psi = \psi'$

(2') $\mu \stackrel{\beta}{\Longleftarrow} \theta$

(3') $\mathcal{DOM}(\mu) = \mathcal{FV}(\psi\overline{s_n})$ + new variables, $\mathcal{DOM}(\theta) = \mathcal{FV}(\psi\overline{s_n}) \cup \{\overline{H_n}\}$ + new variables

(4') $(\mathcal{DOM}(\theta) \backslash \mathcal{DOM}(\mu)) \cap \mathcal{RNG}(\psi) = \emptyset$

(5') $\mathcal{RNG}(\theta)$ and $\mathcal{FV}(\overline{s'_n})$ consist of new variables

(6') $\theta H_i \xrightarrow{*}_\beta \psi'(\lambda_m\ s'_i)$ for $0 \leqslant i \leqslant (n-1)$

The assumption can be used on *res* and res_3. We check the prerequisites:

ad (a') We show that $F \notin \mathcal{RNG}\langle\varphi, \mathcal{DOM}(\psi)\rangle$: suppose that $F \in \mathcal{RNG}\langle\varphi, \mathcal{DOM}(\psi)\rangle$. Then there is a $G \in \mathcal{DOM}(\psi)$ such that $F \in \mathcal{FV}(\varphi G)$. But because of assumption (a) we know that $\varphi G \xrightarrow{*}_\beta \psi G$. Because of Fact 3 on page 6 it is $F \in \mathcal{FV}(\psi G)$. But $F \notin \mathcal{FV}(\psi)$ by assumption (c).
With $F \notin \mathcal{RNG}\langle\varphi, \mathcal{DOM}(\psi)\rangle$, $F \notin \mathcal{DOM}(\psi)$ (because of assumption (c)) and assumption (a) it is easy to see that $\psi \overset{\beta}{\Longleftarrow} (\{F \mapsto \lambda_m\ q(\overline{H_n(\!|zs|\!)})\} \circ \varphi)$

ad (b') This follows from assumption (b). It is

$$(\mathcal{DOM}(\{F \mapsto \lambda_m\ q(\overline{H_n(\!|zs|\!)})\} \circ \varphi) \backslash \mathcal{DOM}(\psi)) \cap \mathcal{FV}(\overline{s_n}) = \emptyset$$

because with $\varphi s = s$ by (b) and $F \notin \mathcal{FV}(s)$ by (c) one can conclude that

$$(\mathcal{DOM}(\{F \mapsto \lambda_m\ q(\overline{H_n(\!|zs|\!)})\} \circ \varphi) \cap \mathcal{FV}(\overline{s_n})) = \emptyset$$

ad (c') Obvious, since $\{\overline{H_n}\}$ is new with respect to ψ, φ and s

Therefore we conclude that

(1-1) there are θ' and μ with $\theta' \circ \{F \mapsto \lambda_m\ q(\overline{H_n(\!|zs|\!)})\} \circ \varphi = \varphi'$ and $\mu \circ \psi = \psi'$

(1-2) $\mu \overset{\beta}{\Longleftarrow} \theta'$

(1-3) $\mathcal{DOM}(\mu) = \mathcal{FV}(\psi\overline{s_n})$ + new variables, $\mathcal{DOM}(\theta') = \mathcal{FV}(\psi\overline{s_n}) \cup \{\overline{H_n}\}$ + new variables

(1-4) $(\mathcal{DOM}(\theta') \backslash \mathcal{DOM}(\mu)) \cap \mathcal{RNG}(\psi) = \emptyset$

(1-5) $\mathcal{RNG}(\theta')$ and $\mathcal{FV}(\overline{s'_n})$ consist of new variables

(1-6) $\theta' H_i \xrightarrow{*}_\beta \psi'(\lambda_m\ s'_i)$ for $0 \leqslant i \leqslant (n-1)$

We now show the consequences 1 — 6.

ad (1) Define $\theta := \theta' \circ \{F \mapsto \lambda_m\ q(\overline{H_n(\!|zs|\!)})\}$ and use (1-1)

ad (2) With (1-3) and assumption (c) follows that $F \notin \mathcal{DOM}(\mu)$. Using (1-2) it is easy to show that $\mu \overset{\beta}{\Longleftarrow} (\theta' \circ \{F \mapsto \lambda_m\ q(\overline{H_n(\!|zs|\!)})\})$

ad (3) Using $\varphi s = s$ from assumption (b) one shows easily that $\psi\overline{s_n} = \overline{s_n}$ since $\mathcal{DOM}(\psi) \subseteq \mathcal{DOM}(\varphi)$ by assumption (a). With (1-3) it follows that

$$\mathcal{DOM}(\mu) = \mathcal{FV}(\psi\overline{s_n}) + \text{new variables} = \mathcal{FV}(p(\!|\overline{s_n}|\!)) + \text{new variables}$$

The condition for $\mathcal{DOM}(\theta) = \mathcal{DOM}(\theta') \cup \{F\}$ is proved in a similar way. Remember that $\{\overline{H_n}\}$ is a set of variables new with respect to φ, F and s.

ad (4) This follows from (1-4) and from $F \notin \mathcal{RNG}(\psi)$ (assumption (c))

ad (5) From (1-5) we know that $\mathcal{RNG}(\theta')$ and $\mathcal{FV}(\overline{s'_n})$ consist of new variables. Since $s' = q(\!|\overline{s'_n}|\!)$ it is obvious that $\mathcal{FV}(\overline{s'_n})$ consists of new variables. Further we know that $\{\overline{H_n}\} \subseteq \mathcal{DOM}(\theta')$ from (1-6). Thus $\mathcal{RNG}(\theta) = \mathcal{RNG}(\theta')$ and therefore new.

ad (6) $\theta F = \theta'(\lambda_m\ q(\overline{H_n(\!|zs|\!)})) = \lambda_m\ q(\!|[(\theta' H_{n-1})(\!|zs|\!), \ldots, (\theta' H_0)(\!|zs|\!)]|\!)$
$\xrightarrow{*}_\beta$ ⟨with (1-6)⟩
$\lambda_m\ q(\!|[(\psi'(\lambda_m\ s'_{n-1}))(\!|zs|\!), \ldots, (\psi'(\lambda_m\ s'_0))(\!|zs|\!)]|\!) = \psi'(\lambda_m\ q(\overline{(\lambda_m\ s'_n)(\!|zs|\!)}))$
$\xrightarrow{*}_\beta$ ⟨with Lemma 21 on page 55⟩
$\psi'(\lambda_m\ q(\overline{s'_n}))$

We now prove the assumption:

induction over n

$n = 0$ Both in r_1 and r_2 there is an empty list of arguments. Thus:

$$\begin{aligned} r_1 &= \llcorner\varphi\lrcorner \\ r_2 &= \llcorner\langle\psi, []\rangle\lrcorner \end{aligned}$$

With $\sigma = \mu = \emptyset$ it is easy to see that all consequences hold.

$n \to n+1$ r_1 is transformed using Fact 4 on page 26:

$r_1 =$ **let** $r_1' = unif_3\ \langle[H_n(\overline{[\![x_m]\!]}) \stackrel{?}{=} (\varphi s_n)\downarrow_\beta], \varphi\rangle$
in case r_1' **of**
| None $\to$ None
| $\llcorner\varphi''\lrcorner \to unif_3\langle[\varphi''(H_{n-1}(\overline{[\![x_m]\!]}) \stackrel{?}{=} s_{n-1})\downarrow_\beta, \ldots, H_0(\overline{[\![x_m]\!]}) \stackrel{?}{=} s_0], \varphi''\rangle =: r_1''$

Using the definition of $proj_4$ on page 25, r_2 results to

$r_2 =$ **let** $r_2' = proj_4\ \overline{x_m}\ \langle\psi, (\psi s_n)\downarrow_\beta\rangle$
in case r_2' **of**
| None $\to$ None
| $\llcorner\langle\psi'', s_n'\rangle\lrcorner \to$**let** $r_2'' = lproj_4\ \overline{x_m}\ \langle\psi'', \overline{s_n}\rangle$
in case r_2'' **of**
| None $\to$ None
| $\llcorner\langle\psi', \overline{s_n'}\rangle\lrcorner \to \llcorner\langle\psi', s_n' : \overline{s_n'}\rangle\lrcorner$

Since $(\varphi s_n)\downarrow_\beta = (\psi s_n)\downarrow_\beta$ because of (a') and (b'), we can try to use the induction hypothesis of the outer induction on r_1' and r_2'. We check its prerequisites:

ad (a) by (a')

ad (b) that $(\varphi s_n)\downarrow_\beta$ is in β-normal form is obvious. That $\varphi(\varphi s_n)\downarrow_\beta = (\varphi s_n)\downarrow_\beta$ follows from the fact that φ is a constructed substitution, and therefore idempotent.

ad (c) by (c')

So we conclude that either $r_1' = r_2' =$ None or $r_1' = \llcorner\varphi''\lrcorner$ and $r_2' = \llcorner\langle\psi'', s_n'\rangle\lrcorner$. In the latter case

(2-1) there are θ' and μ' with $\theta' \circ \varphi = \varphi''$ and $\mu' \circ \psi = \psi''$

(2-2) $\mu' \stackrel{\beta}{\Longleftarrow} \theta'$

(2-3) $\mathcal{DOM}(\mu') = \mathcal{FV}((\psi s_n)\downarrow_\beta)$ + new variables, $\mathcal{DOM}(\theta') = \mathcal{FV}((\psi s_n)\downarrow_\beta) \cup \{H_n\}$ + new variables

(2-4) $(\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\mu')) \cap \mathcal{RNG}(\psi) = \emptyset$

(2-5) $\mathcal{RNG}(\theta')$ and $\mathcal{FV}(s_n')$ consist of new variables

(2-6) $\theta' H_n \stackrel{*}{\longrightarrow}_\beta \psi''(\lambda_m\ s_n')$

Note that because of (c') and (2-3) it is $\varphi'' H_{n-1} = H_{n-1}$ and therefore

$$r_1'' = unif_3\langle[H_{n-1}(\overline{[\![x_m]\!]}) \stackrel{?}{=} \varphi''(s_{n-1})\downarrow_\beta, H_{n-2}(\overline{[\![x_m]\!]}) \stackrel{?}{=} s_{n-2}, \ldots, H_0(\overline{[\![x_m]\!]}) \stackrel{?}{=} s_0], \varphi''\rangle$$

Can the inner induction hypothesis be used on r_1'' and r_2''? We check the prerequisites.

ad (a') it is $(\mu' \circ \psi) \stackrel{\beta}{\Longleftarrow} (\theta' \circ \varphi)$ because of (2-2), (a'), (2-4) and Lemma 8 on page 28

ad (b') that $\varphi''(s_{n-1})\downarrow_\beta$ is in β-normal form is obvious, that $\overline{s_{n-1}}$ is in β-normal form follows from (b'). Further

$$\begin{aligned}((\mathcal{DOM}(\theta' \circ \varphi)\backslash\mathcal{DOM}(\psi'')) \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta, s_{n-2}, \ldots, s_0)) = \\ ((\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\psi'')) \cup (\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi''))) \\ \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta, s_{n-2}, \ldots, s_0)\end{aligned}$$

Using the distributivity of $\cap$ over $\cup$ we have to show:

$$\begin{aligned}(\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\psi'')) \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta, s_{n-2}, \ldots, s_0) &= \emptyset \\ (\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi'')) \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta, s_{n-2}, \ldots, s_0) &= \emptyset\end{aligned}$$

Consider
$(\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\psi'') = \mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\mu' \circ \psi)) \subseteq \mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\mu')$
$=$ $\langle$with (2-3)$\rangle$
$\{H_n\}$ + new variables
Because of (c') it is obvious that

$$H_n \cap \mathcal{FV}(s_{n-1}, \ldots, s_0) = \emptyset$$

Further, since $H_n \in \mathcal{DOM}(\theta')$ and θ' is constructed, it is $H_n \notin \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta)$. So

$$\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\psi'') \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta, s_{n-2}, \ldots, s_0) = \emptyset$$

We now consider $\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi'')$. It is

$$\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi'') = \mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\mu' \circ \psi) \subseteq \mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi)$$

We have $\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi) \cap \mathcal{FV}(s_{n-2}, \ldots, s_0) = \emptyset$ because of (b'). Further also by (b') it is

$$\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi) \cap \mathcal{FV}(s_{n-1}) = \emptyset$$

We also have $\mathcal{DOM}(\varphi) \cap \mathcal{RNG}(\varphi'') = \emptyset$ because $\varphi'' = \theta' \circ \varphi$ is a constructed substitution. Thus $\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi) \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta) = \emptyset$. All in all it is clear that

$$\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi'') \cap \mathcal{FV}(\varphi''(s_{n-1})\downarrow_\beta, s_{n-2}, \ldots, s_0) = \emptyset$$

ad (c') For $0 \leqslant i \leqslant (n-1)$ it is $H_i \notin \mathcal{FV}(\psi)$ by (c'). Further $H_i \notin \mathcal{DOM}(\mu')$ by (2-3) and (c') and $H_i \notin \mathcal{RNG}(\mu')$ by (2-5). Thus $H_i \notin \mathcal{FV}(\mu' \circ \psi)$. It is $H_i \notin \mathcal{FV}(\overline{s_n})$ by (c'). Also $H_i \notin \mathcal{DOM}(\varphi)$ by (2-3) and $H_i \notin \mathcal{DOM}(\theta')$ by (2-3). Thus $H_i \notin \mathcal{DOM}(\theta' \circ \varphi)$

Thus either $r_1'' = r_2'' = \mathsf{None}$ or $r_1'' = \llcorner\varphi'\lrcorner$ and $r_2'' = \llcorner\langle\psi', \overline{s_n'}\rangle\lrcorner$. In the later case

(3-1) there are θ'' and μ'' with $\theta'' \circ \varphi'' = \varphi'$ and $\mu'' \circ \psi'' = \psi'$

(3-2) $\mu'' \overset{\beta}{\Longleftarrow} \theta''$

(3-3) $\mathcal{DOM}(\mu'') = \mathcal{FV}(\psi''\overline{s_n})$ + new variables, $\mathcal{DOM}(\theta'') = \mathcal{FV}(\psi''\overline{s_n}) \cup \{\overline{H_n}\}$ + new variables

(3-4) $(\mathcal{DOM}(\theta'')\backslash\mathcal{DOM}(\mu'')) \cap \mathcal{RNG}(\psi'') = \emptyset$

(3-5) $\mathcal{RNG}(\theta'')$ and $\mathcal{FV}(\overline{s_n'})$ consist of new variables

(3-6) $\theta'' H_i \xrightarrow{*}_\beta \psi'(\lambda_m\ s_i')$ for $0 \leqslant i \leqslant (n-1)$

We conclude the proof by showing the consequences (1') — (6')

ad (1') Define $\theta := \theta'' \circ \theta'$ and $\mu := \mu'' \circ \mu'$. Use (2-1) and (3-1).

ad (2') We already have shown that $(\mu' \circ \psi) \overset{\beta}{\Longleftarrow} (\theta' \circ \varphi)$. From (3-2) we know that $\mu'' \overset{\beta}{\Longleftarrow} \theta''$. Use (3-4) and Lemma 8 on page 28.

ad (3') From (2-3) we know that $\mathcal{DOM}(\mu') = \mathcal{FV}((\psi s_n)\downarrow_\beta)$ + new variables. With Fact 3 on page 6 we can conclude that $\mathcal{DOM}(\mu') = \mathcal{FV}(\psi s_n)$ + new variables. Further we know from (3-3) that

$$\mathcal{DOM}(\mu'') = \mathcal{FV}((\mu' \circ \psi)\overline{s_n}) + \text{new variables}$$

Since $\mathcal{RNG}(\mu') \subseteq \mathcal{RNG}(\theta')$ consists of new variables (see (2-5)) with respect to an "older" context than the context of the new variables in $\mathcal{DOM}(\mu'')$, we can deduce that

$$\mathcal{DOM}(\mu'') = (\mathcal{FV}(\psi\overline{s_n})\backslash\mathcal{DOM}(\mu')) + \text{new variables}$$

where "new" is to be read with respect to the older context. All in all it is

$$\mathcal{DOM}(\mu'' \circ \mu') = \mathcal{DOM}(\mu'') \cup \mathcal{DOM}(\mu') = \mathcal{FV}(\psi(s_n{:}\overline{s_n})) + \text{new variables}$$

The condition for $\theta'' \circ \theta'$ is shown with a similar argument.

ad (4') $(\mathcal{DOM}(\theta'' \circ \theta')\backslash\mathcal{DOM}(\mu'' \circ \mu')) = (\mathcal{DOM}(\theta'') \cup \mathcal{DOM}(\theta'))\backslash\mathcal{DOM}(\mu'' \circ \mu')$
$= \langle$distributivity of $\backslash$ over $\cup\rangle$

$(\mathcal{DOM}(\theta'')\backslash\mathcal{DOM}(\mu''\circ\mu'))\cup(\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\mu''\circ\mu'))$
Using distributivity of $\cap$ over $\cup$ we have to show:

$$\begin{aligned}(\mathcal{DOM}(\theta'')\backslash\mathcal{DOM}(\mu''\circ\mu'))\cap\mathcal{RNG}(\psi) &= \emptyset\\ (\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\mu''\circ\mu'))\cap\mathcal{RNG}(\psi) &= \emptyset\end{aligned}$$

The second equation is clear because of (2-4). In order to see the first equation assume that $G\in\mathcal{RNG}(\psi)$. If $G\in\mathcal{RNG}(\psi'')$ then use (3-4). So suppose that $G\notin\mathcal{RNG}(\psi'')$. Then obviously $G\in\mathcal{DOM}(\mu')$ and thus $G\notin\mathcal{DOM}(\theta')\backslash\mathcal{DOM}(\mu''\circ\mu')$

ad (5') use (2-5) and (3-5)

ad (6') for $0\leqslant i\leqslant(n-1)$ it is $(\theta''\circ\theta')H_i =$
$=$ ⟨because of (2-3) and (c') it is $\theta' H_i = H_i$⟩
$\theta'' H_i$
$\xrightarrow{*}_\beta$ ⟨because of (3-6)⟩
$\psi'(\lambda_m\ s'_i)$
So we check for H_n: Because of (2-6) and Fact 8 on page 57 we know that

$$(\theta''\circ\theta')H_n\xrightarrow{*}_\beta\theta''(\psi''(\lambda_m\ s'_n))$$

We conclude the proof by showing that $\theta''(\psi''(\lambda_m\ s'_n))\xrightarrow{*}_\beta\mu''(\psi''(\lambda_m\ s'_n))$. Suppose $F\in\mathcal{FV}(\psi''(\lambda_m\ s'_n))$
case-distinction
$F\in\mathcal{DOM}(\mu'')$ Then because of (3-2) it is $\theta''F\xrightarrow{*}_\beta\mu''F$
$F\notin\mathcal{DOM}(\mu'')$ case-distinction
$F\notin\mathcal{RNG}(\psi'')$ $\Rightarrow F\in\mathcal{FV}(s'_n)$. From (3-3) we know that

$$\mathcal{DOM}(\theta'')\backslash\mathcal{DOM}(\mu'') = \{\overline{H_n}\} + \text{new variables}$$

Since $\mathcal{FV}(s'_n)$ is new with respect to $\{\overline{H_n}\}$ (see (2-5)) and the rest of $\mathcal{DOM}(\theta'')\backslash\mathcal{DOM}(\mu'')$ is new with respect to s'_n, it is $\theta''F=\mu''F=F$. Thus $\theta''F\xrightarrow{*}_\beta\mu''F$.
$F\in\mathcal{RNG}(\psi'')$ Then $\theta''F=\mu''F=F$ because of (3-4) and therefore

$$\theta''F\xrightarrow{*}_\beta\mu''F$$

So since for any $F\in\mathcal{FV}(\psi''(\lambda_m\ s'_n))$ it is $\theta''F\xrightarrow{*}_\beta\mu''F$, we conclude that indeed $\theta''(\psi''(\lambda_m\ s'_n))\xrightarrow{*}_\beta\mu''(\psi''(\lambda_m\ s'_n))$. This concludes the proof of the assumption.

$s=G(\!|\overline{y_n}|\!)$, $G\neq F$ Using the definition of $unif_3$ on page 23 and of $proj_4$ on page 25, we rewrite res_3 and res_4. It is easy to see that the lists of bound variables created in res_4 are equal to the variable lists xs and ys created in res_3. Since we assume that the same new variables are introduced, we conclude that
$res_3 = \llcorner\{F\mapsto\lambda_m\ H(\!|xs|\!)\}\circ\{G\mapsto\lambda_m\ H(\!|ys|\!)\}\circ\varphi\lrcorner$

$$\begin{aligned}res_4 = \textbf{let}\ &\psi' = \{G\mapsto\lambda_m\ H(\!|ys|\!)\}\circ\psi\\ &s' = H(\!|xs|\!)\\ \textbf{in}\ &\llcorner\langle\psi', s'\rangle\lrcorner\end{aligned}$$

It is easy to show the conclusions (1) — (6), choosing $\theta := \{F\mapsto\lambda_m\ H(\!|xs|\!)\}\circ\{G\mapsto\lambda_m\ H(\!|ys|\!)\}$ and $\mu := \{G\mapsto\lambda_m\ H(\!|ys|\!)\}$.

□

The correctness of $unif_4$ follows directly from the following proposition:

Proposition 12. *Let $res_3 := unif_3\langle uus,\varphi\rangle$ and $res_4 := unif_4\langle uus,\psi\rangle$ with uus in β-normal form and if $uus=(u{:}us)$ then $\varphi u = u$ and $(\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi))\cap\mathcal{FV}(us)=\emptyset$. Let further $\psi\overset{\beta}{\Longleftarrow}\varphi$. Then either $res_3 = res_4 = \mathsf{None}$ or $res_3 = \llcorner\varphi'\lrcorner$ and $res_4 = \llcorner\psi'\lrcorner$. In the latter case it is $\psi'\overset{\beta}{\Longleftarrow}\varphi'$ and $\mathcal{FV}(uus)\subseteq\mathcal{DOM}(\psi')$*

Proof. We prove by computational induction over $unif_4$

case distinction over uus

$uus = []$ Then $res_3 = \llcorner\varphi\lrcorner$ and $res_4 = \llcorner\psi\lrcorner$. It is $\psi \overset{\beta}{\Longleftarrow} \varphi$ by assumption and $\mathcal{FV}(uus) = \emptyset$, so $\mathcal{FV}(uus) \subseteq \mathcal{DOM}(\psi)$

$uus = (u{:}us)$ case distinction over $v = (tw \circ otf\eta)u$

Obviously $(otf\eta\ u)$ is in β-normal form if u is in β-normal form. With the definition of $otf\eta$ on page 10 it is also easy to see that $\varphi(otf\eta\ u) = (otf\eta\ u)$ because $\varphi u = u$ and that $\mathcal{FV}(otf\eta\ u) = \mathcal{FV}(u)$. Since tw only orients the equation it is obvious that v is in β-normal form, that $\mathcal{FV}(u) = \mathcal{FV}(v)$ and that $\varphi v = v$.

(rigid-rigid) straightforward with use of the induction hypothesis

(flex-rigid) let $v = (F(\overline{x_m}) \overset{?}{=} p(\overline{s_n}))$ for some F, p, $\overline{x_m}$ and $\overline{s_n}$. With the definitions of $unif_3$ on page 23 and $unif_4$ on page 25 it is obvious that $res_3 = res_4 = \mathsf{None}$ if $F \in \mathcal{FV}(\overline{s_n})$. Otherwise (rewriting res_3 with Lemma 7 on page 26)

$$
\begin{aligned}
res_3 = {} & \textbf{let } res = unif_3\ \langle [u], \emptyset\rangle \\
& \textbf{in case } res \textbf{ of} \\
& \quad \mid \mathsf{None} \ \to \mathsf{None} \\
& \quad \mid \llcorner\varphi''\lrcorner \to unif_3\ \langle head_ap\ \lambda v.((\varphi'' \circ \varphi)v)\downarrow_\beta\ \ us, \varphi'' \circ \varphi\rangle =: res_3'
\end{aligned}
$$

$$
\begin{aligned}
res_4 = {} & \textbf{let } res' = proj_4\ \overline{x_m}\ \langle \emptyset, p(\overline{s_n})\rangle \\
& \textbf{in case } res' \textbf{ of} \\
& \quad \mid \mathsf{None} \ \to \mathsf{None} \\
& \quad \mid \llcorner\langle\psi'', s'\rangle\lrcorner \to unif_4\ \langle head_ap\ (\lambda v.((\psi'' \circ \{F{\mapsto}\lambda_m\ s'\} \circ \psi)v)\downarrow_\beta)\ us \\
& \qquad\qquad , \psi'' \circ \{F{\mapsto}\lambda_m\ s'\} \circ \psi\rangle =: res_4'
\end{aligned}
$$

Obviously Proposition 11 on page 29 can be used on res and res'. So either $res = res' = \mathsf{None}$ or $res = \llcorner\varphi''\lrcorner$ and $res' = \llcorner\langle\psi'', s'\rangle\lrcorner$. In the latter case it is

(1) $\psi'' \overset{\beta}{\Longleftarrow} \varphi''$

(2) $\mathcal{DOM}(\psi'') = \mathcal{FV}(p(\overline{s_n}))$+new variables, $\mathcal{DOM}(\varphi'') = \mathcal{FV}(p(\overline{s_n}))\cup\{F\}$+new var.

(3) $\mathcal{RNG}(\varphi'')$ and $\mathcal{FV}(s')$ consist of new variables

(4) $\varphi''F \overset{*}{\longrightarrow}_\beta \psi''(\lambda_m\ s')$

Suppose that $us = []$. Then $res_3 = \llcorner\varphi'' \circ \varphi\lrcorner$ and $res_4 = \llcorner\psi'' \circ \{F{\mapsto}\lambda_m\ s'\} \circ \psi\lrcorner$. It is

$$(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\}) \overset{\beta}{\Longleftarrow} \varphi''$$

because (see definition of a β-approximation on page 28)

- $\mathcal{DOM}(\psi'') \subseteq \mathcal{DOM}(\varphi'')$ using (1), $F \in \mathcal{DOM}(\varphi'')$ because of (2). Therefore it is $\mathcal{DOM}(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\}) \subseteq \mathcal{DOM}(\varphi'')$
- $\mathcal{RNG}(\psi'') \subseteq \mathcal{RNG}(\varphi'')$ because of (1). It is $\mathcal{RNG}(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\}) = \mathcal{RNG}(\psi'') \cup \mathcal{FV}(\psi''s')$. But since $\varphi''F \overset{*}{\longrightarrow}_\beta \psi''(\lambda_m\ s')$ by (4) it is easy to see that $\mathcal{RNG}(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\}) \subseteq \mathcal{RNG}(\varphi'')$
- It is $\varphi''G \overset{*}{\longrightarrow}_\beta (\psi'' \circ \{F{\mapsto}\lambda_m\ s'\})G$ for $G \in \mathcal{DOM}(\psi'') \cup \{F\}$: Because of (1) only F has to be checked. For F it is $\varphi''F \overset{*}{\longrightarrow}_\beta \psi''(\lambda_m\ s')$ because of (4) and $\psi''(\lambda_m\ s') = (\psi'' \circ \{F{\mapsto}\lambda_m\ s'\})F$.

Further by assumption it is $\psi \overset{\beta}{\Longleftarrow} \varphi$. Since, using (2), $\mathcal{DOM}(\varphi'')\backslash\mathcal{DOM}(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\})$ is new with respect to $\mathcal{RNG}(\psi)$, we conclude with Lemma 8 on page 28 that

$$(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\} \circ \psi) \overset{\beta}{\Longleftarrow} (\varphi'' \circ \varphi) \tag{5}$$

We also have that $\mathcal{FV}([u]) = \mathcal{FV}([v]) \subseteq \mathcal{DOM}(\psi'' \circ \{F{\mapsto}\lambda_m\ s'\})$, because $\mathcal{FV}(p(\overline{s_n})) \subseteq \mathcal{DOM}(\psi'')$ by (2), and $F \in \mathcal{DOM}(\{F{\mapsto}\lambda_m\ s'\})$. Thus for $us = []$ the proposition holds. So suppose that $us = (u'{:}us')$. Then

$$
\begin{aligned}
res_3 &= unif_3\ \langle((\varphi'' \circ \varphi)u')\downarrow_\beta {:}us', \varphi'' \circ \varphi\rangle \\
res_4 &= unif_4\ \langle((\psi'' \circ \{F{\mapsto}\lambda_m\ s'\} \circ \psi)u')\downarrow_\beta {:}us', \psi'' \circ \{F{\mapsto}\lambda_m\ s'\} \circ \psi\rangle
\end{aligned}
$$

Can the induction hypothesis be used?
With $(\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi))\cap\mathcal{FV}(us)=\emptyset$ by assumption and since $\mathcal{DOM}(\varphi'')\backslash\mathcal{DOM}(\psi''\circ\{F\mapsto\lambda_m\ s'\})$ is new with respect to us (use (2)) we can deduce that

$$(\mathcal{DOM}(\varphi''\circ\varphi)\backslash\mathcal{DOM}(\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi))\cap\mathcal{FV}(u'{:}us')=\emptyset \tag{6}$$

With (5) follows from (6) that $(\varphi''\circ\varphi)u'\xrightarrow{*}_\beta(\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi)u'$ and therefore

$$((\varphi''\circ\varphi)u')\downarrow_\beta=((\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi)u')\downarrow_\beta \tag{7}$$

The remaining prerequisites are straightforward. Thus we use the induction hypothesis: either $\varphi''=\mathsf{None}$ or $res_3=\llcorner\varphi'\lrcorner$ and $res_4=\llcorner\psi'\lrcorner$. In the latter case it is $\psi'\overset{\beta}{\Longleftarrow}\varphi'$ and

$$\mathcal{FV}(((\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi)u')\downarrow_\beta{:}us')\subseteq\mathcal{DOM}(\psi') \tag{8}$$

We conclude the proof of the (flex-rigid)-case by showing that $\mathcal{FV}(uus)\subseteq\mathcal{DOM}(\psi')$. Because of (8) we only have to show that

$$\begin{aligned}\mathcal{FV}(u)&\subseteq\mathcal{DOM}(\psi')\\ \mathcal{FV}(u')&\subseteq\mathcal{DOM}(\psi')\end{aligned}$$

Since $unif_4$ constructs substitutions by stepwise composition, there obviously is a substitution μ such that $\psi'=\mu\circ\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi$. So $\mathcal{FV}(u)=\mathcal{FV}(v)\subseteq\mathcal{DOM}(\psi')$ follows with (2).
Now for u' : Suppose that $G\in\mathcal{FV}(u')$. If $G\in\mathcal{FV}(((\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi)u')\downarrow_\beta)$ then $G\in\mathcal{DOM}(\psi')$ because of (8). So suppose that $G\notin\mathcal{FV}(((\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi)u')\downarrow_\beta)$. Since $G\in\mathcal{FV}(u')$ it is clear that $G\in\mathcal{DOM}(\psi''\circ\{F\mapsto\lambda_m\ s'\}\circ\psi)$ and thus $G\in\mathcal{DOM}(\psi')$.

(flex-flex-1) let $v=(F(\!|\overline{x_m}|\!)\stackrel{?}{=}F(\!|\overline{x_m}|\!))$ for some F and, $\overline{x_m}$. We assume that both in $unif_3$ and in $unif_4$ the same new variable is introduced. Thus it is

$$\begin{aligned}res_3&=unif_3\ \langle head_ap\ \lambda v.((\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)v)\downarrow_\beta\ \ us,\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi\rangle\\ res_4&=unif_4\ \langle head_ap\ (\lambda v.((\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)v)\downarrow_\beta)\ us,\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi\rangle\end{aligned}$$

It is $\psi\overset{\beta}{\Longleftarrow}\varphi$ by assumption and obviously $\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\overset{\beta}{\Longleftarrow}\{F\mapsto\lambda_m\ H(\!|zs|\!)\}$. Lemma 8 on page 28 can be used and thus

$$(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)\overset{\beta}{\Longleftarrow}(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi) \tag{9}$$

So if $us=[]$ the proposition obviously holds. Suppose $us=(u'{:}us')$. Can the induction hypothesis be used? We show

- $((\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)u')\downarrow_\beta=((\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)u')\downarrow_\beta$
  Consider $G\in\mathcal{FV}(u')$
  case distinction
  $G\notin\mathcal{DOM}(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)$ we know that $G\neq F$ and $G\notin\mathcal{DOM}(\varphi)$ since by assumption $(\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi))\cap\mathcal{FV}(us)=\emptyset$. So

  $$(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)G=(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)G=G$$

  $G\in\mathcal{DOM}(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)$ because of (9) it is $G\in\mathcal{DOM}(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)$. So $(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)G\xrightarrow{*}_\beta(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)G$.
  We have shown that for every $G\in\mathcal{FV}(u')$ it is

  $$(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)G\xrightarrow{*}_\beta(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)G$$

  so

  $$(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)u'\xrightarrow{*}_\beta(\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)u'$$

  and therefore

  $$u'':=((((\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\psi)u')\downarrow_\beta=((\{F\mapsto\lambda_m\ H(\!|zs|\!)\}\circ\varphi)u')\downarrow_\beta)$$

- $(\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}{\circ}\varphi)u'' = u''$ since $(\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}{\circ}\varphi)$ is a constructed substitution and therefore idempotent.
- $(\mathcal{DOM}(\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}\circ\varphi)\backslash\mathcal{DOM}(\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}\circ\psi))\cap\mathcal{FV}(us') = \emptyset$ since $(\mathcal{DOM}(\varphi)\backslash\mathcal{DOM}(\psi))\cap\mathcal{FV}(us) = \emptyset$ by assumption.

So the induction hypothesis can be used. It is $res_3 = res_4 = \mathsf{None}$ or $res_3 = \llcorner\varphi'\lrcorner$ and $res_4 = \llcorner\psi'\lrcorner$. In the latter case it is $\psi' \overset{\beta}{\Longleftarrow} \varphi'$ and

$$\mathcal{FV}(u''{:}us') \subseteq \mathcal{DOM}(\psi') \tag{10}$$

We have to show that $\mathcal{FV}(uus) \subseteq \mathcal{DOM}(\psi')$. Obviously it suffices to prove that

$$\begin{array}{rcl}\mathcal{FV}(u) & \subseteq & \mathcal{DOM}(\psi')\\ \mathcal{FV}(u') & \subseteq & \mathcal{DOM}(\psi')\end{array}$$

Since $unif_4$ constructs substitutions by stepwise composition, there is a substitution μ such that $\psi' = (\mu\circ\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}\circ\psi)$. Therefore it is clear that $\mathcal{FV}(u) = \mathcal{FV}(v) \subseteq \mathcal{DOM}(\psi')$ Now for u' : Suppose that $G \in \mathcal{FV}(u')$. If $G \in \mathcal{FV}((((\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}\circ\psi)u')\downarrow_\beta$) then $G \in \mathcal{DOM}(\psi')$ because of (10). So suppose that $G \notin \mathcal{FV}((((\psi''\circ\{F{\mapsto}\lambda_m\ s'\}\circ\psi)u')\downarrow_\beta$). Since $G \in \mathcal{FV}(u')$ it is clear that $G \in \mathcal{DOM}(\{F{\mapsto}\lambda_m\ H(\!|zs|\!)\}\circ\psi)$ and thus $G \in \mathcal{DOM}(\psi')$

(flex-flex-2) similar to the proof of case (flex-flex-1)

□

Finally we show the correctness of $unif_4$.

Proposition 13 (Correctness of $unif_4$). *For a unification problem u in β-normal form it is $unif_4\ \langle[u],\emptyset\rangle = \mathsf{None}$ iff u is not unifyable. Otherwise $unif_4\ \langle[u],\emptyset\rangle = \llcorner\psi\lrcorner$ and ψ is a most general unifier of u*

Proof. Consider $unif_3\ \langle[u],\emptyset\rangle$. Obviously Proposition 12 on page 34 can be used. So either $unif_3\ \langle[u],\emptyset\rangle = unif_4\ \langle[u],\emptyset\rangle = \mathsf{None}$ or $unif_3\ \langle[u],\emptyset\rangle = \llcorner\varphi\lrcorner$, $unif_4\ \langle[u],\emptyset\rangle = \llcorner\psi\lrcorner$. In the latter case it is $\psi \overset{\beta}{\Longleftarrow} \varphi$ and $\mathcal{FV}(u) \subseteq \mathcal{DOM}(\psi)$.

We know that $unif_3$ only returns None iff u is not unifyable, so also $unif_4$ only returns None if u is not unifyable. If u is unifyable then φ is a most general unifier of u.

The substitution ψ also is a unifier of u: Suppose that $u = (s \overset{?}{=} t)$. Then it is

$$\psi s \xleftarrow{*}_\beta \varphi s \equiv \varphi t \xrightarrow{*}_\beta \psi t$$

and therefore $\psi s \equiv \psi t$.

Suppose that σ is a unifier of u. Since φ is a most general unifier, there is a substitution γ such that $\gamma\circ\varphi \equiv \sigma$ with respect to $\mathcal{FV}(u)$. Suppose that $G \in \mathcal{FV}(u)$. Then
$(\gamma\circ\psi)G = \gamma(\psi G)$
$\xleftarrow{*}_\beta$ ⟨since $\psi G \xleftarrow{*}_\beta \varphi G$ and because of Fact 8 on page 57⟩
$\gamma(\varphi G) = (\gamma\circ\varphi)G \equiv \sigma G$
so $\gamma\circ\psi \equiv \sigma$ with respect to $\mathcal{FV}(u)$.

□

Chapter 6

Introducing partial substitution

$unif_4$ calculates a function from lambda terms to lambda terms. To receive something more tangible, a list representation for substitutions is introduced. Obviously any constructed substitution is representable by a list of bindings (or representations thereof). Rewriting the algorithm in order to calculate the substitution's representation rather than the substitution is straightforward — the first section of this chapter gives the details. Thus we gain a new but isomorphic version of the algorithm.

Before the first unification problem u in the list is examined by $cases_5$, $unif_5$ applies the substitution calculated up to this point to u and β-reduces the result. Like in the case of first order unification, this practice may lead to exponentially large representations of the substitution [9]. In order to avoid this, one tries to partially instantiate and β-reduce a unification problem. The function *devar* will carry out such a partial substitution. After defining *devar*, two lemmas will clarify the relation between *devar* and full substitution. These lemmas are central to the following proof of correctness for a new version of the algorithm.

6.1 Representation of substitutions

The algorithm constructs a substitution φ by composing bindings:

$$\varphi = \{F_{n-1} \mapsto t_{n-1}\} \circ \ldots \circ \{F_0 \mapsto t_0\}$$

A single binding $\{F \mapsto t\}$ is representable by a pair $\langle F, t\rangle$, which is turned into a binding using *mk_subst* and *mk_bind* (definitions on page 4):

$$\{F \mapsto t\} = mk_subst\ (mk_bind\ \langle F, t\rangle)$$

Therefore a substitution may be represented by an association list:

$$\varphi = subst\ [\langle F_0, t_0\rangle, \ldots, \langle F_{n-1}, t_{n-1}\rangle]$$

where

$$\begin{aligned} subst\ pps\ t = {} & \textbf{case}\ pps\ \textbf{of} \\ & \mid [] \to t \\ & \mid \langle F, s\rangle\textbf{:}ps \to subst\ ps\ (mk_subst\ (mk_bind\ \langle F, s\rangle)\ t) \end{aligned}$$

In notation there will be no distinction between a substitution and its representation. Obviously

$$\varphi t = subst\ \varphi\ t \qquad (*)$$

where 'φ' denotes the actual substitution on the lhs and its list representation on the rhs.

Consider now

$$
\begin{array}{l}
b_subst\ pps\ t = \textbf{case}\ pps\ \textbf{of} \\
\qquad\qquad | \ [] \to t \\
\qquad\qquad | \ \langle F, s\rangle\textbf{:}ps \to b_subst\ ps\ (mk_subst\ (mk_bind\ \langle F, s\rangle)\ t)\downarrow_\beta
\end{array}
$$

Because of Fact 1 on page 6 it is obvious that

$$(\varphi t)\downarrow_\beta = (\{F_{n-1}\mapsto t_{n-1}\} \circ \ldots \circ \{F_0\mapsto t_0\}\ t)\downarrow_\beta = (\{F_{n-1}\mapsto t_{n-1}\}(\ldots(\{F_0\mapsto t_0\}\ t)\downarrow_\beta \cdots)\downarrow_\beta)\downarrow_\beta$$

It is easy to see that (again with the actual substitution on the lhs, its representation on the rhs)

$$(\varphi t)\downarrow_\beta = b_subst\ \varphi\ t \qquad (**)$$

Motivated by $(*)$ and $(**)$ we abbreviate '$subst\ \varphi\ t$' as 'φt' and '$b_subst\ \varphi\ t$' as '$(\varphi t)\downarrow_\beta$'.

Formulating *proj*, *cases* and *unif* with substitution representations instead of substitutions, abstracting from the represenation only in case of application by using *subst* and *b_subst*, we can obviously construct a version which is isomorphic to $unif_4$ and claim correctness by construction. Using the abbreviations given above and interpreting '$\{F\mapsto t\}$' as '$\langle F, t\rangle$' and '$\{F\mapsto t\} \circ \varphi$' as '$\varphi +\!\!+ [\langle F, t\rangle]$', these new versions also look exactely like $proj_4$, $cases_4$ and $unif_4$. Therefore we continue the development with version four of the algorithm, using the isomorphism between substitutions and their representations whenever needed.

6.2 Partial substitution and β-reduction by *devar*

6.2.1 Definition of *devar*

Function *devar* takes the representation of a substitution φ and a term t. It ignores leading lambdas of t, only examining the top-level functor in t. If the functor is a free variable F, *devar* searches φ for a substitution of F. If there is a binding $\{F\mapsto t'\}$ in φ, the corresponding substitution is carried out (note that *not* necessarily $(\varphi F) = t'$ — following bindings might instantiate free variables in t'). Function *red* carries out all possible β-reductions on a β-redex created by the substitution. Then *devar* is applied recursively to the result with the remaining substitution as argument.

$$
\begin{array}{l}
devar\ \varphi\ t = \textbf{case}\ t\ \textbf{of} \\
\qquad\qquad | \ \lambda t' \to \lambda(devar\ \varphi\ t') \\
\qquad\qquad | \ F(\!|\overline{x_n}|\!) \to \textbf{case}\ lookup\ \varphi\ F\ \textbf{of} \\
\qquad\qquad\qquad\qquad | \ \mathsf{None} \ \to t \\
\qquad\qquad\qquad\qquad | \ \llcorner\langle s, \sigma\rangle\lrcorner \to devar\ \sigma\ (red\ s\ \overline{x_n}) \\
\qquad\qquad | \ \textbf{otherwise} \to t \\
red\ t\ yys = \textbf{case}\ yys\ \textbf{of} \\
\qquad\qquad | \ [] \to t \\
\qquad\qquad | \ y\textbf{:}ys \to \textbf{case}\ t\ \textbf{of} \\
\qquad\qquad\qquad\qquad | \ \lambda t' \to red\ (t'[\underline{0}\backslash y])\ ys \\
\qquad\qquad\qquad\qquad | \ g(\!|\overline{s_n}|\!) \to g(\!|\overline{s_n} +\!\!+ yys|\!)
\end{array}
$$

6.2.2 Relation between *devar* and full substitution

The relation between *devar* and a full substitution is expressed by the following proposition:

Lemma 9. *If t is in β-normal form and ψ is a constructed substitution then there are h, $\overline{t'_l}$ and $\overline{s'_l}$ such that*

$$
\begin{array}{rcl}
devar\ \psi\ t & = & \lambda_m\ h(\!|\overline{t'_l}|\!) \\
(\psi t)\downarrow_\beta & = & \lambda_m\ h(\!|\overline{s'_l}|\!) \\
(\psi(\lambda_m\ h(\!|\overline{t'_l}|\!)))\downarrow_\beta & = & \lambda_m\ h(\!|\overline{s'_l}|\!)
\end{array}
$$

This lemma actually is a corollary of a more complicated lemma comparing *devar* and full substitution:

Lemma 10. *Let θ, ψ be two substitutions such that $\psi \circ \theta$ is a constructed substitution. Then for terms $\lambda_m\ g(\overline{[\![t_k]\!]})$ and $\lambda_m\ g(\overline{[\![s_k]\!]})$ with $(\theta(\lambda_m\ g(\overline{[\![t_k]\!]})))\downarrow_\beta = (\lambda_m\ g(\overline{[\![s_k]\!]}))$, there are h, $\overline{t'_l}$ and $\overline{s'_l}$ such that*

$$\begin{aligned} \mathit{devar}\ \psi\ \lambda_m\ g(\overline{[\![t_k]\!]}) &= \lambda_{m+j}\ h(\overline{[\![t'_l]\!]}) \\ (\psi(\lambda_m\ g(\overline{[\![s_k]\!]})))\downarrow_\beta &= \lambda_{m+j}\ h(\overline{[\![s'_l]\!]}) \\ ((\psi \circ \theta)(\lambda_{m+j}\ h(\overline{[\![t'_l]\!]})))\downarrow_\beta &= \lambda_{m+j}\ h(\overline{[\![s'_l]\!]}) \end{aligned}$$

As a commutative diagram, writing the application of a substiution φ with a following reduction to β-normal form as '$\varphi\downarrow_\beta$', this spells out to

$$\begin{array}{ccc} \lambda_m\ g(\overline{[\![t_k]\!]}) & \xrightarrow{\mathit{devar}\ \psi} & \lambda_{m+j}\ h(\overline{[\![t'_l]\!]}) \\ \Big\downarrow{\scriptstyle \theta\downarrow_\beta} & & \Big\downarrow{\scriptstyle (\psi\circ\theta)\downarrow_\beta} \\ \lambda_m\ g(\overline{[\![s_k]\!]}) & \xrightarrow{\psi\downarrow_\beta} & \lambda_{m+j}\ h(\overline{[\![s'_l]\!]}) \end{array}$$

Assuming Lemma 10, it is easy to show Lemma 9

Proof. Use proposition 10 with $\theta = \emptyset$:

$$\begin{array}{ccc} t & \xrightarrow{\mathit{devar}\ \psi} & \lambda_m\ h(\overline{[\![t'_l]\!]}) \\ \Big\downarrow{\scriptstyle \emptyset\downarrow_\beta} & & \Big\downarrow{\scriptstyle \psi\downarrow_\beta} \\ t & \xrightarrow{\psi\downarrow_\beta} & \lambda_m\ h(\overline{[\![s'_l]\!]}) \end{array}$$

□

We now show Lemma 10.

Proof. Let θ, ψ be two substiutions such that $\psi \circ \theta$ is a constructed substitution. Suppose that ψ has the form $\{G_{n-1} \mapsto r_{n-1}\} \circ \ldots \circ \{G_0 \mapsto r_0\}$. Since $\psi \circ \theta$ is a constructed substitution, it is

- $\forall F \in \mathcal{FV}(r_{n-1}, \ldots, r_0).\theta F = F$
- $G_i \notin \mathcal{FV}(r_{n-1}, \ldots, r_i)$ for $0 \leqslant i \leqslant n-1$
- r_i is in β-normal form for $0 \leqslant i \leqslant n-1$

induction over length of ψ

$\psi = \emptyset$ $\Rightarrow$ the list-representation of ψ is []. Using the definition of *devar* on the page before it is easy to see that

$$\mathit{devar}\ []\ \lambda_m\ g(\overline{[\![t_k]\!]}) = \lambda_m\ g(\overline{[\![t_k]\!]})$$

and of course, since $\lambda_m\ g(\overline{[\![s_k]\!]})$ obviously is in β-normal form,

$$(\emptyset(\lambda_m\ g(\overline{[\![s_k]\!]})))\downarrow_\beta = \lambda_m\ g(\overline{[\![s_k]\!]})$$

By assumption $(\theta(\lambda_m\ g(\overline{[\![t_k]\!]})))\downarrow_\beta = \lambda_m\ g(\overline{[\![s_k]\!]})$. Since $\theta = (\emptyset \circ \theta) = (\psi \circ \theta)$ it is

$$((\psi \circ \theta)(\lambda_m\ g(\overline{[\![t_k]\!]})))\downarrow_\beta = \lambda_m\ g(\overline{[\![s_k]\!]})$$

$\psi = \psi' \circ \{G_0 \mapsto r_0\}$ $\Rightarrow$ the list representation of ψ is $\langle G_0, r_0\rangle{:}\psi'$

induction over the structure of $\lambda_m\ g(\overline{t_k})$

$m > 0$ $\Rightarrow$ there is a leading lambda. Then by definition of *devar* on page 39:

$$devar\ \psi\ \lambda_m\ g(\overline{t_k}) = \lambda\ (devar\ \psi\ \lambda_{m-1}\ g(\overline{t_k}))$$

Also:

$$(\psi(\lambda_m\ g(\overline{s_k})))\downarrow_\beta = \lambda\ ((\psi(\lambda_{m-1}\ g(\overline{s_k})))\downarrow_\beta)$$

Use the induction hypothesis of the inner induction.

$m = 0$ $\Rightarrow$ the terms are in functor-argument form: $g(\overline{t_k})$ and $g(\overline{s_k})$

case distinction over g

$g = G_0$ $\Rightarrow$ since we are dealing with patterns, $\overline{t_k}$ and $\overline{s_k}$ only can be a list of bound variables $\overline{x_k}$.
With definition of *devar* on page 39:

$$devar\ \langle G_0, r_0\rangle\ (G_0(\overline{x_k})) = red\ r_0\ \overline{x_k} \tag{1}$$

Also:

$$(\{G_0 \mapsto r_0\}(G_0(\overline{x_k})))\downarrow_\beta = (r_0(\overline{x_k}))\downarrow_\beta \tag{2}$$

Since $\forall F \in \mathcal{FV}(r_0).\theta F = F$ and $G_0 \notin \mathcal{FV}(r_0)$ by assumption, it is

$$((\{G_0 \mapsto r_0\} \circ \theta)(r_0(\overline{x_k}))\downarrow_\beta)\downarrow_\beta = (r_0(\overline{x_k}))\downarrow_\beta \tag{3}$$

Suppose that $(r_0(\overline{x_k}))\downarrow_\beta$ has the form $\lambda_{m'}\ f(\overline{r'_{k'}})$. With Lemma 11 on page 43 we conclude that $(r_0(\overline{x_k}))\downarrow_\beta = red\ r_0\ \overline{x_k}$, so

$$\lambda_{m'}\ f(\overline{r'_{k'}}) = (r_0(\overline{x_k}))\downarrow_\beta = red\ r_0\ \overline{x_k} \tag{4}$$

The equations 1—4 and the fact that $\theta G_0 = G_0$ (since $\psi' \circ \{G_0 \mapsto r_0\} \circ \theta$ is a constructed substitution) give rise to the following commutative diagram

$$\begin{array}{ccc} G_0(\overline{x_k}) & \xrightarrow{devar\ \langle G_0, r_0\rangle} & \lambda_{m'}\ f(\overline{r'_{k'}}) \\ \Big\downarrow{\scriptstyle \theta\downarrow_\beta} & & \Big\downarrow{\scriptstyle (\{G_0 \mapsto r_0\} \circ \theta)\downarrow_\beta} \\ G_0(\overline{x_k}) & \xrightarrow{\{G_0 \mapsto r_0\}\downarrow_\beta} & \lambda_{m'}\ f(\overline{r'_{k'}}) \end{array}$$

The induction hypothesis of the outer induction can be used:

$$\begin{array}{ccc} \lambda_{m'}\ f(\overline{r'_{k'}}) & \xrightarrow{devar\ \psi'} & \lambda_{m'+j}\ h(\overline{t'_l}) \\ \Big\downarrow{\scriptstyle (\{G_0 \mapsto r_0\} \circ \theta)\downarrow_\beta} & & \Big\downarrow{\scriptstyle (\psi \circ \theta)\downarrow_\beta} \\ \lambda_{m'}\ f(\overline{r'_{k'}}) & \xrightarrow{\psi'\downarrow_\beta} & \lambda_{m'+j}\ h(\overline{s'_l}) \end{array}$$

The two diagrams above obviously can be joined:

$$\begin{array}{ccccc} G_0(\overline{x_k}) & \xrightarrow{devar\ \langle G_0, r_0\rangle} & \lambda_{m'}\ f(\overline{r'_{k'}}) & \xrightarrow{devar\ \psi'} & \lambda_{m'+j}\ h(\overline{t'_l}) \\ \Big\downarrow{\scriptstyle \theta\downarrow_\beta} & & \Big\downarrow{\scriptstyle (\{G_0 \mapsto r_0\} \circ \theta)\downarrow_\beta} & & \Big\downarrow{\scriptstyle (\psi \circ \theta)\downarrow_\beta} \\ G_0(\overline{x_k}) & \xrightarrow{\{G_0 \mapsto r_0\}\downarrow_\beta} & \lambda_{m'}\ f(\overline{r'_{k'}}) & \xrightarrow{\psi'\downarrow_\beta} & \lambda_{m'+j}\ h(\overline{s'_l}) \end{array}$$

With Lemma 12 on the next page and Fact 1 on page 6 we conclude that

$$\begin{aligned}(\mathit{devar}\ \psi') \circ (\mathit{devar}\ \langle G_0, r_0\rangle) &= \mathit{devar}\ \psi \\ \psi'{\downarrow_\beta} \circ \{G_0 \mapsto r_0\}{\downarrow_\beta} &= \psi{\downarrow_\beta}\end{aligned}$$

Since the left square and the right square both commute, the outer square also commutes and we receive the following commutative diagram:

$$\begin{array}{ccc} G_0(\!|\overline{x_k}|\!) & \xrightarrow{\mathit{devar}\ \psi} & \lambda_{m+j}\ h(\!|\overline{t'_l}|\!) \\ \Big\downarrow{\scriptstyle \theta{\downarrow_\beta}} & & \Big\downarrow{\scriptstyle (\psi \circ \theta){\downarrow_\beta}} \\ G_0(\!|\overline{x_k}|\!) & \xrightarrow{\psi{\downarrow_\beta}} & \lambda_{m+j}\ h(\!|\overline{s'_l}|\!) \end{array}$$

$g = G$ and $G \neq G_0$ again $\overline{t_k}$ and $\overline{s_k}$ only can be a list of bound variables $\overline{x_k}$.
With definition of *devar* on page 39:

$$\mathit{devar}\ \langle G_0, r_0\rangle\ G(\!|\overline{x_k}|\!) = G(\!|\overline{x_k}|\!) \tag{5}$$

Also:

$$(\{G_0 \mapsto r_0\} G(\!|\overline{x_k}|\!)){\downarrow_\beta} = (G(\!|\overline{x_k}|\!)){\downarrow_\beta} = G(\!|\overline{x_k}|\!) \tag{6}$$

Equations (5) and (6) give rise to the commutative diagram

$$\begin{array}{ccc} G(\!|\overline{x_k}|\!) & \xrightarrow{\mathit{devar}\ \langle G_0, r_0\rangle} & G(\!|\overline{x_k}|\!) \\ \Big\downarrow{\scriptstyle \theta{\downarrow_\beta}} & & \Big\downarrow{\scriptstyle (\{G_0 \mapsto r_0\} \circ \theta){\downarrow_\beta}} \\ G(\!|\overline{x_k}|\!) & \xrightarrow{\{G_0 \mapsto r_0\}{\downarrow_\beta}} & G(\!|\overline{x_k}|\!) \end{array}$$

The induction hypothesis of the outer induction can be used:

$$\begin{array}{ccc} G(\!|\overline{x_k}|\!) & \xrightarrow{\mathit{devar}\ \psi'} & \lambda_j\ h(\!|\overline{t'_l}|\!) \\ \Big\downarrow{\scriptstyle (\{G_0 \mapsto r_0\} \circ \theta){\downarrow_\beta}} & & \Big\downarrow{\scriptstyle (\psi \circ \theta){\downarrow_\beta}} \\ G(\!|\overline{x_k}|\!) & \xrightarrow{\psi'{\downarrow_\beta}} & \lambda_j\ h(\!|\overline{s'_l}|\!) \end{array}$$

Joining the two commutative diagrams like in the previous case concludes this case.
$g = p$ The definition of *devar* on page 39 shows that *devar* leaves $p(\!|\overline{t_k}|\!)$ unchanged. Therefore:

$$\mathit{devar}\ \langle G_0, r_0\rangle\ p(\!|\overline{t_k}|\!) = p(\!|\overline{t_k}|\!) \tag{7}$$

Also:

$$(\{G_0 \mapsto r_0\} p(\!|\overline{s_k}|\!)){\downarrow_\beta} = p(\!|(\{G_0 \mapsto r_0\}\overline{s_k}){\downarrow_\beta}|\!) \tag{8}$$

Consider $((\{G_0 \mapsto r_0\} \circ \theta)(p(\!|\overline{t_k}|\!))){\downarrow_\beta} = p(\!|((\{G_0 \mapsto r_0\} \circ \theta)\overline{t_k}){\downarrow_\beta}|\!)$
$=$ ⟨with Fact 1 on page 6⟩
$p(\!|(\{G_0 \mapsto r_0\}(\theta\overline{t_k}){\downarrow_\beta}){\downarrow_\beta}|\!)$
$=$ ⟨by assumption $(\theta(p(\!|\overline{t_k}|\!))){\downarrow_\beta} = p(\!|\overline{s_k}|\!)$, thus $(\theta\overline{t_k}){\downarrow_\beta} = \overline{s_k}$⟩
$p(\!|(\{G_0 \mapsto r_0\}\overline{s_k}){\downarrow_\beta}|\!)$
So equations (7) and (8) give rise to the following commutative diagram:

$$\begin{array}{ccc} p(\!|\overline{t_k}|\!) & \xrightarrow{\mathit{devar}\ \langle G_0, r_0\rangle} & p(\!|\overline{t_k}|\!) \\ \Big\downarrow{\scriptstyle \theta{\downarrow_\beta}} & & \Big\downarrow{\scriptstyle (\{G_0 \mapsto r_0\} \circ \theta){\downarrow_\beta}} \\ p(\!|\overline{s_k}|\!) & \xrightarrow{\{G_0 \mapsto r_0\}{\downarrow_\beta}} & p(\!|(\{G_0 \mapsto r_0\}\overline{s_k}){\downarrow_\beta}|\!) \end{array}$$

The induction hypothesis of the outer induction can be used:

$$\begin{array}{ccc} p(\!|\overline{t_k}|\!) & \xrightarrow{\quad devar\ \psi' \quad} & \lambda_j\ h(\!|\overline{t'_l}|\!) \\ \Big\downarrow (\{G_0{\mapsto}r_0\}\circ\theta)\!\downarrow_\beta & & \Big\downarrow (\psi\circ\theta)\!\downarrow_\beta \\ p(\!|(\{G_0{\mapsto}r_0\}\overline{s_k})\!\downarrow_\beta|\!) & \xrightarrow{\quad \psi'\!\downarrow_\beta \quad} & \lambda_j\ h(\!|\overline{s'_l}|\!) \end{array}$$

Joining the two commutative diagrams like in the cases before concludes this case.

□

The following two lemmas about *red* and *devar* were used in the proof of Lemma 10:

Lemma 11. *If r is in β-normal form, then* $red\ r\ \overline{x_k} = (r(\!|\overline{x_k}|\!))\!\downarrow_\beta$

Proof. Assume that r is of the form $\lambda_j\ h(\!|\overline{r_n}|\!)$.

induction over j

$j = 0$ By definition of *red* on page 39 it is

$$red\ h(\!|\overline{r_n}|\!)\ \overline{x_k} = h(\!|\overline{r_n} +\!\!+ \overline{x_k}|\!)$$

Because $(h(\!|\overline{r_n}|\!))(\!|\overline{x_k}|\!) = h(\!|\overline{r_n} +\!\!+ \overline{x_k}|\!)$ by definition and $h(\!|\overline{r_n}|\!)$ is in β-normal form, we have

$$((h(\!|\overline{r_n}|\!))(\!|\overline{x_k}|\!))\!\downarrow_\beta = (h(\!|\overline{r_n} +\!\!+ \overline{x_k}|\!))\!\downarrow_\beta = h(\!|\overline{r_n} +\!\!+ \overline{x_k}|\!)$$

$j > 0$ With definition of *red* on page 39:

$$red\ (\lambda_j\ h(\!|\overline{r_n}|\!))\ \overline{x_k} = \begin{cases} \lambda_j\ h(\!|\overline{r_n}|\!) & \text{if } k = 0 \\ red\ ((\lambda_{j-1}\ h(\!|\overline{r_n}|\!))[0\backslash x_{k-1}])\ \overline{x_{k-1}} & \text{if } k > 0 \end{cases}$$

For the β-reduction we have (since $\lambda_j\ h(\!|\overline{r_n}|\!)$ is in β-normal form)

$$((\lambda_j\ h(\!|\overline{r_n}|\!))(\!|\overline{x_k}|\!))\!\downarrow_\beta = \begin{cases} \lambda_j\ h(\!|\overline{r_n}|\!) & \text{if } k = 0 \\ (((\lambda_{j-1}\ h(\!|\overline{r_n}|\!))[0\backslash x_{k-1}])(\!|\overline{x_{k-1}}|\!))\!\downarrow_\beta & \text{if } k > 0 \end{cases}$$

So for $k = 0$ the equality holds. For $k > 0$ the induction hypothesis can be used, since '$[0\backslash x_{k-1}]$' (definition on page 52) does not affect the number of leading lambdas in $\lambda_{j-1}\ h(\!|\overline{r_n}|\!)$ and does not introduce β-redexes.

□

Lemma 12. $devar\ (\langle G, s\rangle{:}\varphi)\ t = ((devar\ \varphi)\circ(devar\ \langle G, s\rangle))\ t$

Proof. structural induction over t

$t = \lambda\, t'$ $devar\ (\langle G, s\rangle{:}\varphi)\ \lambda\ t'$

$= \langle$definition of *devar* on page 39$\rangle$

$\lambda\ (devar\ (\langle G, s\rangle{:}\varphi)\ t')$

$= \langle$induction hypothesis$\rangle$

$\lambda\ (((devar\ \varphi)\circ(devar\ \langle G, s\rangle))\ t') = ((devar\ \varphi)\circ(devar\ \langle G, s\rangle))\ \lambda\ t'$

$t = f(\!|\overline{s_n}|\!)$ case distinction over $f(\!|\overline{s_n}|\!)$

$f(\!|\overline{s_n}|\!) = F(\!|\overline{x_n}|\!)$ case distinction over F

$F \neq G$ $devar\ (\langle G, s\rangle{:}\varphi)\ F(\!|\overline{x_n}|\!)$

$= \langle$definition of *devar*$\rangle$

$devar\ \varphi\ F(\!|\overline{x_n}|\!) = devar\ \varphi\ (devar\ \langle G, s\rangle\ F(\!|\overline{x_n}|\!)) = (devar\ \varphi)\circ(devar\ \langle G, s\rangle)\ F(\!|\overline{x_n}|\!)$

$F = G$ $devar\ (\langle G, s\rangle{:}\varphi)\ G(\!|\overline{x_n}|\!)$

$= \langle$definition of *devar*$\rangle$

$devar\ \varphi\ (red\ s\ \overline{x_n}) = devar\ \varphi\ (devar\ \langle G, s\rangle\ G(\!|\overline{x_n}|\!)) = (devar\ \varphi)\circ(devar\ \langle G, s\rangle)\ G(\!|\overline{x_n}|\!)$

$f(\!|\overline{s_n}|\!) = p(\!|\overline{s_n}|\!)$ again with the definition of *devar*:

$devar\ (\langle G, s\rangle{:}\varphi)\ p(\!|\overline{s_n}|\!) = p(\!|\overline{s_n}|\!) = (devar\ \varphi)\circ(devar\ \langle G, s\rangle)\ p(\!|\overline{s_n}|\!)$

□

6.2.3 The occurs-check

Since we will be dealing with partially instantiated terms, the condition $F \in \mathcal{FV}(\overline{s_n})$ in the (flex-rigid)-branch will have to be modified. The occurs-check *occ* will check for occurences of a free variable F in a partially instantiated term.

$$
\begin{array}{l}
occ\ F\ \varphi\ t = \textbf{case}\ t\ \textbf{of} \\
\quad \mid G \to (G = F) \\
\quad\ \ \textbf{orelse case}\ lookup\ \varphi\ G\ \textbf{of} \\
\quad\quad \mid \mathsf{None} \ \to\ \textbf{false} \\
\quad\quad \mid \llcorner\langle s, \sigma\rangle\lrcorner \to occ\ F\ \sigma\ s \\
\quad \mid \lambda t' \to occ\ F\ \varphi\ t' \\
\quad \mid r\ @\ s \to (occ\ F\ \varphi\ r)\ \textbf{orelse}\ (occ\ F\ \varphi\ s) \\
\quad \mid \textbf{otherwise} \to\ \textbf{false}
\end{array}
$$

The following lemma clarifies the role of the occurs-check:

Lemma 13. *If* $\theta = \{G_{n-1}{\mapsto}r_{n-1}\} \circ \ldots \circ \{G_0{\mapsto}r_0\}$ *and for* $0 \leqslant i \leqslant (n-1)$ *it is* $G_i \neq F$ *then* $occ\ F\ \theta\ t$ *iff* $F \in \mathcal{FV}(\theta t)$

Proof. induction over the length of θ

$\theta = \emptyset$ $\Rightarrow$ list-representation of θ is []:

structural induction over t

$t = G$ If $G = F$ then by definition of *occ* on this page it is $occ\ F\ \emptyset\ F =$ **true**. But $\mathcal{FV}(\emptyset F) = \mathcal{FV}(F) = \{F\}$ and $F \in \{F\}$. So suppose $G \neq F$. But then $occ\ F\ \emptyset\ G =$ **false**, $\mathcal{FV}(\emptyset G) = \mathcal{FV}(G) = \{G\}$ and $F \notin \{G\}$

$t = \lambda\ t'$ By definition of *occ* on the current page it is

$$occ\ F\ \theta\ (\lambda\ t') = occ\ F\ \theta\ t'$$

Since $\mathcal{FV}(\theta(\lambda\ t')) = \mathcal{FV}(\lambda\ \theta t') = \mathcal{FV}(\theta t')$, the induction hypothesis can be used.

$t = r\ @\ s$ It is

$$
\begin{array}{rcl}
occ\ F\ \theta\ (r\ @\ s) & = & (occ\ F\ \theta\ r)\textbf{orelse}(occ\ F\ \theta\ s) \\
\mathcal{FV}(\theta(r\ @\ s)) & = & \mathcal{FV}(\theta r\ @\ \theta s) = \mathcal{FV}(\theta r) \cup \mathcal{FV}(\theta s)
\end{array}
$$

Use the induction hypothesis.

$t = x$ $occ\ F\ \theta\ x =$ **false** and $\mathcal{FV}(\theta x) = \mathcal{FV}(x) = \emptyset$

$\theta = (\theta' \circ \{G_0{\mapsto}r_0\})$ $\Rightarrow$ the list-representation of θ is $\langle G_0, r_0\rangle$**:**θ'

We prove by structural induction over t. The cases $t = \lambda\ t'$, $t = r\ @\ s$ and $t = x$ are solved exactely like in the base case $\theta = \emptyset$: The fact that θ was the empty substitution has not been used. The case $t = G$ however is different:

case distinction over G

$G = F$ $occ\ F\ \theta\ F = (F = F) =$ **true** and since by assumption $\theta F = F$, we have $\mathcal{FV}(\theta F) = \mathcal{FV}(F) = \{F\}$

$G = G_0$ with the definition of *occ* on this page and of *lookup* on page 7:

$$occ\ F\ \langle G_0, r_0\rangle\textbf{:}\theta'\ G_0 = occ\ F\ \theta'\ r_0$$

Use the induction hypothesis of the outer induction. Since $\mathcal{FV}(\theta' \circ \{G_0{\mapsto}r_0\}G_0) = \mathcal{FV}(\theta' r_0)$, this concludes the proof.

$G \neq F$ and $G \neq G_0$ with definition of *occ* on this page and *lookup* on page 7:

$$occ\ F\ \langle G_0, r_0\rangle\textbf{:}\theta'\ G = occ\ F\ \theta'\ G$$

Use the outer induction hypothesis. Since $\mathcal{FV}(\theta' \circ \{G_0{\mapsto}r_0\}G) = \mathcal{FV}(\theta' G)$, this concludes the proof.

□

6.3 Definition of algorithm with partial substitution

The algorithm now is rewritten with '*devar* θ' wherever the application of θ with a following β-reduction occured in the last version.

6.3.1 Definition of $proj_5$

Function $proj_4$ is transformed to $proj_5$ by using *devar* instead of actual application of a substitution. Since $proj_5$ will receive only partially instantiated terms, the calculated substitution θ has to be passed along. Note that substitutions θ and φ have to be distinct arguments and cannot be combined to a single argument $\varphi \circ \theta$. In this case $proj_5$ would return a substitution $\varphi' \circ \theta$, but it is φ' which is needed in $cases_5$.

$$
\begin{array}{l}
proj_5\ \theta\ zs\ \langle\varphi, t\rangle = \textbf{case}\ t\ \textbf{of} \\
\quad \mid \lambda t' \rightarrow \textbf{let}\ res\ = proj_5\ \theta\ ((map\ (\lambda x.lift\ x\ 0)\ zs) +\!\!+ [\underline{0}])\ \langle\varphi, t'\rangle \\
\qquad \textbf{in case}\ res\ \textbf{of} \\
\qquad\quad \mid \mathsf{None} \rightarrow \mathsf{None} \\
\qquad\quad \mid \llcorner\langle\varphi', t''\rangle\lrcorner \rightarrow \llcorner\langle\varphi', \lambda t''\rangle\lrcorner \\
\quad \mid p(\!|\overline{s_m}|\!) \rightarrow \textbf{case}\ idx\ zs\ p\ \textbf{of} \\
\qquad \mid \mathsf{None} \rightarrow \mathsf{None} \\
\qquad \mid \llcorner j\lrcorner \rightarrow \textbf{let}\ res\ = lproj_5\ \theta\ zs\ \langle\varphi, \overline{s_m}\rangle \\
\qquad\qquad \textbf{in case}\ res\ \textbf{of} \\
\qquad\qquad\quad \mid \mathsf{None} \rightarrow \mathsf{None} \\
\qquad\qquad\quad \mid \llcorner\langle\varphi', \overline{s'_m}\rangle\lrcorner \rightarrow \llcorner\langle\varphi', \underline{j}(\!|\overline{s'_m}|\!)\rangle\lrcorner \\
\quad \mid F(\!|\overline{x_m}|\!) \rightarrow \textbf{let}\ \overline{x'_m} = map\ (idx\ zs)\ \overline{x_m} \\
\qquad\qquad\quad \varphi' = \{F \mapsto \lambda_m\ H(\!|[\underline{k} \mid 0 \leqslant k \leqslant (m-1), x'_k = \llcorner_\lrcorner]|\!)\} \circ \varphi \\
\qquad\qquad\quad t' = H(\!|[\underline{l} \mid 0 \leqslant k \leqslant (m-1), x'_k = \llcorner l\lrcorner]|\!) \\
\qquad\quad \textbf{in}\ \llcorner\langle\varphi', t'\rangle\lrcorner
\end{array}
$$

$$
\begin{array}{l}
lproj_5\ \theta\ zs\ \langle\varphi, tts\rangle = \textbf{case}\ tts\ \textbf{of} \\
\quad \mid [] \rightarrow \llcorner\langle\varphi, []\rangle\lrcorner \\
\quad \mid t\textbf{:}ts \rightarrow \textbf{let}\ res_1\ = proj_5\ \theta\ zs\ \langle\varphi, devar\ (\varphi \circ \theta)\ t\rangle \\
\qquad\quad \textbf{in case}\ res_1\ \textbf{of} \\
\qquad\qquad \mid \mathsf{None} \rightarrow \mathsf{None} \\
\qquad\qquad \mid \llcorner\langle\varphi', t'\rangle\lrcorner \rightarrow \textbf{let}\ res_2\ = lproj_5\ \theta\ zs\ \langle\varphi', ts\rangle \\
\qquad\qquad\qquad\quad \textbf{in case}\ res_2\ \textbf{of} \\
\qquad\qquad\qquad\quad \mid \mathsf{None} \rightarrow \mathsf{None} \\
\qquad\qquad\qquad\quad \mid \llcorner\langle\varphi'', ts'\rangle\lrcorner \rightarrow \llcorner\langle\varphi'', t'\textbf{:}ts'\rangle\lrcorner
\end{array}
$$

6.3.2 Definition of $cases_5$

Function $cases_5$ also takes the calculated substitution as an additional argument. It is needed in order to carry out the occurs-check and to use $proj_5$ in the branch (flex-rigid).

$$
\begin{array}{ll}
cases_5\ \theta\ (p(\!|\overline{s_n}|\!) \stackrel{?}{=} p(\!|\overline{t_n}|\!)) = \llcorner\langle[s_{n-1} \stackrel{?}{=} t_{n-1}, \ldots, s_0 \stackrel{?}{=} t_0], \emptyset\rangle\lrcorner & \text{(rigid-rigid)} \\
cases_5\ \theta(F(\!|\overline{x_m}|\!) \stackrel{?}{=} p(\!|\overline{s_n}|\!)) = \textbf{if}\ occ\ F\ \theta\ p(\!|\overline{s_n}|\!) & \text{(flex-rigid)} \\
\qquad\qquad \textbf{then}\ \mathsf{None} \\
\qquad\qquad \textbf{else let}\ res\ = proj_5\ \theta\ \overline{x_m}\ \langle\emptyset, p(\!|\overline{s_n}|\!)\rangle \\
\qquad\qquad\quad \textbf{in case}\ res\ \textbf{of} \\
\qquad\qquad\quad \mid \mathsf{None} \rightarrow \mathsf{None} \\
\qquad\qquad\quad \mid \llcorner\langle\psi, t\rangle\lrcorner \rightarrow \llcorner\langle[], \psi \circ \{F \mapsto \lambda_m\ t\}\rangle\lrcorner \\
cases_5\ \theta\ (F(\!|\overline{x_m}|\!) \stackrel{?}{=} F(\!|\overline{y_m}|\!)) = \llcorner\langle[], \{F \mapsto \lambda_m\ H(\!|zs|\!)\}\rangle\lrcorner & \text{(flex-flex-1)}
\end{array}
$$

$$\begin{array}{l} \qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{where } zs = [\underline{k} \mid 0 \leqslant k \leqslant (m-1), x_k = y_k] \\ cases_5\ \theta\ (F(\overline{x_m}) \stackrel{?}{=} G(\overline{y_n})) = \llcorner\langle [], \{F \mapsto \lambda_m\ H(xs)\} \circ \{G \mapsto \lambda_n\ H(ys)\}\rangle\lrcorner \qquad \text{(flex-flex-2)} \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{where } \overline{y'_n} = map\ (idx\ \overline{x_m})\overline{y_n}, \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad xs = [j \mid 0 \leqslant k \leqslant (n-1), y'_k = \llcorner j \lrcorner], \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad ys = [k \mid 0 \leqslant k \leqslant (n-1), y'_k = \llcorner _ \lrcorner] \end{array}$$

6.3.3 Definition of $unif_5$

The function $unif_5$ uses *devar* where $unif_4$ applied the calculated substitution to the first unification problem. $cases_5$ receives the calculated substitution as argument.

$$\begin{array}{l} unif_5\ \langle uus, \varphi\rangle = \textbf{case } uus \textbf{ of} \\ \qquad\qquad | \ [] \rightarrow \llcorner\varphi\lrcorner \\ \qquad\qquad | \ u{:}us \rightarrow \textbf{case } ((cases_5\ \varphi) \circ tw \circ otf\eta)\ u \textbf{ of} \\ \qquad\qquad\qquad\qquad | \ \mathsf{None} \ \rightarrow \mathsf{None} \\ \qquad\qquad\qquad\qquad | \ \llcorner\langle us', \varphi'\rangle\lrcorner \rightarrow unif_5\ \langle head_ap\ (devar\ \varphi' \circ \varphi)(us' +\!\!+ us), \varphi' \circ \varphi\rangle \end{array}$$

6.4 Proof of correctness

6.4.1 Relation between $proj_4$ and $proj_5$

We prove equality for the results of $unif_4$ and $unif_5$ under the condition that their arguments are in a certain relation. So like in the proof of correctness for $unif_2$ we have to make sure that in the calculations carried out by $unif_4$ and $unif_5$ the same new variables are introduced. We will assume this fact for the following proofs; it can be shown like in the correctness proof for $unif_2$.

Proposition 14.
Let θ, ψ be two substitutions such that $\psi \circ \theta$ is a constructed substitution and

$$((\psi \circ \theta)(\lambda_k\ g(\overline{t_l})))\downarrow_\beta = (\psi(\lambda_k\ g(\overline{s_l})))\downarrow_\beta$$

for terms $\lambda_m\ g(\overline{t_l})$ and $\lambda_m\ g(\overline{s_l})$ in β-normal form. Then

$$proj_4\ \overline{x_m}\ \langle\psi, \lambda_k\ g(\overline{s_l})\rangle = proj_5\ \theta\ \overline{x_m}\ \langle\psi, \lambda_k\ g(\overline{t_l})\rangle$$

Proof. We prove by computational induction over $proj_4$.

case distinction over k

$k \geqslant 0$ With definition of $proj_4$ on page 25 and definition of $proj_5$ on the page before:

$$\begin{array}{l} proj_4\ \overline{x_m}\ \langle\psi, \lambda_k\ g(\overline{s_l})\rangle = \textbf{let } res = proj_4\ ((map\ (\lambda x.lift\ x\ 0)\ \overline{x_m}) +\!\!+ [\underline{0}])\ \langle\psi, \lambda_{k-1}\ g(\overline{s_l})\rangle \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{in } \ldots \end{array}$$

$$\begin{array}{l} proj_5\ \theta\ \overline{x_m}\ \langle\psi, \lambda_k\ g(\overline{t_l})\rangle = \textbf{let } res = proj_5\ \theta\ ((map\ (\lambda x.lift\ x\ 0)\ \overline{x_m}) +\!\!+ [\underline{0}])\ \langle\psi, \lambda_{k-1}\ g(\overline{t_l})\rangle \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{in } \ldots \end{array}$$

Obviously the induction hypothesis can be used.

$k = 0$ case-distinction over g

$g = p$ From the definition of $proj_4$ on page 25 and definition of $proj_5$ on the page before it is easy to see that $proj_4$ and $proj_5$ calculate the same functor q (or both fail in its calculation). So:

$$\begin{array}{l} proj_4\ \overline{x_m}\ \langle\psi, p(\overline{s_l})\rangle = \textbf{let } res = lproj_4\ \overline{x_m}\ \langle\psi, \overline{s_l}\rangle \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{in case } res \textbf{ of} \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad | \ \mathsf{None} \ \rightarrow \mathsf{None} \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad | \ \llcorner\langle\psi', \overline{s'_l}\rangle\lrcorner \rightarrow \langle\psi', q(\overline{s'_l})\rangle \end{array}$$

$$\begin{aligned} proj_5\ \theta\ \overline{x_m}\ \langle\psi, p(\overline{t_l})\rangle = \textbf{let}\ res' &= lproj_5\ \theta\ \overline{x_m}\ \langle\psi, \overline{t_l}\rangle \\ \textbf{in case}\ res'\ \textbf{of}& \\ \mid \mathsf{None} &\to \mathsf{None} \\ \mid \llcorner\langle\psi', \overline{t'_l}\rangle\lrcorner &\to \langle\psi', q(\overline{t'_l})\rangle \end{aligned}$$

Assumption: Let ψ and θ be substitutions such that $\psi \circ \theta$ is a constructed substitution and

$$((\psi \circ \theta)\overline{t_l})\downarrow_\beta = (\psi\overline{s_l})\downarrow_\beta$$

Let further $\overline{t_l}$, $\overline{s_l}$ be in β-normal form. Then

$$lproj_4\ \overline{x_m}\ \langle\psi, \overline{s_l}\rangle = lproj_5\ \theta\ \overline{x_m}\ \langle\psi, \overline{t_l}\rangle$$

Suppose the assumption holds. It obviously can be used on *res* and *res*′, thus closing the argument.

We now prove the assumption:

induction over l:

$l = 0$ obvious

$l \to (l+1)$ with the definition of $lproj_4$ on page 26 and the definition of $lproj_5$ on page 45:

$$\begin{aligned} lproj_4\ \overline{x_m}\ \langle\psi, s_l{:}\overline{s_l}\rangle = \textbf{let}\ res_4 &= proj_4\ \overline{x_m}\ \langle\psi, (\psi s_l)\downarrow_\beta\rangle \\ \textbf{in case}\ res_4\ \textbf{of}& \\ \mid \mathsf{None} &\to \mathsf{None} \\ \mid \llcorner\langle\psi'', s'_l\rangle\lrcorner &\to \textbf{let}\ r_4 = lproj_4\ \overline{x_m}\ \langle\psi'', \overline{s_l}\rangle \\ & \quad \textbf{in case}\ r_4\ \textbf{of} \\ & \quad \mid \mathsf{None} \to \mathsf{None} \\ & \quad \mid \llcorner\langle\psi', \overline{s'_l}\rangle\lrcorner \to \llcorner\langle\psi', s'_l{:}\overline{s'_l}\rangle\lrcorner \end{aligned}$$

$$\begin{aligned} lproj_5\ \theta\ \overline{x_m}\ \langle\psi, t_l{:}\overline{t_l}\rangle = \textbf{let}\ res_5 &= proj_5\ \theta\ \overline{x_m}\ \langle\psi, devar\ (\psi \circ \theta)\ t_l\rangle \\ \textbf{in case}\ res_5\ \textbf{of}& \\ \mid \mathsf{None} &\to \mathsf{None} \\ \mid \llcorner\langle\psi'', t'_l\rangle\lrcorner &\to \textbf{let}\ r_5 = lproj_5\ \theta\ \overline{x_m}\ \langle\psi'', \overline{t_l}\rangle \\ & \quad \textbf{in case}\ r_5\ \textbf{of} \\ & \quad \mid \mathsf{None} \to \mathsf{None} \\ & \quad \mid \llcorner\langle\psi', \overline{t'_l}\rangle\lrcorner \to \llcorner\langle\psi', t'_l{:}\overline{t'_l}\rangle\lrcorner \end{aligned}$$

By assumption it is $((\psi \circ \theta)t_l)\downarrow_\beta = (\psi s_l)\downarrow_\beta$. Suppose that $(\psi s_l)\downarrow_\beta$ is of the form $\lambda_{k'}\ h(\overline{r_n})$. Then $devar\ (\psi \circ \theta)\ t_l = \lambda_{k'}\ h(\overline{r'_n})$ and $((\psi \circ \theta)(\lambda_{k'}\ h(\overline{r'_n})))\downarrow_\beta = \lambda_{k'}\ h(\overline{r_n})$ because of Lemma 9 on page 39. Furthermore $(\psi(\lambda_{k'}\ h(\overline{r_n})))\downarrow_\beta = \lambda_{k'}\ h(\overline{r_n})$ because ψ is a constructed substitution and thus idempotent. So all in all

$$((\psi \circ \theta)(\lambda_{k'}\ h(\overline{r'_n})))\downarrow_\beta = (\psi(\lambda_{k'}\ h(\overline{r_n})))\downarrow_\beta$$

and the induction hypothesis of the outer induction can be used on res_4 and res_5. Thus either $res_4 = res_5 = \mathsf{None}$ or $res_4 = res_5 = \llcorner\langle\psi'', t'_l\rangle\lrcorner$. So in case of success:

$$\begin{aligned} lproj_4\ \overline{x_m}\ \langle\psi, s_l{:}\overline{s_l}\rangle = \textbf{let}\ r_4 &= lproj_4\ \overline{x_m}\ \langle\psi'', \overline{s_l}\rangle \\ \textbf{in case}\ r_4\ \textbf{of}& \\ \mid \mathsf{None} &\to \mathsf{None} \\ \mid \llcorner\langle\psi', \overline{s'_l}\rangle\lrcorner &\to \llcorner\langle\psi', t'_l{:}\overline{s'_l}\rangle\lrcorner \end{aligned}$$

$$\begin{aligned} lproj_5\ \theta\ \overline{x_m}\ \langle\psi, t_l{:}\overline{t_l}\rangle = \textbf{let}\ r_5 &= lproj_5\ \theta\ \overline{x_m}\ \langle\psi'', \overline{t_l}\rangle \\ \textbf{in case}\ r_5\ \textbf{of}& \\ \mid \mathsf{None} &\to \mathsf{None} \\ \mid \llcorner\langle\psi', \overline{t'_l}\rangle\lrcorner &\to \llcorner\langle\psi', t'_l{:}\overline{t'_l}\rangle\lrcorner \end{aligned}$$

Analyzing the way $proj_4$ builds up substitutions we see that there is a φ with $(\varphi \circ \psi) = \psi''$. Since $((\psi \circ \theta)\overline{t_l})\downarrow_\beta = (\psi\overline{s_l})\downarrow_\beta$ we therefore can conclude that

$$\varphi((\psi \circ \theta)\overline{t_l})\downarrow_\beta = \varphi(\psi\overline{s_l})\downarrow_\beta$$

Using Fact 1 on page 6 we see that

$$((\psi'' \circ \theta)\overline{t_l})\downarrow_\beta = (\psi''\overline{s_l})\downarrow_\beta$$

The induction hypothesis of the inner induction can be used. So either $r_4 = r_5 = \mathsf{None}$ or $r_4 = r_5 = \llcorner\langle\psi', \overline{t'_l}\rangle\lrcorner$. In any case it follows easily that indeed $lproj_4\ \overline{x_m}\ \langle\psi, s_l{:}\overline{s_l}\rangle = lproj_5\ \theta\ \overline{x_m}\ \langle\psi, t_l{:}\overline{t_l}\rangle$.

$g = G$ Since we are dealing with patterns, it is $\overline{s_l} = \overline{t_l} = \overline{x_l}$ for some list of bound variables $\overline{x_l}$. So the arguments of $proj_4$ and $proj_5$ are the same. Furthermore $proj_4$ and $proj_5$ have the same definition for this case. Since we assume that the same free variables will be introduced in $proj_4$ and $proj_5$ it follows that they return the same result.

□

6.4.2 Relation between *cases*$_4$ and *cases*$_5$

Proposition 15.

Let $(\theta g(\overline{s'_m}))\downarrow_\beta = g(\overline{s_m})$ *and* $(\theta h(\overline{t'_n}))\downarrow_\beta = h(\overline{t_n})$ *for terms* $g(\overline{s'_m})$, $g(\overline{s_m})$, $h(\overline{t'_n})$ *and* $h(\overline{t_n})$ *in* β*-normal form and a substitution* θ*. Set* $u := (g(\overline{s_m}) \stackrel{?}{=} h(\overline{t_n}))$ *and* $v := (g(\overline{s'_m}) \stackrel{?}{=} h(\overline{t'_n}))$*. Suppose that if only* g *or* h *is a free variable then it is* g*. Then either* $cases_4\ u = (cases_5\ \theta)\ v = \mathsf{None}$ *or* $cases_4\ u = \llcorner\langle us, \psi\rangle\lrcorner$, $(cases_5\ \theta)\ v = \llcorner\langle vs, \psi\rangle\lrcorner$*. In the latter case it is* $us = (\theta vs)\downarrow_\beta$*,* us *and* vs *are in* β*-normal form and* $\theta us = us$*.*

Proof. Since u and v have the same functors on corresponding sides we can prove via

case distinction over branches of *cases*

(rigid-rigid) So $m = n$ and with the definition of $cases_4$ on page 25 and of $cases_5$ on page 45:

$$cases_4\ u = \langle\overbrace{[s_{m-1} \stackrel{?}{=} t_{m-1}, \ldots, s_0 \stackrel{?}{=} t_0]}^{us}, \emptyset\rangle$$

$$(cases_5\ \theta)\ v = \langle\overbrace{[s'_{m-1} \stackrel{?}{=} t'_{m-1}, \ldots, s'_0 \stackrel{?}{=} t'_0]}^{vs}, \emptyset\rangle$$

and obviously $us = (\theta vs)\downarrow_\beta$ since $(\theta g(\overline{s'_m}))\downarrow_\beta = g(\overline{s_m})$ and $(\theta h(\overline{t'_m}))\downarrow_\beta = h(\overline{t_m})$. Further $\theta us = us$ since θ is constructed and therefore idempotent. That us and vs are in β-normal form is obvious.

(flex-rigid) Suppose that $u = (F(\overline{x_m}) \stackrel{?}{=} p(\overline{t_n}))$ and $v = (F(\overline{x_m}) \stackrel{?}{=} p(\overline{t'_n}))$. Firstly we have to make sure that the occurs check $occ\ F\ \theta\ p(\overline{t'_n})$ returns **true** iff $F \in \mathcal{FV}(p(\overline{t_n}))$. Since θ is a constructed substitution and $\theta(F(\overline{x_m})) = F(\overline{x_m})$ by assumption we can conclude that $\theta F = F$. So Lemma 13 on page 44 can be used:

$$occ\ F\ \theta\ p(\overline{t'_n}) \Leftrightarrow F \in \mathcal{FV}(\theta(p(\overline{t'_n})))$$

With Fact 3 on page 6 we see that

$$\mathcal{FV}(\theta(p(\overline{t'_n}))) = \mathcal{FV}((\theta(p(\overline{t'_n})))\downarrow_\beta)$$

and by assumption $(\theta(p(\overline{t'_n})))\downarrow_\beta = p(\overline{t_n})$. Therefore

$$occ\ F\ \theta\ p(\overline{t'_n}) \Leftrightarrow F \in \mathcal{FV}(p(\overline{t_n}))$$

So suppose that $F \notin \mathcal{FV}(p(\overline{t_n}))$. Then

$cases_4\ u =$ **let** $res_4 = proj_4\ \overline{x_m}\ \langle\emptyset, h(\overline{t_n})\rangle$
in case res_4 **of**
| $\mathsf{None} \rightarrow \mathsf{None}$
| $\llcorner\langle\psi, t\rangle\lrcorner \rightarrow \langle[], \psi \circ \{F \mapsto \lambda_m\ t\}\rangle$

$$\begin{array}{l} cases_5\ u = \textbf{let}\ res_5 = proj_5\ \theta\ \overline{x_m}\ \langle\emptyset, h(\overline{[t'_n]})\rangle \\ \qquad\qquad \textbf{in case}\ res_5\ \textbf{of} \\ \qquad\qquad\ \mid \mathsf{None}\ \to \mathsf{None} \\ \qquad\qquad\ \mid \llcorner\langle\psi, t\rangle\lrcorner \to \langle[], \psi \circ \{F \mapsto \lambda_m\ t\}\rangle \end{array}$$

Using Proposition 14 on page 46 we see that $res_4 = res_5$. The proposition obviously holds.

(flex-flex-1) Since only bound variables are in argument position, it is $u = v$. Using the assumption that the same new variables will be introduced in $cases_4$ and $cases_5$ it is obvious that the result is the same. Since in (flex-flex-1) an empty list of new substitution problems is returned, the proposition obviously holds.

(flex-flex-2) same argument as for the previous case.

□

6.4.3 Proving correctness

Proposition 16. *Let two lists uus and vvs of unification problems in β-normal form be such that either $uus = vvs = []$ or $uus = u{:}us$, $vvs = v{:}vs$. In the latter case let $(\theta us)\downarrow_\beta = (\theta vs)\downarrow_\beta$ and let $(\theta v)\downarrow_\beta = \theta u = u$ for a substitution θ. Let further u and v be such that the lhs/rhs of u has the same number of leading lambdas and the same top-level functor as the lhs/rhs of v. Then*

$$unif_4\ \langle uus, \theta\rangle = unif_5\ \langle vvs, \theta\rangle$$

Proof. We prove by induction over $unif_4$. If $uus = vvs = []$ then in both cases the result is $\llcorner\theta\lrcorner$. So suppose $uus = u{:}us$ and $vvs = v{:}vs$. Then

$$\begin{array}{rcl} unif_4\ \langle uus, \theta\rangle & = & \textbf{let}\ res_4 = (cases_4 \circ tw \circ otf\eta)\ u\ \textbf{in}\ \ldots \\ unif_5\ \langle uus, \theta\rangle & = & \textbf{let}\ res_5 = ((cases_5\ \theta) \circ tw \circ otf\eta)\ v\ \textbf{in}\ \ldots \end{array}$$

With Lemma 14 on the next page we can conclude that $(otf\eta\ u)$ and $(otf\eta\ v)$ have no leading lambdas, have the same top-level functor on their lhs/rhs and that $(\theta(otf\eta\ v))\downarrow_\beta = (otf\eta\ u)$. Since the function $otf\eta$ does not change the free variables in u it follows from $\theta u = u$ that $\theta(otf\eta\ u) = (otf\eta\ u)$.

Obviously Proposition 15 on the facing page can be used on res_4 and res_5. Thus either $res_4 = res_5 = \mathsf{None}$ or $res_4 = \langle us', \psi\rangle$, $res_5 = \langle vs', \psi\rangle$ and $us' = (\theta vs')\downarrow_\beta$, us' and vs' are in β-normal form and $\theta us' = us'$. So in case of success:

$$\begin{array}{rcl} unif_4\ \langle uus, \theta\rangle & = & unif_4\ \langle \overbrace{head_ap\ (\lambda v.((\psi \circ \theta)v)\downarrow_\beta)\ us' +\!\!+ us}^{uus'}, \psi \circ \theta\rangle \\ unif_5\ \langle uus, \theta\rangle & = & unif_5\ \langle \overbrace{head_ap\ (devar\ (\psi \circ \theta))\ vs' +\!\!+ vs}^{vvs'}, \psi \circ \theta\rangle \end{array}$$

We make the following observations:

(a) $us' +\!\!+ us$ is in β-normal form (since us in β-normal form by assumption)

(b) $vs' +\!\!+ vs$ is in β-normal form (since vs in β-normal form by assumption)

(c) $(\theta(us' +\!\!+ us))\downarrow_\beta = (\theta(vs' +\!\!+ vs))\downarrow_\beta$ (since $(\theta us)\downarrow_\beta = (\theta vs)\downarrow_\beta$ by assumption)

(d) $((\psi \circ \theta)(us' +\!\!+ us))\downarrow_\beta = ((\psi \circ \theta)(vs' +\!\!+ vs))\downarrow_\beta$ (using (c) and Fact 1 on page 6)

Suppose that $us' +\!\!+ us$ is nonempty. Let u'' be the first element of $us' +\!\!+ us$ and v'' be the first element of $vs' +\!\!+ vs$. So $u''' := ((\psi \circ \theta)u'')\downarrow_\beta$ and $v''' := devar\ (\psi \circ \theta)\ v''$ are the first elements of uus' and vvs' respectively. With Lemma 9 on page 39 follows that on the lhs/rhs of v''' are the the same number of leading lambdas and the same top-level functor as on the lhs/rhs of $((\psi \circ \theta)v'')\downarrow_\beta$. The fact also states that $((\psi \circ \theta)v''')\downarrow_\beta = ((\psi \circ \theta)v'')\downarrow_\beta$. Because of (d) it is $((\psi \circ \theta)v'')\downarrow_\beta = ((\psi \circ \theta)u'')\downarrow_\beta = u'''$. So

- on the lhs/rhs of v''' are the the same number of leading lambdas and the same top level functor as on the lhs/rhs of u'''
- $((\psi \circ \theta)v''')\downarrow_\beta = u'''$
- $u''' = (\psi \circ \theta)u'''$ because $\psi \circ \theta$ is constructed and thus idempotent

Because of the last three observations and observation (d) the induction hypothesis can be used.

□

The following lemma about $otf\eta$ was used in the proof of correctness:

Lemma 14. *Let* $u = (\lambda_k\ g(\!|sl|\!) \stackrel{?}{=} \lambda_l\ h(\!|sr|\!))$ *and* $v = (\lambda_k\ g(\!|tl|\!) \stackrel{?}{=} \lambda_l\ h(\!|tr|\!))$ *and* $(\theta v)\downarrow_\beta = u$ *for a constructed substitution* θ. *Then* $otf\eta\ u = (g(\!|sl'|\!) \stackrel{?}{=} h(\!|sr'|\!))$, $otf\eta\ v = (g(\!|tl'|\!) \stackrel{?}{=} h(\!|tr'|\!))$ *for some* sl', sr', tl' *and* tr' *and* $(\theta(otf\eta\ v))\downarrow_\beta = (otf\eta\ u)$

Proof. We prove by

full induction over $k + l$

$k + l = 0$ The unification problems u and v have no leading lambdas. Therefore the function $otf\eta$ changes nothing and the proposition obviously holds.

$k + l > 0$ case distinction

$k > 0$ and $l > 0$ it is

$$otf\eta\ u = otf\eta\ (\lambda_{k-1}\ g(\!|sl|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|sr|\!))$$
$$otf\eta\ v = otf\eta\ (\lambda_{k-1}\ g(\!|tl|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|tr|\!))$$

Since by assumption it is $(\theta v)\downarrow_\beta = u$ we can conclude that

$$(\theta(\lambda_{k-1}\ g(\!|tl|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|tr|\!)))\downarrow_\beta = (\lambda_{k-1}\ g(\!|sl|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|sr|\!))$$

Use the induction hypothesis.

$k = 0$ or $l = 0$ Suppose that $k = 0$. Then

$$otf\eta\ u = otf\eta\ (g(\!|(map\ (\lambda x.lift\ x\ 0)\ sl) +\!\!+ [\underline{0}]|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|sr|\!))$$
$$otf\eta\ v = otf\eta\ (g(\!|(map\ (\lambda x.lift\ x\ 0)\ tl) +\!\!+ [\underline{0}]|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|tr|\!))$$

Since by assumption it is $(\theta v)\downarrow_\beta = u$ we can conclude that

$$(\theta(\lambda_{l-1}\ h(\!|tr|\!)))\downarrow_\beta = \lambda_{l-1}\ h(\!|sr|\!) \tag{1}$$

From $(\theta v)\downarrow_\beta = u$ also follows that $(\theta g(\!|tl|\!))\downarrow_\beta = g(\!|sl|\!)$. Since θ is a constructed substitution one can deduce that $(\theta\ tl)\downarrow_\beta = sl$: If g is a bound variable this holds for any substitution. If g is a free variable then $\theta g = g$ since $\mathcal{RNG}(\theta) \cap \mathcal{DOM}(\theta) = \emptyset$ (Lemma 2 on page 5) and therefore $(\theta\ tl)\downarrow_\beta = sl$.
The function *lift* only manipulates loose bound variables and a substitution does not introduce new loose bound variables or capture loose bound variables. Therefore with $(\theta\ tl)\downarrow_\beta = sl$ it is clear that

$$(\theta\ ((map\ (\lambda x.lift\ x\ 0)\ sl) +\!\!+ [\underline{0}]))\downarrow_\beta = ((map\ (\lambda x.lift\ x\ 0)\ tl) +\!\!+ [\underline{0}]) \tag{2}$$

So with (1) and (2) we have shown that

$$(\theta(g(\!|(map\ (\lambda x.lift\ x\ 0)\ tl) +\!\!+ [\underline{0}]|\!)) \stackrel{?}{=} \lambda_{l-1}\ h(\!|tr|\!))\downarrow_\beta =$$
$$g(\!|(map\ (\lambda x.lift\ x\ 0)\ sl) +\!\!+ [\underline{0}]|\!) \stackrel{?}{=} \lambda_{l-1}\ h(\!|sr|\!)$$

Use the induction hypothesis.

□

Now it is easy to show the correctness of $unif_5$:

Proposition 17 (Correctness of $unif_5$). *For a unification problem u in β-normal form it is $unif_5\ \langle [u], \emptyset \rangle = \mathsf{None}$ iff u is not unifyable. Otherwise $unif_5\ \langle [u], \emptyset \rangle = \llcorner \psi \lrcorner$ and ψ is a most general unifier of u.*

Proof. The correctness of $unif_4$ has been established in Proposition 13 on page 37. With Proposition 16 on page 49 we see that $unif_4\ \langle [u], \emptyset \rangle = unif_5\ \langle [u], \emptyset \rangle$. So $unif_5$ also is correct.

□

6.5 Some thoughts about an implementation of $unif_5$

The fifth version of the algorithm is very close to a functional program in ML or Haskell — the generation of new variable names could be solved with references in ML, thus using an impure feature, or a state monad in Haskell.

The only thing that requires some thought is the handling of the isomorphism between $lterm_b$ and $lterm_{f'}$ — version five still uses both representations. If the system which is to use higher-order unification is based on a lambda term representation with binary application (i.e. $lterm_b$), transformations to $lterm_{f'}$ representation have to be carried out. The easiest (and probably least efficient) option is to use the isomorphisms between the two representations. Nipkow [7] uses a function called *strip* which gives access to the top-level functor-arguments form of a term in $lterm_b$ representation. Functor and arguments are left in $lterm_b$ representation, *strip* only transforms a term as far as necessary (this can be compared to the behavior of *devar* which only instantiates a term as far as necessary). The retransformation is carried out with a slightly different function *red*.

Of course one also could chose to make the functor-arguments form primitive — in this case the few auxilliary functions based on $lterm_b$ representation would be adapted for $lterm_{f'}$ representation, thus avoiding transformations altogether.

Appendix A

Lambda terms

A.1 A theory of lambda terms

The following theory about lambda terms is a modification of the theory presented in [8].

A.1.1 Datatypes, functions and relations

The theory of lambda terms is based on the representation $lterm_b$:

$$\begin{aligned}\textbf{datatype } lterm_b = \;& \mathsf{BVar}\ nat \\ \mid\; & \mathsf{Var}\ nat \\ \mid\; & \lambda\ lterm_b \\ \mid\; & lterm_b\ @\ lterm_b\end{aligned}$$

β-reduction

For β-reduction a notion of substitution of *bound* variables is needed. In $t[s\backslash k]$ any bound variable[1] $\underline{k+n}$ of t which lies "under" n lambdas in the structure of t is substituted by s', where s' is derived from s by adding n to each of its loose bound variables (this prevents that loose bound variables are captured). The necessary adjustments in s are carried out by the *lift*-function.
Note that also loose bound variables in t are adjusted by $[_\backslash_]$. This is because β-reduction removes a lambda. So the value of loose bound variables of t which might be bound by some lambda in the context of t has to be decremented by one.

$$\begin{aligned}&t[s\backslash k] = \textbf{case } t \textbf{ of} \\ &\quad \mid \underline{i} \to \textbf{if } k < i \textbf{ then } \underline{i-1} \\ &\qquad\qquad \textbf{else if } k = i \textbf{ then } s \textbf{ else } \underline{i} \\ &\quad \mid (r\ @\ r') \to (r[s\backslash k])\ @\ (r'[s\backslash k]) \\ &\quad \mid (\lambda\ r) \to \lambda\ (t[lift\ s\ 0\backslash k+1]) \\ &\quad \mid \textbf{otherwise} \to t \\ &lift\ t\ k = \textbf{case } t \textbf{ of} \\ &\quad \mid \underline{i} \to \textbf{if } i < k \textbf{ then } \underline{i} \\ &\qquad\qquad \textbf{else } \underline{i+1} \\ &\quad \mid r\ @\ s \to (lift\ r\ k)\ @\ (lift\ s\ k) \\ &\quad \mid \lambda\ r \to \lambda\ (lift\ s\ (k+1)) \\ &\quad \mid \textbf{otherwise} \to t\end{aligned}$$

[1] as explained in section 1.1.2 we often write '$\underline{i}$' for '$\mathsf{BVar}\ i$'. For a natural number n the expression '$\underline{i}+n$' denotes the bound variable $\underline{i+n}$

β-reduction is defined by the following rules:

$$\frac{}{(\lambda\ s)\ @\ t \longrightarrow_\beta s[t\backslash 0]} \quad \frac{s \longrightarrow_\beta t}{r\ @\ s \longrightarrow_\beta r\ @\ t} \quad \frac{s \longrightarrow_\beta t}{s\ @\ r \longrightarrow_\beta t\ @\ r} \quad \frac{s \longrightarrow_\beta t}{\lambda\ s \longrightarrow_\beta \lambda\ t}$$

η-conversion

η-conversion in $\lambda x.(t\ @\ x)$ (written in a representation with named bound variables) only is possible, if x does not occur freely in t. For de Bruijn-notation we test this condition with the function *free*. Since η-conversion in $\lambda(t\ @\ \underline{0})$ removes the leading lambda, bound variables in t have to be adjusted. This is carried out by *decr*, which is definied in terms of the substitution of bound variables.

$$\begin{array}{l} \mathit{free}\ t\ i = \textbf{case}\ t\ \textbf{of} \\ \qquad \mid \underline{j} \to (i = j) \\ \qquad \mid r\ @\ s \to (\mathit{free}\ r\ i)\textbf{orelse}(\mathit{free}\ s\ i) \\ \qquad \mid \lambda\ s \to \lambda\ (\mathit{free}\ s\ (i+1)) \\ \qquad \mid \textbf{otherwise} \to \textbf{false} \\ \mathit{decr}\ t\ i = t[\underline{i}\backslash i] \end{array}$$

η-conversion is defined by the following rules:

$$\frac{\neg(\mathit{free}\ s\ 0)}{(\lambda\ (s\ @\ \underline{0})) \longrightarrow_\eta \mathit{decr}\ s\ 0} \quad \frac{s \longrightarrow_\eta t}{r\ @\ s \longrightarrow_\eta r\ @\ t} \quad \frac{s \longrightarrow_\eta t}{s\ @\ r \longrightarrow_\eta t\ @\ r} \quad \frac{s \longrightarrow_\eta t}{\lambda\ s \longrightarrow_\eta \lambda\ t}$$

A.1.2 Lemmas about de Bruijn-notation

Since the de Bruijn-notation is less intuitive than the notation of lambda terms with named bound variables, some facts, which otherwise are easy to see, may require some more thought. In the following lemmas some of these facts are shown. They will be needed for a number of proofs.

Lemma 15. $(\lambda_m\ t)(\overline{x_m}) \xrightarrow{m}_\beta t[x_{m-1} + (m-1)\backslash m-1] \ldots [x_0 + 0\backslash 0]$

Proof. induction over m

$m = 0$ $\quad t \xrightarrow{0}_\beta t$

$m \longrightarrow (m+1)$

$(\lambda_{m+1}\ t)(\overline{x_{m+1}}) \longrightarrow_\beta ((\lambda_m\ t)[x_m\backslash 0])(\overline{x_m})$

$=$ ⟨with definition of $[_\backslash_]$ on the facing page, induction over the number of leading lambdas⟩

$(\lambda_m\ (t[x_m + m\backslash m]))(\overline{x_m})$

$\xrightarrow{m}_\beta$ ⟨induction hypothesis⟩

$t[x_m + m\backslash m][x_{m-1} + (m-1)\backslash m-1] \ldots [x_0 + 0\backslash 0]$

□

Lemma 16. $\underline{i}[\underline{m-1} + (m-1) + j\backslash m-1+j] \ldots [\underline{0} + j\backslash 0 + j] = \underline{i}$ *for* $0 \leqslant i \leqslant (m-1) + j$

Proof. Case distinction over i

$i < j$ then by definition of $[_\backslash_]$ on the preceding page $\underline{i}$ remains unchanged.

$j \leqslant i \leqslant (m-1) + j$

induction over m

$m = 1 \quad \Rightarrow i = j$

$\underline{j}[\underline{0} + j\backslash j] = \underline{0} + j = \underline{j}$

$m \longrightarrow (m+1)$

$\underline{i}[\underline{m} + m + j\backslash m + j] \ldots [\underline{0} + j\backslash j] =$

case distinction over i

$\underline{i = (m + j)}$ then by definition of $[_\backslash_]$
$(\underline{m} + m + j)[\underline{m-1} + (m-1) + j \backslash m - 1 + j] \ldots [\underline{0} + j \backslash 0 + j]$
$= \langle$definition of $[_\backslash_]\rangle$
$\underline{m} + j = \underline{m + j}$
$\underline{i \leqslant (m + j)}$ then by definition of $[_\backslash_]$:
$\underline{i}[\underline{m-1} + (m-1) + j \backslash m - 1 + j] \ldots [\underline{0} + j \backslash 0 + j]$
$= \langle$induction hypothesis$\rangle$
$\underline{i}$

□

Lemma 17. $\underline{i}[x_{m-1} + (m-1) \backslash m-1] \ldots [x_0 + 0 \backslash 0] = x_i$ *for* $0 \leqslant i \leqslant (m-1)$

Proof. induction over m

$\underline{m = 1} \Rightarrow i = 0$
$\underline{0}[x_0 \backslash 0] = x_0$

$\underline{m \longrightarrow (m+1)}$
$\underline{i}[x_m + m \backslash m] \ldots [x_0 \backslash 0] =$
case distinction over i

$\underline{i = m}$ then by definition of $[_\backslash_]$
$(x_m + m)[x_{m-1} + (m-1) \backslash m - 1] \ldots [x_0 \backslash 0]$
$= \langle$definition of $[_\backslash_]\rangle$
x_m
$\underline{i \leqslant (m-1)}$ then by definition of $[_\backslash_]$:
$\underline{i}[x_{m-1} + (m-1) \backslash m - 1] \ldots [x_0 + 0 \backslash 0]$
$= \langle$induction hypothesis$\rangle$
x_i

□

Lemma 18.
If there are no lambdas in t *and for every bound variables* $\underline{i}$ *of* t *it is* $0 \leqslant i \leqslant (m-1)$ *then* $(\lambda_m\ t)(\overline{x_m}) \xrightarrow{m}_\beta t'$ *where* t' *is* t *with any bound variable* $\underline{i}$ *of* t *replaced by* x_i.

Proof. $(\lambda_m\ t)(\overline{x_m})$
$\xrightarrow{m}_\beta$ ⟨Lemma 15 on the preceding page⟩
$t[x_{m-1} + (m-1) \backslash m - 1] \ldots [x_0 + 0 \backslash 0]$

The chain of substitutions traverses the structure and is applied to any bound variable $\underline{i}$. Since there are no lambdas in t, the chain of substitutions remains unchanged (definition of $[_\backslash_]$ on page 52). Every bound variable $\underline{i}$ is instantiated with x_i according to Lemma 17. □

A very similar lemma is

Lemma 19.
If s *has no loose bound variables and* $\overline{z_m}$ *is a list of bound variables* $\underline{i}$ *with* $0 \leqslant i \leqslant (m-1)$ *then* $(\lambda_m\ s(\!|\overline{z_n}|\!))(\overline{x_m}) \xrightarrow{m}_\beta s(\!|[x_{z_{n-1}}, \ldots, x_{z_0}]|\!)$

Proof. $(\lambda_m\ \ s(\!|\overline{z_n}|\!))(\!|\overline{x_m}|\!)$
$\xrightarrow{m}_\beta$ ⟨Lemma 15 on the page before⟩
$(s(\!|\overline{z_n}|\!))[x_{m-1} + (m-1) \backslash m - 1] \ldots [x_0 + 0 \backslash 0]$

The chain of substitutions traverses the structure and is applied to s and every argument z_i. Since s has no loose bound variables, it remains unchanged. Any z_i is instantiated according to Lemma 17, resulting in x_{z_i}. □

Lemma 20. *If* t *has no loose bound variables, then* $\lambda_m\ (t(\!|[\underline{m-1}, \ldots, \underline{0}]|\!)) \xrightarrow{m}_\eta t$

Proof. induction over m

$m = 0$ $\quad t \xrightarrow{0}_\eta t$

$m \to (m+1)$

$\lambda_{m+1}\ (t(\!|[\underline{m}, \ldots, \underline{0}]|\!))$

$\longrightarrow_\eta$ ⟨since t has no loose bound variables: $\neg(\mathit{free}\ t(\!|[\underline{m}, \ldots, \underline{1}]|\!)\ 0)$. With definition of $\longrightarrow_\eta$, on page 53⟩

$\lambda_m\ (\mathit{decr}\ t(\!|[\underline{m}, \ldots, \underline{1}]|\!)\ 0)$

$=$ ⟨definition of *decr*, on page 53⟩

$\lambda_m\ ((t(\!|[\underline{m}, \ldots, \underline{1}]|\!))[\underline{0}\backslash 0])$

$=$ ⟨definition of $[_\backslash_]$, on page 52⟩

$\lambda_m\ ((t[\underline{0}\backslash 0])(\!|[\underline{m-1}, \ldots, \underline{0}]|\!))$

$=$ ⟨no loose bound variables in t⟩

$\lambda_m\ (t(\!|[\underline{m-1}, \ldots, \underline{0}]|\!))$

$\xrightarrow{m}_\eta$ ⟨induction hypothesis⟩

t

□

Lemma 21. *If $\lambda_m\ t$ has no loose bound variables then*

$$(\lambda_m\ t)(\!|[\underline{m-1}, \ldots, \underline{0}]|\!) \xrightarrow{m}_\beta t$$

Proof. With the definition of *free* on page 53 it is easy to see that $\neg(\mathit{free}\ t\ n)$ for $n \geqslant m$.We make the following assumption: If for all $n \geqslant (m+j)$ it is $\neg(\mathit{free}\ t\ n)$ then

$$t[\underline{m-1} + (m-1) + j \backslash (m-1) + j] \ldots [\underline{0} + j \backslash j] = t$$

Using lemma 15 on page 53 it is

$(\lambda_m\ t)(\!|[\underline{m-1}, \ldots, 0]|\!) \xrightarrow{m}_\beta t[\underline{m-1} + m - 1 \backslash m - 1] \ldots [\underline{0}\backslash 0]$

$=$ ⟨with the assumption, setting j to 0⟩

t

We now show the assumption:

induction over t

$\underline{t} = \underline{i}$ Since $\neg(\mathit{free}\ \underline{i}\ n)$ for all $n \geqslant (m+j)$ it follows that $0 \leqslant i \leqslant (m+j-1)$ with the definition of *free* on page 53. Using lemma 16 on page 53 we find that $\underline{i}[\underline{m-1} + (m-1) + j \backslash (m-1) + j] \ldots [\underline{0} + j \backslash j] = \underline{i}$

$\underline{t} = \lambda\ t'$ With the definition of *free* it follows that $\neg(\mathit{free}\ t'\ n)$ for all $n \geqslant (m+j+1)$

$(\lambda\ t')[\underline{m-1} + (m-1) + j \backslash (m-1) + j] \ldots [\underline{0} + j \backslash j]$

$=$ ⟨definition of $[_\backslash_]$ on page 52⟩

$\lambda\ (t'[\underline{m-1} + m + j \backslash m + j] \ldots [\underline{0} + (j+1) \backslash (j+1)])$

$=$ ⟨induction hypothesis⟩

$\lambda\ t'$

$\underline{t} = \underline{r}\ @\ \underline{s}$ With the definition of *free* it follows that $\neg(\mathit{free}\ r\ n)$ and $\neg(\mathit{free}\ s\ n)$ for all $n \geqslant (m+j)$. The chain of substitutions distributes over the application. Use the induction hypothesis.

□

The following two lemmas provide a basis for proofs of Facts 10 — 13 on page 57. Their proofs have rather the character of proof-sketches, since they appeal to an intutitive understanding of reductions on lambda terms in a notation with named bound variables.

Lemma 22. *Let t have no loose bound variables and let*

$$t(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \equiv t(\!|ys +\!\!+ y{:}\overline{y_m}|\!)$$

for two lists of bound variables $xs +\!\!+ x{:}\overline{x_m}$ and $ys +\!\!+ y{:}\overline{y_m}$. Then

- *if $x = y$ it is $t(\!|xs +\!\!+ z{:}\overline{x_m}|\!) \equiv t(\!|ys +\!\!+ z{:}\overline{y_m}|\!)$ for any bound variable z*
- *if $x \neq y$ it is $t \equiv \lambda_{\lfloor xs \rfloor+1}\ t'$ and $\neg(\mathit{free}\ t'\ 0)$*

Proof. For lambda terms in a representation with named bound variables it is obvious that loose bound variables are treated like constants in β- or η-reductions. This holds in a certain form also for de Bruijn-notation: In de Bruijn-notation for a loose bound variable $\underline{i}$ with k lambdas "above" it in the term structure, the natural number $(i-k)$ can be seen as its identity, since it is preserved by β- and η-reductions.

Because of the Church-Rosser property of $(\beta\cup\eta)$ there is a term s such that $t(\!|xs +\!\!+ x{:}\overline{x_m}|\!)\ (\beta\cup\eta)^*\ s$ and $t(\!|ys +\!\!+ y{:}\overline{y_m}|\!)\ (\beta\cup\eta)^*\ s$. If t has no leading lambdas, the loose bound variables in $xs +\!\!+ x{:}\overline{x_m}$ and $ys +\!\!+ y{:}\overline{y_m}$ obviously will stay unaffected by β- and η-reductions. Otherwise they only can be subject to a β-reduction. Since the β-reduction with a loose bound variable does not introduce new β- or η-redexes, the reductions involving $xs +\!\!+ x{:}\overline{x_m}$ and $ys +\!\!+ y{:}\overline{y_m}$ can be carried out as the last reductions. Thus there is a t'' with $t\ (\beta\cup\eta)^*\ t''$ such that $t''(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \xrightarrow{*}_\beta s_1$ and $t''(\!|ys +\!\!+ y{:}\overline{y_m}|\!) \xrightarrow{*}_\beta s_2$ where every β-reduction involves one of the loose bound variables in the argument lists and of course $s_1 = s_2 = s$.

Suppose that $x = y$. Obviously for every x in s_1 which stems from the x in $xs +\!\!+ x{:}\overline{x_m}$, s_2 has a y which stems from the y in $ys +\!\!+ y{:}\overline{y_m}$. So the x in $xs +\!\!+ x{:}\overline{x_m}$ and the y in $ys +\!\!+ y{:}\overline{y_m}$ can be replaced by any z without affecting equality of the completely β-reduced terms. Thus indeed

$$t''(\!|xs +\!\!+ z{:}\overline{x_m}|\!) \xrightarrow{*}_\beta s \xleftarrow{*}_\beta t''(\!|ys +\!\!+ z{:}\overline{y_m}|\!)$$

Suppose that $x \neq y$. Since β-reductions preserve the identity of loose bound variables it is obvious that x and y must "vanish" in order to reach the same term s from $t''(\!|xs +\!\!+ x{:}\overline{x_m}|\!)$ and from $t''(\!|ys +\!\!+ y{:}\overline{y_m}|\!)$. This is only possible if x and y are subject of a β-reduction where the bound variable associated with the lambda of the β-redex in question does not occur in the term "under" that lambda. Thus $t'' = \lambda_{\lfloor xs \rfloor+1}\ t'$ and $\neg(\mathit{free}\ t'\ 0)$. □

Lemma 23. *Let s and t have no loose bound variables and let*

$$t(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \equiv s(\!|\overline{y_n}|\!)$$

for two lists of bound variables $xs +\!\!+ x{:}\overline{x_m}$ and $\overline{y_n}$. Then

- *if $\exists i.y_i = x$ and $x \notin \{xs\}$ it is $t(\!|xs +\!\!+ z{:}\overline{x'_m}|\!) \equiv s(\!|\overline{y'_n}|\!)$ for any bound variable z where*

$$x'_i = \begin{cases} x_i & \mathit{if}\ x_i \neq z \\ z & \mathit{otherwise} \end{cases}$$

  and

$$y'_i = \begin{cases} y_i & \mathit{if}\ y_i \neq z \\ z & \mathit{otherwise} \end{cases}$$

  for any bound variable z

- *if $\neg(\exists i.y_i = x)$ it is $t\ (\beta\cup\eta)^*\ \lambda_{\lfloor xs \rfloor+1}\ t'$ and $\neg(\mathit{free}\ t'\ 0)$*

Proof. Reasoning like in the proof of Lemma 22 with the Church-Rosser property of $(\beta\cup\eta)$ we see that there are t'' and s'' with $t\ (\beta\cup\eta)^*\ t''$ and $s\ (\beta\cup\eta)^*\ s''$ and there are terms $r_1 = r_2 = r$ such that $t''(\!|xs +\!\!+ x{:}\overline{x_m}|\!) \xrightarrow{*}_\beta r_1$ and $s''(\!|\overline{y_n}|\!) \xrightarrow{*}_\beta r_2$.

Suppose that $\exists i.y_i = x$ and $x \notin \{xs\}$. If we replace all occurences of x in r_1 and r_2, equality is not affected. But every occurence of x in r_1 and r_2 must stem from an occurence of x in $x{:}\overline{x_m}$ and $\overline{y_n}$.

This is due to the fact that $x \notin \{xs\}$ and that t'' and s'' have no loose bound variables (no loose bound variables are introduced by β- or η-reductions and t and s have no loose bound variables by assumption). Therefore obviously we can replace all occurrences of x in $x{:}\overline{x_m}$ and in $\overline{y_n}$ with z and still have

$$t''(\!|xs +\!\!+\ z{:}\overline{x_m}|\!) \xrightarrow{*}_\beta s \xleftarrow{*}_\beta t''(\!|ys +\!\!+\ z{:}\overline{y_m}|\!)$$

Suppose that $\neg(\exists i.y_i = x)$. Since β-reductions preserve the identity of loose bound variables it is obvious that x must "vanish" in order to reach the same term r from $t''(\!|xs +\!\!+\ x{:}\overline{x_m}|\!)$ and from $s''(\!|\overline{y_n}|\!)$. This is only possible if x is subject to a β-reduction where the bound variable associated with the lambda of the β-redex in question does not occur in the term "under" that lambda. Thus $t'' = \lambda_{|xs|+1}\ t'$ and $\neg(\mathit{free}\ t'\ 0)$. □

A.1.3 Facts from the lambda calculus

The following facts will be used without further proof.

Fact 5 and Fact 6 are easy to see for a representation of λ-terms with named bound variables.

Fact 5. $s \equiv t \ \Leftrightarrow\ \lambda\ s \equiv \lambda\ t$

Fact 6. $\lambda\ s \equiv t \ \Leftrightarrow\ s \equiv (\mathit{lift}\ t\ 0)\ @\ \underline{0}$

Since substitutions never introduce loose bound variables and *lift* only manipulates loose bound variables, the following fact is quite intuitive:

Fact 7. $\sigma(\mathit{lift}\ t\ 0) = (\mathit{lift}\ \sigma t\ 0)$

Substitutions do not interfere with β- and η-reduction:

Fact 8. *For terms s, t and a substitution σ:*

1. *If $s \xrightarrow{*}_\beta t$ then $\sigma s \xrightarrow{*}_\beta \sigma t$*
2. *If $s \xrightarrow{*}_\eta t$ then $\sigma s \xrightarrow{*}_\eta \sigma t$*
3. *If $s \equiv t$ then $\sigma s \equiv \sigma t$*

Fact 9. *Two terms in β-normal form $F(\!|\overline{x_m}|\!)$ and $a(\!|\overline{s_m}|\!)$ are not unifyable if $F \in \mathcal{FV}(\overline{s_m})$*

Lemma 22 on the preceding page and 23 on the facing page are used to show the following facts:

Fact 10. *If t has no loose bound variables and $t(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_m}|\!)$ for argument lists $\overline{x_m}$ and $\overline{y_m}$ where for $0 \leqslant i \leqslant (m-1)$ it is $x_i \neq y_i$ then there is a t' without loose bound variables such that $t \equiv \lambda_m\ t'$.*

Fact 11. *If s and t have no loose bound variables and $s(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_n}|\!)$ for argument lists $\overline{x_m}$ and $\overline{y_m}$ where $\{\overline{x_m}\} \cap \{\overline{y_n}\} = \emptyset$ then there are t' and s' without loose bound variables such that $t \equiv \lambda_m\ t'$, $s \equiv \lambda_n\ s'$ and $t' \equiv s'$.*

Fact 12. *Let t have no loose bound variables and let $t(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_m}|\!)$. If for $\overline{z_m}$ and $\overline{z'_m}$ it is $z_i = z'_i$ whenever $x_i = y_i$ then $t(\!|\overline{z_m}|\!) \equiv t(\!|\overline{z'_m}|\!)$.*

Fact 13. *Let s and t have no loose bound variables and let $s(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_n}|\!)$. If for $\overline{z'_m}$ and $\overline{z_n}$ it is $z'_i = z_j$ whenever $x_i = y_j$ then $s(\!|\overline{z'_m}|\!) \equiv t(\!|\overline{z_n}|\!)$.*

Fact 10 and Fact 12 are shown with induction over the length of the argument list in a relatively straightforward way (however the hypothesis has to be strengthend) using Lemma 22. Fact 11 and Fact 13 are also shown by induction over the length of the argument lists, this time with Lemma 23. For Fact 13 an additional lemma has to be shown which basically says that the arguments of one side of the equivalence can be renamed, if those arguments that also occur on the the other side are renamed, too.

Using Fact 13 twice and applying the transitivity of $\equiv$ one proves

Fact 14. *Let t and s have no loose bound variables and let $s(\!|\overline{x_m}|\!) \equiv t(\!|\overline{y_n}|\!)$. If for $\overline{z_m}$ and $\overline{z'_m}$ it is $z_i = z'_i$ whenever $\exists j.x_i = y_j$ then $s(\!|\overline{z_m}|\!) \equiv s(\!|\overline{z'_m}|\!)$*

A.2 Isomorphisms between $lterm_b$ and $lterm_f$

The isomorphism between $lterm_f$ as described on page 3 and $lterm_b$ will be shown by using a third data type $lterm_{f'}$. The only difference to $lterm_f$ is that there are no empty argument lists in $lterm_{f'}$: free or bound variables without arguments to them are not treated as functors. As a datatype this results to

$$\begin{aligned}
&\textbf{datatype } l_basic = \ \mathsf{BVar}\ nat \\
&\qquad\qquad\qquad\quad \mid \mathsf{Var}\ nat \\
&\qquad\qquad\qquad\quad \mid \lambda\ lterm_{f'} \\
&\textbf{datatype } lterm_{f'} = \ l_basic \\
&\qquad\qquad\qquad\quad \mid l_basic(\!|(lterm_{f'})list|\!)
\end{aligned}$$

The argument lists are assumed to be nonempty. $lterm_{f'}$ looks somewhat complicated because of the use of *l_basic*. This is done in order to keep functor-argument forms from occuring in functor position.

As shown below, there are isomorphisms $to_b :: lterm_{f'} \to lterm_b$ and $to_f' :: lterm_b \to lterm_{f'}$:

$$to_b' \circ to_f' = id_{lterm_b} \qquad\qquad to_f' \circ to_b' = id_{lterm_{f'}}$$

Furthermore, there is an obvious isomorphism between $lterm_f$ and $lterm_{f'}$: in order to transform $lterm_f$ to $lterm_{f'}$, empty argument lists are removed. For the reverse direction bound and free variables which are not in functor position are put in functor position with an empty list of arguments. Thus there are isomorphisms $to_b :: lterm_f \to lterm_b$ and $to_f :: lterm_b \to lterm_f$.

The isomorphisms to_f' and to_b' are

$$\begin{aligned}
&to_f'\ t = \textbf{case } t \textbf{ of} \\
&\qquad \mid \lambda\ t' \to \lambda\ (to_f'\ t') \\
&\qquad \mid r\ @\ s \to col\ r\ [to_f'\ s] \\
&\qquad \mid \textbf{otherwise} \to t \\
&\qquad\ \ \textbf{where} \\
&\qquad\qquad col\ t\ rrs = \textbf{case } t \textbf{ of} \\
&\qquad\qquad\qquad\qquad \mid r\ @\ s \to col\ r\ ((to_f'\ s) \mathbf{:} rrs) \\
&\qquad\qquad\qquad\qquad \mid \textbf{otherwise} \to (to_f'\ t)(\!|rrs|\!) \\
&to_b'\ t = \textbf{case } t \textbf{ of} \\
&\qquad \mid \lambda\ t' \to \lambda\ (to_b'\ t') \\
&\qquad \mid s(\!|rrs|\!) \to foldl\ \ @\ (to_b'\ s)\ (map\ to_b'\ rrs) \\
&\qquad \mid \textbf{otherwise} \to t
\end{aligned}$$

The isomorphism is proved in the following two propositions.

Proposition 18. $to_f' \circ to_b' = id_{lterm_{f'}}$

Proof. structural induction over t

$t = \lambda\ t'$

$$\begin{aligned}
&(to_f' \circ to_b')\ \lambda\ t' = to_f'\ (\lambda\ (to_b'\ t')) = \lambda\ (to_f' \circ to_b'\ t') = \\
&= \langle \text{induction hypothesis} \rangle \\
&\lambda\ t'
\end{aligned}$$

$t = s(\!|rrs|\!)$

$$\begin{aligned}
&(to_f' \circ to_b')\ s(\!|rss|\!) = to_f'\ (foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rrs)) = \\
&= \langle \text{since } rrs \text{ by definition nonempty, } rss \text{ can be seen as } rs +\!\!+ [r] \rangle \\
&to_f'\ (foldl\ @\ (to_b'\ s)\ ((map\ to_b'\ rs) +\!\!+ [to_b'\ r])) = \\
&= \langle \text{with an easy lemma about } foldl \rangle \\
&to_f'\ ((foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rs))\ @\ (to_b'\ r))
\end{aligned}$$

$= \langle$definition of $to_f'\rangle$

$col\ (foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rs))\ [to_f' \circ to_b'\ r] =$

$= \langle$induction hypothesis and Lemma 24$\rangle$

$s(\!|rs +\!\!+ [r]|\!) = s(\!|rrs|\!)$

$t = p$ or $t = F$ obviously $(to_f' \circ to_b')\ p = p$ and $(to_f' \circ to_b')\ F = F$

□

Proposition 19. $to_b' \circ to_f' = id_{lterm_b}$

Proof. structural induction over t

$t = \lambda\ t'$ $(to_b' \circ to_f')\lambda\ t' = to_b'(\lambda\ to_f' t') = \lambda\ (to_b' \circ to_f')t'$

$= \langle$induction hypothesis$\rangle$

$\lambda\ t'$

$t = r\ @\ s$ obviously r can be written as $(foldl\ @\ r_m\ \overline{r_m})$ for a list of terms $\overline{r_{m+1}}$ such that r_m is not of the form $r'\ @\ r''$. Then

$(to_b' \circ to_f')(r\ @\ s) = to_b'(to_f'\ (r\ @\ s))$

$= \langle$definition of to_f' on the preceding page$\rangle$

$to_b'(col\ r\ [to_f'\ s]) = to_b'(col\ (foldl\ @\ r_m\ \overline{r_m})\ [to_f'\ s])$

$= \langle$because of the induction hypothesis it is $(to_b' \circ to_f')\overline{r_{m+1}} = \overline{r_{m+1}}$ and $(to_b' \circ to_f')s = s$. Use Lemma 25$\rangle$

$foldl\ @\ r_m\ (\overline{r_m} +\!\!+ [s])$

$= \langle$with an easy lemma about $foldl\rangle$

$(foldl\ @\ r_m\ \overline{r_m})\ @\ s = r\ @\ s$

□

Lemma 24. *If* $to_f' \circ to_b'\ s = s$ *and* $map\ (to_f' \circ to_b')\ rrs = rrs$ *for a term of form* $s(\!|rrs|\!)$ *in* $lterm_{f'}$ *representation, then*

$$col\ (foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rrs))\ ts \quad = \quad s(\!|rrs +\!\!+ ts|\!)$$

Proof. induction over rs

$rrs = []$ $col\ (to_b'\ s)\ ts =$

$= \langle s$ cannot be of the form $s_1(\!|_|\!)$ since it is in functor position. Thus $to_b's$ cannot be of the form $s_1'\ @\ s_2'$, since it is in functor-position; with the definition of *col* on the preceding page$\rangle$

$(to_f' \circ to_b'\ s)(\!|ts|\!) = s(\!|ts|\!)$

$rrs = rs +\!\!+ [r]$ $col\ (foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rs +\!\!+ [r]))\ ts =$

$= \langle$with an easy lemma about $foldl\rangle$

$col\ ((foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rs))\ @\ (to_b'\ r))\ ts =$

$= \langle$definition of *col*, on the facing page$\rangle$

$col\ (foldl\ @\ (to_b'\ s)\ (map\ to_b'\ rs))\ (to_f' \circ to_b'\ r){:}ts =$

$= \langle$induction hypothesis$\rangle$

$s(\!|rs +\!\!+ (r{:}ts)|\!) = s(\!|rs +\!\!+ [r] +\!\!+ ts|\!) = s(\!|rrs +\!\!+ ts|\!)$

□

Lemma 25. *If for the lists of terms in* $lterm_b$*-representation* $r{:}rs$ *and* ts *it is* $(to_b' \circ to_f')(r{:}rs) = (r{:}rs)$, $(to_b' \circ to_f')ts = ts$ *and* r *is not of the form* $s\ @\ t$ *then*

$$to_b'(col\ (foldl\ @\ r\ rs)\ (to_f'\ ts)) = foldl\ @\ r\ (rs +\!\!+ ts)$$

Proof. induction over rs

$rs = [\!]$ It is
$to_b'(col\ r\ (to_f'\ ts))$
$= \langle$r is not of the form $s\ @\ t$, definition of *col* on page 58$\rangle$
$to_b'((to_f'\ r)([\![to_f'\ ts]\!]))$
$= \langle$definition of to_b' on page 58$\rangle$
$foldl\ @\ ((to_b' \circ to_f')r)\ ((to_b' \circ to_f')ts)$
$= \langle$with the assumption$\rangle$
$foldl\ @\ r\ ts$
$rs = (rs' +\!\!+ [r'])$ $to_b'(col\ (foldl\ @\ r\ (rs' +\!\!+ [r']))\ (to_f'\ ts))$
$= \langle$with an easy lemma about *foldl*$\rangle$
$to_b'(col\ ((foldl\ @\ r\ rs')\ @\ r')\ (to_f'\ ts))$
$= \langle$definition of *col* on page 58$\rangle$
$to_b'(col\ (foldl\ @\ r\ rs')\ ((to_f'\ r'){:}(to_f'\ ts)))$
$= \langle$with induction hypothesis$\rangle$
$foldl\ @\ r\ (rs' +\!\!+ (r'{:}ts)) = foldl\ @\ r\ (rs +\!\!+ ts)$

□

Bibliography

[1] Henk Barendregt. *The Lambda Calculus: its Syntax and Semantics*. North-Holland, 1981 (1st ed) revised 84.

[2] Nachum Dershowitz and Jean-Pierre Jouannaud. Rewrite systems. In J. van Leeuwen, editor, *Handbook of Theoretical Computer Science*, volume B: Formal Methods and Semantics, chapter 6, pages 243–320. North-Holland, Amsterdam, 1990.

[3] Gérard Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science*, 1:27–57, 1975.

[4] David King and John Launchbury. Lazy depth-first search and linear graph algorithms in haskell. Technical report, University of Glasgow, December 1993.

[5] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In P. Schroeder-Heisler, editor, *Extensions of Logic Programming*, number 475 in LNCS, pages 253–281, 1991.

[6] Tobias Nipkow. Higher-order critical pairs. In G. Kahn, editor, *Sixth Annual IEEE Symposium on Logic in Computer Science*, pages 342–349, Amsterdam, The Netherlands, July 1991.

[7] Tobias Nipkow. Functional unification of higher-order patterns. In *Proceedings, Eighth Annual IEEE Symposium on Logic in Computer Science*, pages 64–74. IEEE Computer Society Press, 1993.

[8] Tobias Nipkow. More Church-Rosser proofs (in Isabelle/HOL). In M. A. McRobbie and J. K. Slaney, editors, *Proceedings of the Thirteenth International Conference on Automated Deduction (CADE-96)*, volume 1104 of *LNAI*, pages 733–747, Berlin, July 30 –August 3 1996. Springer.

[9] Tobias Nipkow. Private communication. 1997.

[10] Lawrence C. Paulson. Designing a theorem prover. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 415–475. Oxford University Press, 1992.

[11] Lawrence C. Paulson. *Isabelle: a generic theorem prover*, volume 828 of *Lecture Notes in Computer Science*. Springer-Verlag Inc., New York, NY, USA, 1994.

[12] J. Peterson[editor], K. Hammond[editor], L. Augustsson, B. Boutel, W. Burton, J. Fasel, A. Gordon, J. Hughes, P. Hudak, T. Johnsson, M. Jones, S. Peyton Jones, A. Reid, and P. Wadler. Haskell 1.3, A non-strict, purely functional language. Report YALEU / DCS / RR-1106, Department of Computer Science, Yale University, May 1996.

[13] Philip Wadler. Comprehending monads. Technical report, University of Glasgow, April 1992.

Index

$(_)\downarrow_\beta$, 6
:, 6
$+\!\!+$, 6
$[_\backslash_]$, 52
$\emptyset$, 4
$\equiv$, 5

β-approximation, 28
β-reduction, 52
binding, 4
    representation, 4
bound variable
    loose, 3
    representation, 3

cases
    version 1, 9
    version 3, 23
    version 4, 25
    version 5, 45
constructed substitution, 4
    free variables of, 4

decr, 53
devar, 39
don't-care-element, 6

η-conversion, 53

foldl, 6
free, 53
free variable, 3
    new, 7
    representation, 3
functor-arguments form, 3
    top-level, 3

head_ap, 8
higher-order pattern, 6

idx, 7

l_basic, 57
leading lambda, 3
lift, 52
list
    comprehension, 6
    indexing scheme, 6
    set of, 6
lists, 6
lookup, 7

lproj
    version 4, 26
    version 5, 45
$lterm_{f'}$, 57
$lterm_b$, 3
$lterm_f$, 3

map, 6
maybe, 6
mk_bind, 4
mk_subst, 4

occ, 44
occurs-check, 44
otfη, 10

proj
    version 4, 25
    version 5, 45

red, 39
renaming, 5

substitution, 4
    β-approximation, 28
    binding, 4
    constructed, 4
    domain, 4
    equivalence, 5
    of bound variables, 52
    range, 4
    renaming, 5
    representation, 38
    restriction, 4

to_b', 58
to_f', 58
to_f, 58
tw, 10

unif
    version 1, 10
    version 2, 21
    version 3, 23
    version 4, 25
    version 5, 46
unification, 5
unification problem, 5